

Lab 1

Jeffrey Lam Ping Fong
116805235

19th January 2026

RIS602 NBB

Table of Contents

Checklist.....	3
Executive Summary.....	5
Chapter 1 Summary.....	6
1.1 Corporate Data Breaches.....	6
1.2 How Hackers Break In.....	6
1.3 Adversarial Attack Simulation: Penetration Testing.....	6
1.4 When a Penetration Test Is Least Effective.....	7
1.5 Executing a Network Penetration Test.....	7
1.6 Setting Up Your Lab Environment.....	8
1.7 Building Your Own Virtual Pentest Platform.....	8
Chapter 1 Figures.....	9
Chapter 2 Summary.....	12
2.1 Understanding Your Engagement Scope.....	12
2.2 Internet Control Message Protocol.....	12
2.3 Discovering Hosts with Nmap.....	13
2.4 Additional Host-Discovery Methods.....	15
Chapter 2 Figures.....	16
Exercise 2.1.....	18
Appendix A (Screenshots).....	19

Checklist

ID	What {Figure, Table, Section ...}	Where (Reference)	Completion (%)
1	Chapter 1 Summary	Page 6, Paragraph 1	100%
2	1.1 Corporate data breaches	Page 6, Paragraph 2	100%
3	1.2 How hackers break in	Page 6, Paragraph 3-5	100%
4	1.3 Adversarial attack simulation: Penetration testing	Page 6, Paragraph 6	100%
5	Figure 1.1	Page 9	100%
6	1.4 When a penetration test is least effective	Page 7, Paragraph 1-3	100%
7	1.5 Executing a network penetration test	Page 7, Paragraph 4-8	100%
8	Figure 1.2	Page 10	100%
9	Figure 1.3	Page 10	100%
10	Figure 1.4	Page 11	100%
11	Figure 1.5	Page 11	100%
12	1.6 Setting up your lab environment	Page 8, Paragraph 1	100%
13	1.7 Building your own virtual pentest platform	Page 8, Paragraph 2	100%
14	Chapter 2 Summary	Page 12, Paragraph 1-2	100%
15	Figure 2.1	Page 16	100%
16	2.1 Understanding your engagement scope	Page 12, Paragraph 3-4	100%
17	Figure 2.3	Page 15	100%
18	2.2 Internet Control Message Protocol	Page 12, Paragraph 5-7 Page 13, Paragraph 1	100%
19	Figure 2.4	Page 15	100%
20	2.3 Discovering hosts with Nmap	Page 13, Paragraph 2-5	100%

21	2.4 Additional host-discovery methods	Page 13, Paragraph 6-7	100%
22	Exercise 2.1	Page 16, Paragraph 1-3	100%

Executive Summary

This engagement focuses on understanding and applying a structured internal network penetration testing (INPT) methodology, emphasizing the importance of preparation, scope awareness, and disciplined information gathering. Chapter 1 establishes the context for penetration testing by highlighting the increasing complexity of modern enterprise networks and the imbalance between defenders and attackers. It introduces the four-phase INPT methodology, information gathering, focused penetration, post-exploitation and privilege escalation, and documentation and underscores the importance of conducting penetration tests at an appropriate maturity level to ensure meaningful and actionable results.

Chapter 2 builds on this foundation by detailing the practical execution of the information-gathering phase, with a specific focus on host discovery. Multiple techniques are examined, including ICMP-based discovery, scripted pingsweeps, Nmap host discovery scans, and alternative methods for environments where ICMP is restricted. The chapter emphasizes the value of accurate target identification, the use of structured input and output files, and performance tuning to efficiently discover live systems while remaining within the defined engagement scope.

Exercise 2.1 reinforces these concepts through hands-on application. A standardized directory structure was created on the penetration testing virtual machine to organize discovery activities, engagement scope, and results. Authorized IP ranges were documented in a `ranges.txt` file, and host discovery techniques were applied to identify live systems. The resulting `targets.txt` file provides a validated list of systems to be assessed throughout subsequent phases of the engagement.

Chapter 1 Summary

Chapter 1 establishes the foundational context for internal network penetration testing (INPT) by explaining why modern organizations are highly susceptible to compromise, how attackers differ fundamentally from defenders, and why penetration testing is a necessary control within an organization's overall security program. The chapter introduces the methodology, scope, and mindset required to conduct a realistic internal penetration test and frames the remainder of the book as a structured, hands-on guide for executing such engagements.

1.1 Corporate Data Breaches

This section introduces the prevalence and impact of corporate data breaches in modern enterprises. Organizations frequently experience breaches that expose sensitive customer and operational data, often without immediately understanding how the breach occurred or the full extent of the damage. Public disclosures commonly emphasize uncertainty, highlighting weaknesses in visibility, monitoring, and incident response. The section establishes the urgency for proactive security testing rather than relying solely on reactive measures after a breach has already occurred.

1.2 How Hackers Break In

This section explains why breaches occur so frequently by comparing the roles and responsibilities of defenders and attackers. Enterprise environments contain large and complex attack surfaces due to the sheer number of connected devices and services. Attackers benefit from this complexity, as they need to find only one exploitable weakness to gain access, whereas defenders must secure all systems continuously.

Defenders are responsible for identifying every system on the network, enforcing secure configurations, applying patches, and maintaining business continuity. Even small failures, such as a missed patch or misconfigured service can introduce significant risk. The operational burden placed on defenders makes perfect security difficult to achieve in practice.

Attackers operate with fewer constraints and focus on identifying low-effort, high-impact weaknesses. Common attack vectors include weak or default credentials, insecure configurations, and unpatched vulnerabilities. This asymmetry explains why attackers often succeed even when organizations have invested in security controls.

1.3 Adversarial Attack Simulation: Penetration Testing

This section introduces penetration testing as a controlled, authorized simulation of real-world attacks. Penetration tests are designed to identify security weaknesses by actively attempting to exploit them, rather than relying solely on theoretical risk assessments or automated scans. The goal is to provide organizations with realistic insight into their exposure and actionable guidance for remediation.

The Internal Network Penetration Test (INPT) is presented as the most common form of enterprise penetration testing. It assumes an attacker has already gained internal network access and evaluates how far they can progress. The workflow is structured into four phases: information gathering, focused penetration, post-exploitation and privilege escalation, and documentation.

1.4 When a Penetration Test Is Least Effective

This section discusses situations where penetration testing may provide limited value. Organizations with immature security programs often contain obvious weaknesses that could be identified without a full adversarial simulation. In such cases, foundational security improvements may be more effective than an advanced penetration test.

Low-hanging fruit refers to easily exploitable weaknesses such as default passwords, shared credentials, missing patches, and excessive administrative privileges. These issues are widespread, highly dangerous, and frequently exploited in real-world breaches. Addressing these weaknesses should be a priority before conducting advanced penetration testing.

Organizations can assess their readiness for a penetration test by evaluating whether they maintain an accurate inventory of assets, apply patches consistently, perform routine vulnerability scanning, enforce strong authentication controls, and deploy multi-factor authentication. Difficulty in confidently addressing these areas suggests elevated exposure to security risks.

1.5 Executing a Network Penetration Test

This section transitions from theory to execution, emphasizing that penetration testing is both a technical and methodological discipline. Successful tests require planning, authorization, discipline, and clear objectives. The penetration tester must ultimately shift from attacker to consultant by clearly communicating findings and remediation steps.

The information-gathering phase focuses on mapping the network, identifying hosts, and enumerating services. The objective is to gain as complete an understanding of the environment as possible without insider knowledge, forming the foundation for later attack phases.

In phase 2, which is about focused penetration, the tester exploits identified weaknesses to gain unauthorized access to systems. This often involves remote code execution, abuse of insecure services, or exploitation of missing patches, resulting in initial footholds within the network.

In Phase 3, post-exploitation focuses on expanding access beyond initial targets. The tester harvests credentials, escalates privileges, and moves laterally to previously inaccessible systems. This phase demonstrates how a single compromised host can lead to widespread network compromise.

The final phase emphasizes evidence collection and reporting. A penetration test is incomplete without clear documentation detailing how systems were compromised and how vulnerabilities can be remediated. This phase delivers the primary value of the engagement to the organization.

1.6 Setting Up Your Lab Environment

Network penetration testing is best learned through hands-on practice rather than theory alone. Because many learners do not have access to a real enterprise network or authorization to perform penetration testing activities, the book introduces the Capsulecorp Pentest lab as a practical alternative. This lab environment provides a realistic, enterprise-style virtual network that allows readers to safely apply the Internal Network Penetration Testing (INPT) methodology throughout the book. The Capsulecorp Pentest project is built using virtualization and automation technologies and includes multiple vulnerable systems as well as a preconfigured Ubuntu Linux attacking machine. By using this lab, students can gain experience performing discovery, exploitation, privilege escalation, and documentation in a controlled and repeatable environment.

1.7 Building Your Own Virtual Pentest Platform

The book also acknowledges that some readers may prefer to build their own penetration testing platform rather than rely on a prebuilt lab. When creating a custom environment, careful consideration should be given to the operating system and tooling used. Linux is strongly recommended as the preferred platform for penetration testing due to its widespread adoption in the security community, strong scripting and automation capabilities, and compatibility with most open-source security tools. Ubuntu is highlighted as a practical choice because of its stability, extensive documentation, and large support community. While specialized pentesting distributions such as Kali Linux offer convenience, they can introduce unnecessary complexity and unused tools. Building a lean, custom platform encourages deeper understanding of the tools in use and improves transparency, which is especially important when explaining testing activities to clients or stakeholders.

Chapter 1 Figures

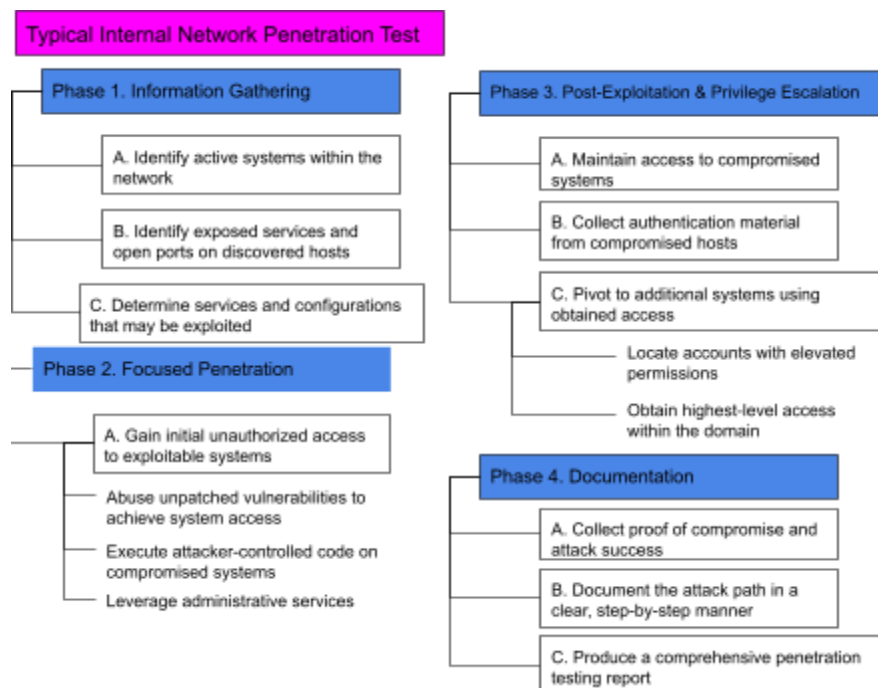


Figure 1.1 The four phases of a network penetration test

A network penetration test follows a four-phase approach: information gathering, focused penetration, post-exploitation and privilege escalation, and documentation. The tester first maps the network and identifies vulnerable targets, then exploits weaknesses to gain unauthorized access. After initial compromise, access is expanded through credential harvesting, lateral movement, and privilege escalation. The engagement concludes with detailed documentation, where evidence is collected and a final report is produced outlining the attack path and recommended remediation steps.

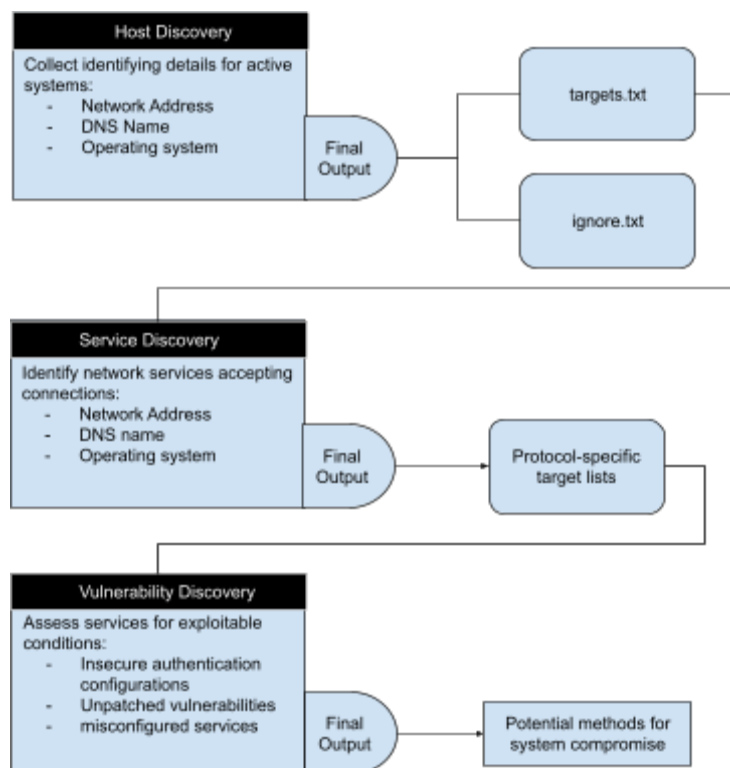


Figure 1.2 The information-gathering phase

Figure 1.2 illustrates the information-gathering phase, showing how host discovery, service enumeration, and vulnerability identification are performed to produce a list of viable targets and potential attack vectors.

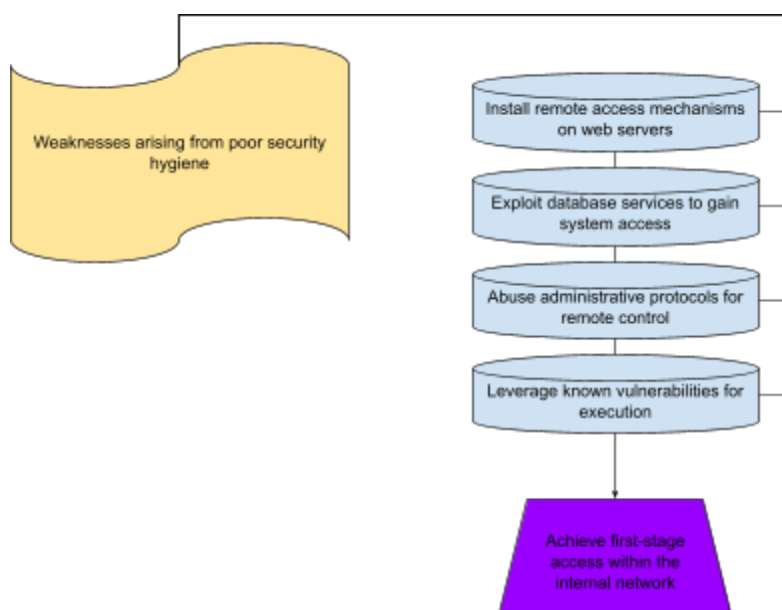


Figure 1.3 The focused penetration phase

Figure 1.3 represents the focused penetration phase, where identified vulnerabilities are exploited to gain initial unauthorized access to systems within the network.

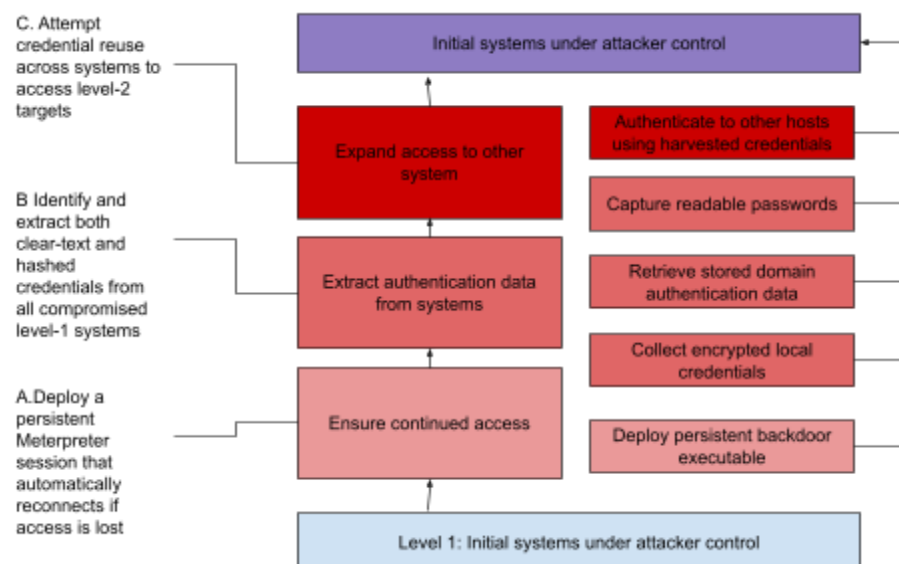


Figure 1.4 The privilege escalation phase

Figure 1.4 depicts the post-exploitation and privilege escalation phase, highlighting how compromised systems are used to harvest credentials, move laterally, and escalate access to higher-privilege targets.

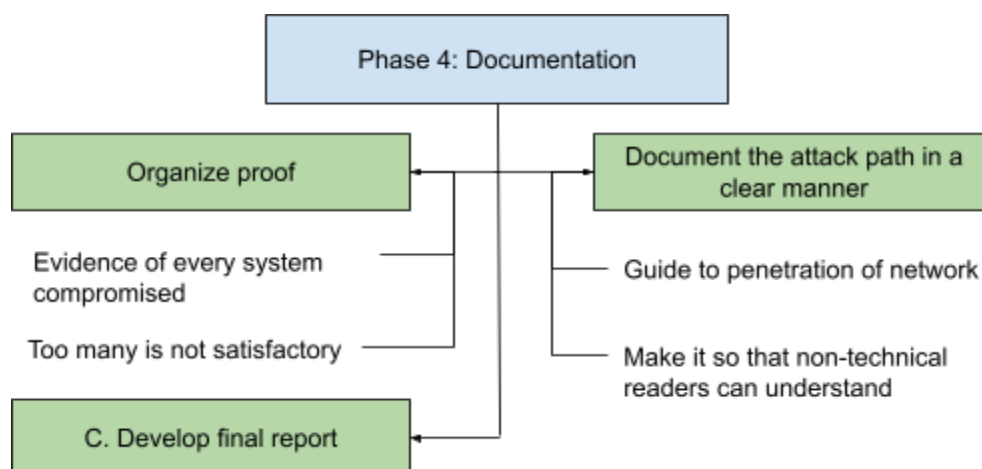


Figure 1.5 The documentation phase

Figure 1.5 outlines the documentation phase, emphasizing evidence collection and the creation of a final report that clearly explains the attack path and provides remediation recommendations.

Chapter 2 Summary

Chapter 2 focuses on the information-gathering phase of an internal network penetration test, with particular emphasis on host discovery. The chapter outlines how penetration testers identify live systems within a defined engagement scope and explains why accurate target identification is critical for effective and responsible testing. It introduces the use of ICMP-based techniques, manual and scripted ping sweeps, and Nmap host discovery scans as primary methods for detecting active hosts on a network.

In addition to basic discovery, the chapter addresses common limitations of ICMP scanning and presents alternative approaches, such as scanning common remote management ports, to identify systems that do not respond to echo requests. Performance optimization techniques for Nmap are also discussed, highlighting how scan efficiency can be significantly improved on larger networks.

2.1 Understanding Your Engagement Scope

The information-gathering phase begins by confirming the engagement scope and ensuring all discovery activity stays within authorized boundaries. Host discovery is intended to identify as many live systems as possible within the approved IP ranges and produce two primary outputs: `targets.txt` (approved systems to test) and `ignore.txt` (systems explicitly excluded). The scope is typically defined as white-box (client provides specific IPs), black-box (no information is provided), or grey-box (client provides IP ranges to sweep). Even in white-box engagements, performing independent discovery can provide value because organizations often have unmanaged or unknown hosts that are not documented.

In the Capsulecorp example, the client provides a small Class C range (10.0.10.0/24). The chapter emphasizes creating a methodical directory structure and storing the scoped ranges in a file so tools like Nmap can consistently reference scope throughout the engagement. The ranges file (one range per line) is stored as:

discovery/ranges.txt → contains entries like 10.0.10.0/24

2.2 Internet Control Message Protocol

ICMP-based discovery is introduced as a simple method for identifying live hosts by sending ICMP echo requests and receiving replies from reachable systems. The chapter demonstrates verifying basic functionality with `ping 127.0.0.1 -c 1`, which sends a single request to the local system. To determine the network range for discovery, the `ifconfig` command is used to identify the pentest VM's IP address and subnet mask.

Because manually pinging each address in a /24 network is inefficient, the chapter presents a bash-based pingsweep that sends parallel ping requests across the subnet and writes the results to a file. The output is then filtered to display only responding hosts, and the IP addresses are extracted into a clean `targets.txt` file for reuse in later testing phases.

The chapter concludes by noting the limitations of ping-based discovery. ICMP scans can be noisy, do not scale well across large or multiple ranges, and may fail when hosts block ICMP traffic. These shortcomings highlight the need for more robust and scalable host discovery techniques, such as those provided by Nmap.

2.3 Discovering Hosts with Nmap

Nmap is presented as the preferred host discovery tool due to its scalability, support for input files, and ability to generate structured output suitable for later analysis. The chapter introduces an ICMP-based discovery scan using `sudo nmap -sn -iL discovery/ranges.txt -oA discovery/hosts/pingsweep -PE`. This command performs a ping-only scan across all IP ranges listed in `ranges.txt` and saves the results in multiple output formats. The `-sn` flag disables port scanning, `-iL` reads target ranges from a file, `-oA` produces standard Nmap output files, and `-PE` specifies ICMP echo request probes.

After the scan completes, the greppable output file is used to identify live hosts. By searching for entries marked as “Up,” the tester can quickly list systems that responded to discovery probes. These results are then processed to extract only IP addresses and store them in a `targets.txt` file, which becomes the authoritative list of hosts for subsequent testing phases.

When ICMP echo requests are blocked, the chapter recommends detecting hosts by scanning a small set of common remote management and service ports. This approach assumes that most enterprise systems expose at least one management interface. The scan is performed without host discovery, disables DNS resolution for efficiency, and targets only specific ports to limit scan time. Although this method is slower than ICMP-based discovery, it remains effective in environments where ping responses are suppressed.

To improve performance, the chapter demonstrates tuning Nmap’s concurrency and packet rate. Increasing the minimum host group size allows more systems to be scanned simultaneously, while raising the minimum packet rate significantly reduces overall scan time. Finally, the greppable output is filtered to identify hosts with at least one open port, producing a refined list of active targets that would otherwise be missed by

2.4 Additional Host-Discovery Methods

The chapter highlights several alternative discovery techniques that may be useful depending on scope, stealth requirements, or network size. DNS brute-forcing is described conceptually (resolving common hostnames to discover systems and subnets). Passive discovery through packet capture (Wireshark/tcpdump) is mentioned as a stealthier approach because it observes traffic rather than actively probing, but it requires stronger protocol familiarity and may take longer.

For very large address spaces (e.g., 10.0.0.0/8), a “subnet hunting” shortcut is introduced that probes likely gateway-style addresses (often .1) across many ranges to quickly find which subnets are active:

```
sudo nmap -sn 10.0-255.0-255.1 -PE --min-hostgroup 10000 --min-rate 10000
```

Purpose: Pings only the .1 address across many /24 subnets to identify which ranges are in use.

Value: Rapidly narrows down active subnets for follow-on scanning.

Chapter 2 Figures

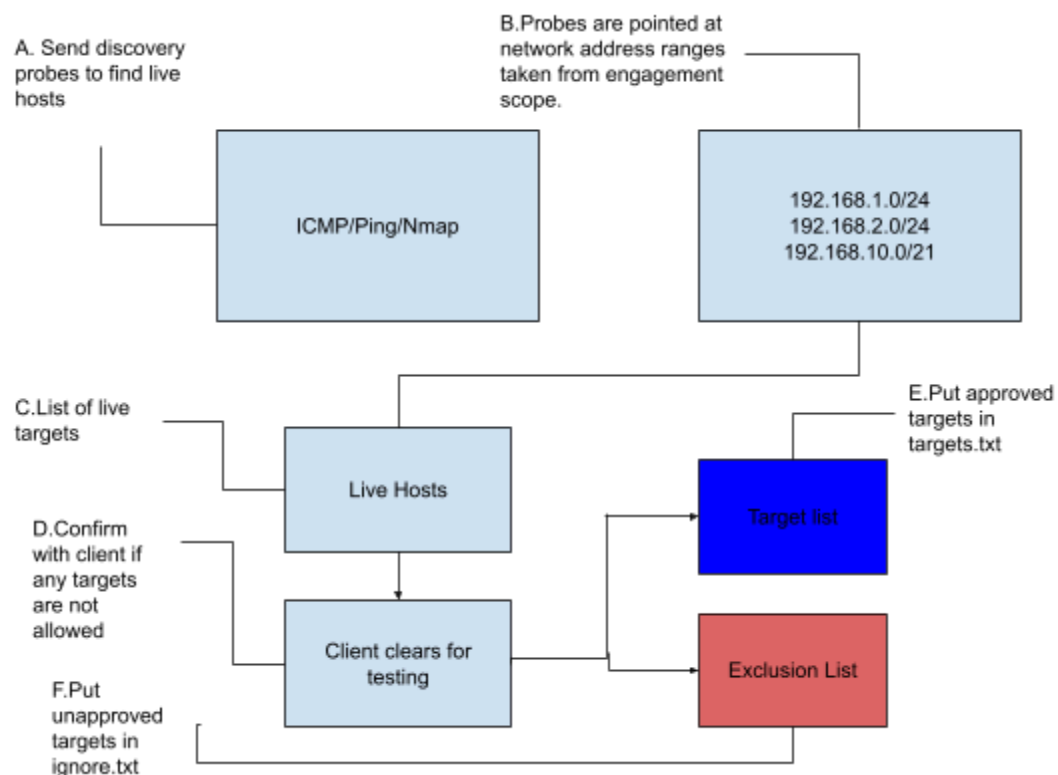


Figure 2.1 The information-gathering phase workflow

This figure shows how discovery scans are used to identify live hosts within the approved engagement scope. Discovered systems are reviewed with the client, with authorized targets added to the target list and restricted systems placed in an exclusion list to ensure scoped and compliant testing.

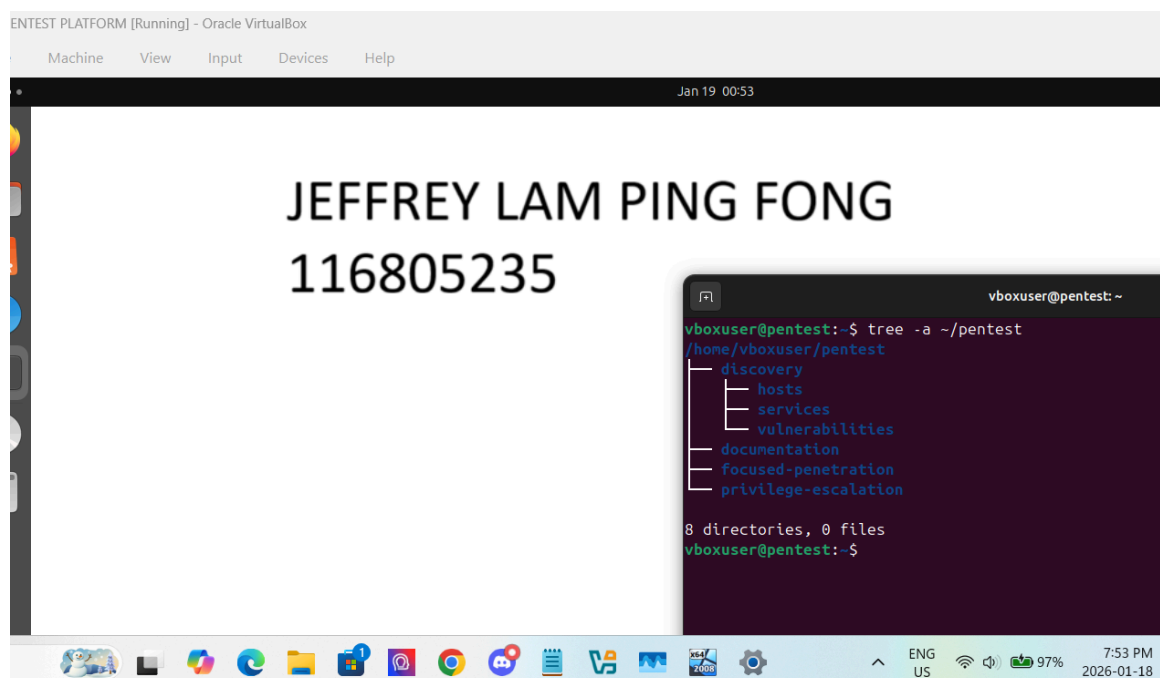


Figure 2.3 Directory structure

This figure illustrates a structured folder layout used to organize discovery data, documentation, and exploitation activities. Separating files by phase improves clarity, traceability, and reporting efficiency throughout the engagement.

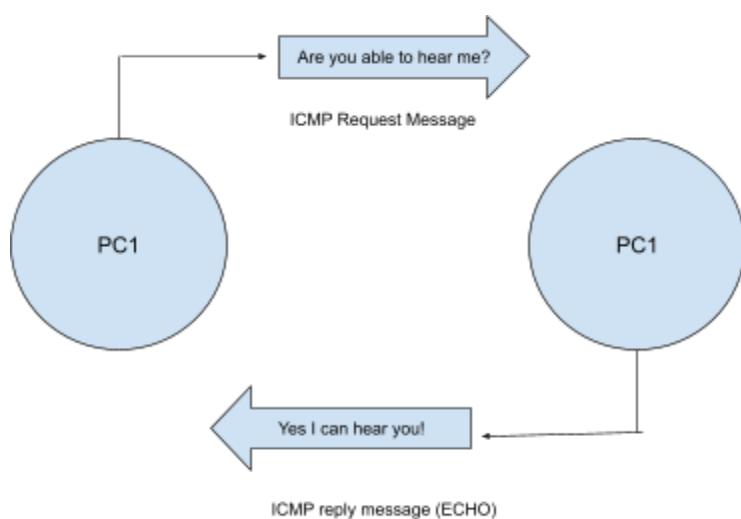
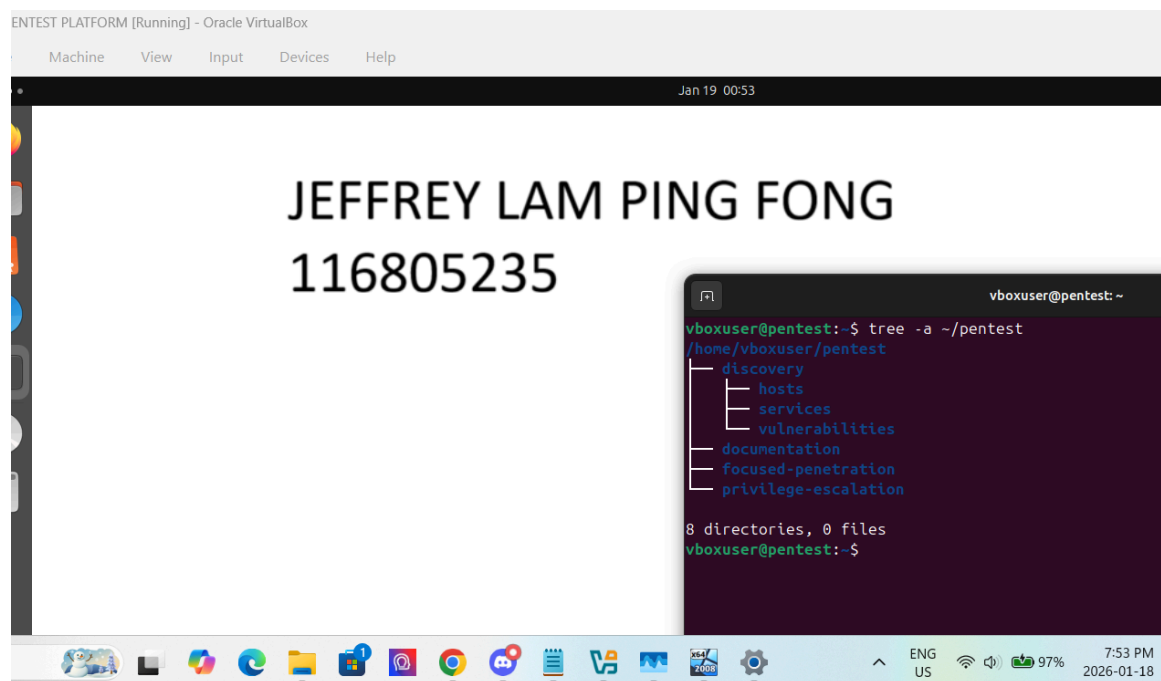


Figure 2.4 Typical ICMP packet exchange

This figure demonstrates a standard ICMP ping process, where one host sends an echo request and the receiving host responds with an echo reply. This exchange is commonly used to determine whether a system is online and reachable during host discovery.

Exercise 2.1

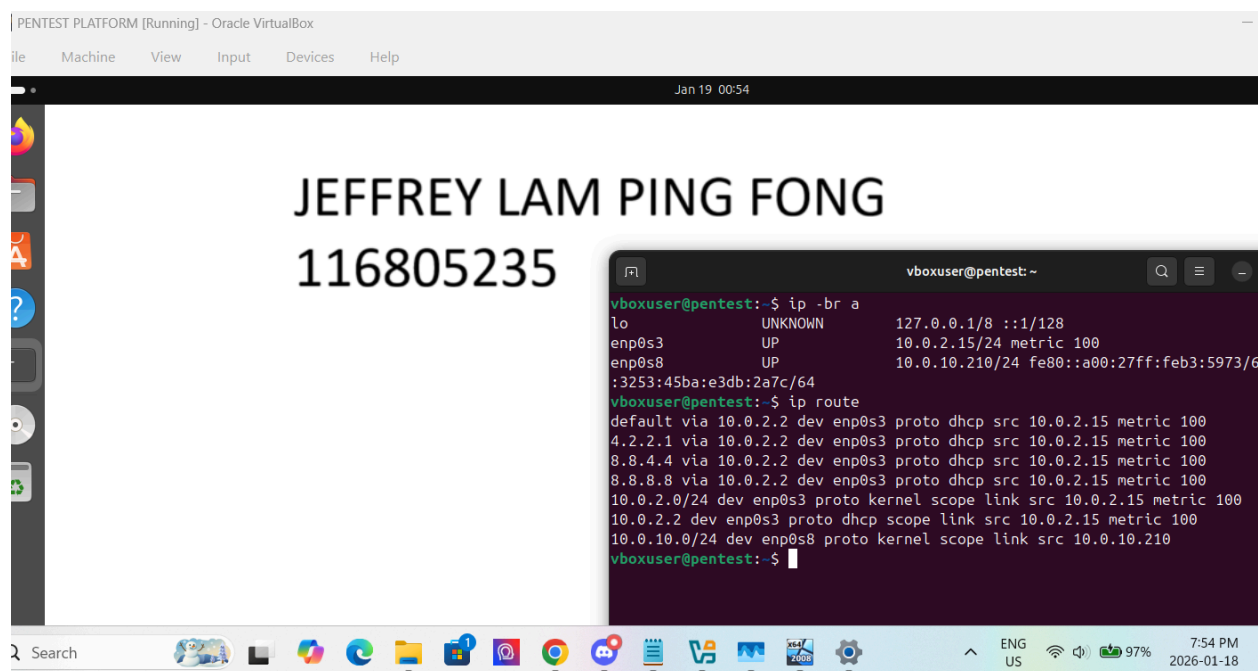
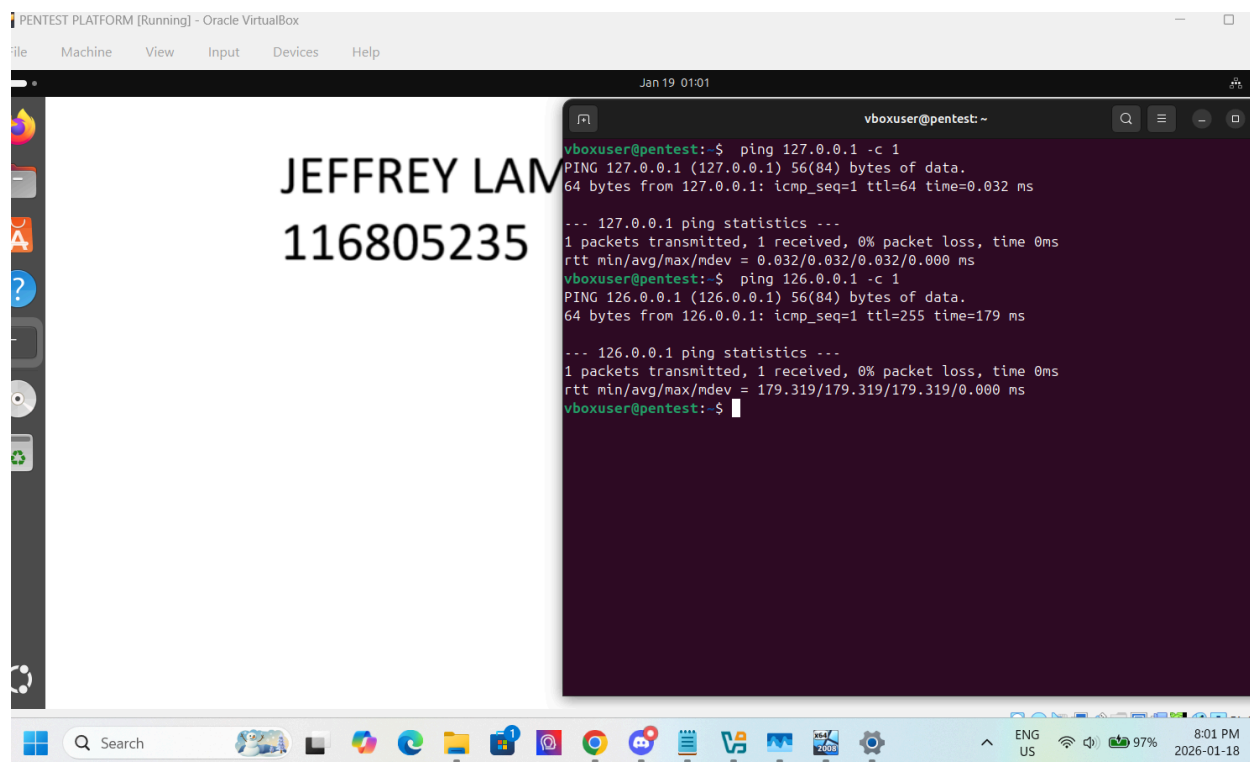


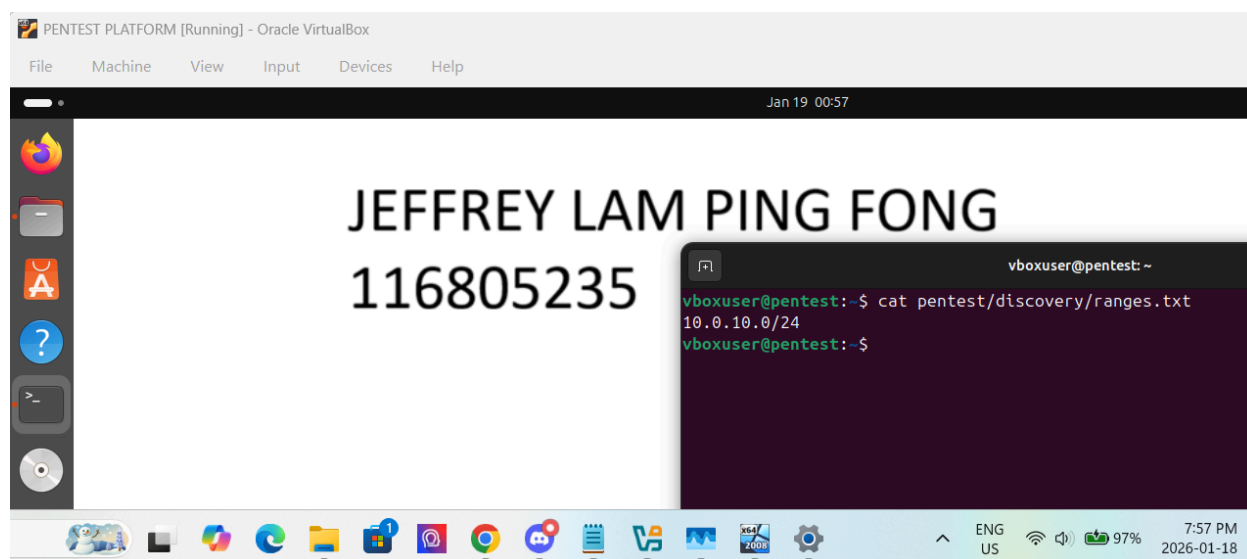
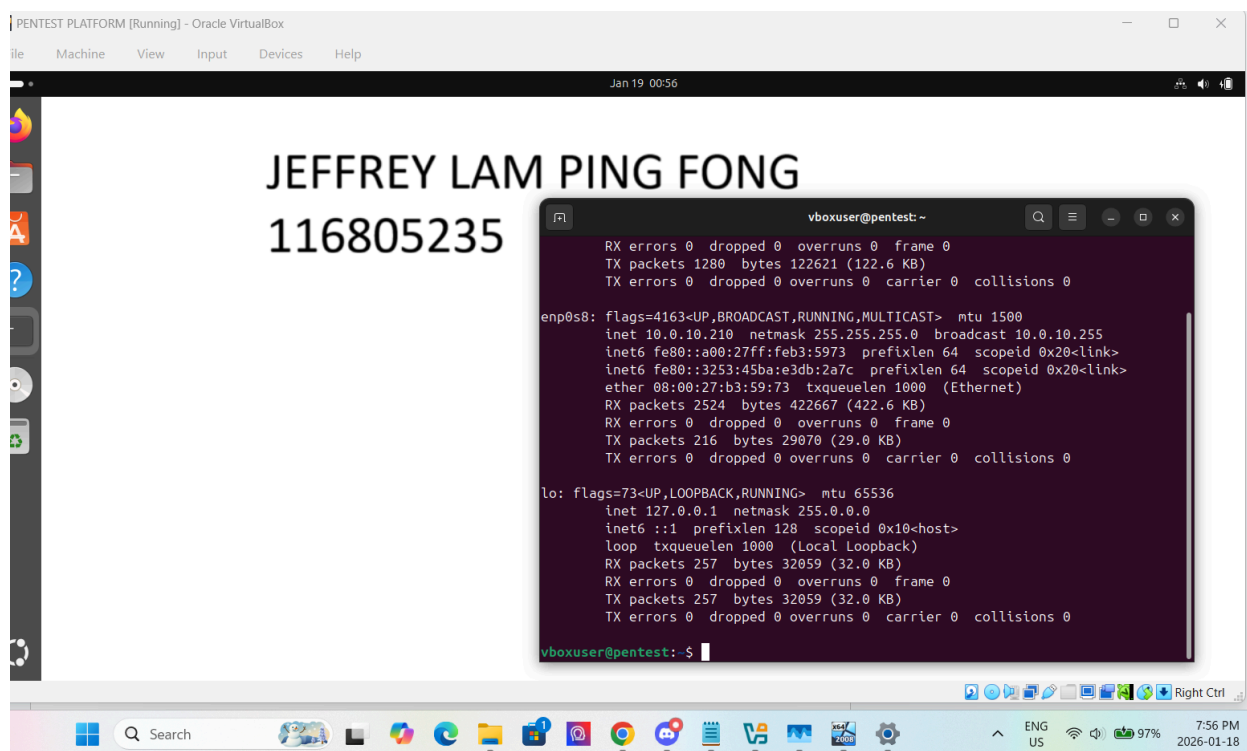
In this exercise, a structured working directory was created on the penetration testing virtual machine to organize all files related to the engagement. Within this directory, a discovery folder was established to store scope and reconnaissance data. The authorized IP address range(s) for the engagement were recorded in a file named `ranges.txt`.

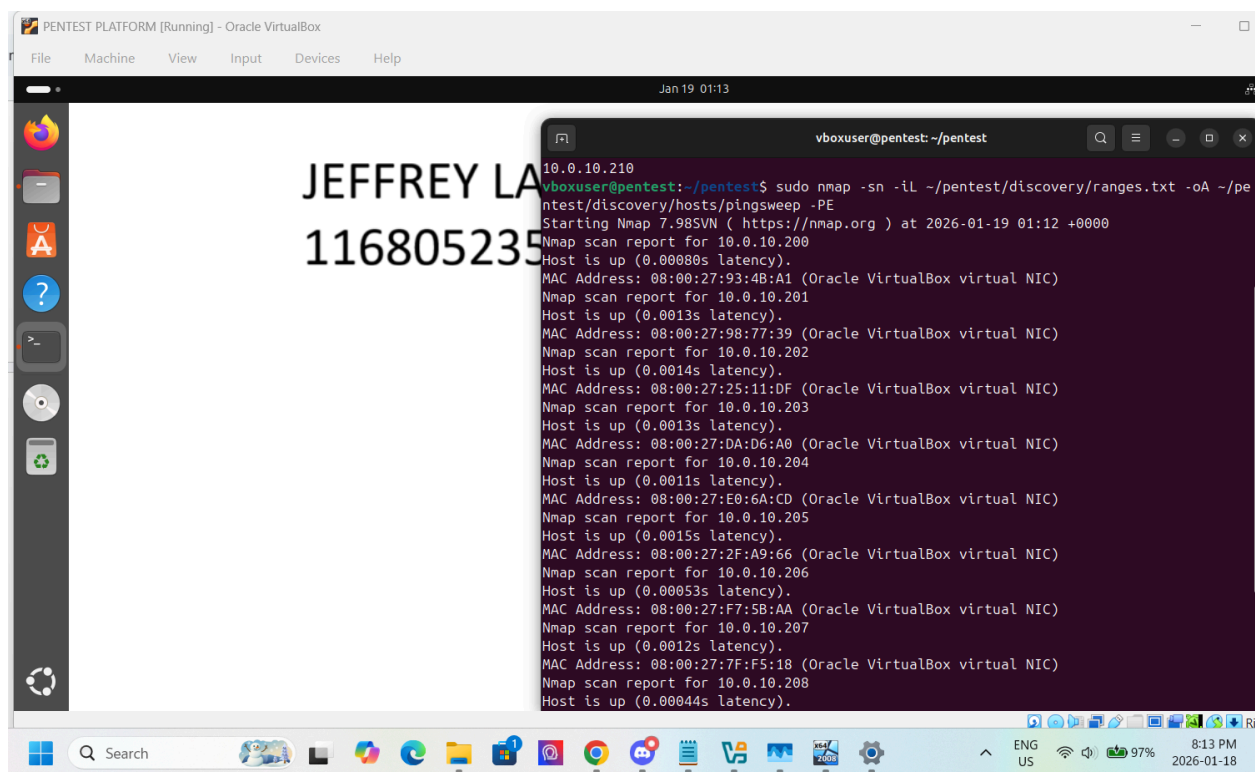
Using the host-discovery techniques covered in the chapter—primarily Nmap-based discovery scans—all live systems within the defined scope were identified. The IP addresses of hosts confirmed to be online were then extracted and saved into a `targets.txt` file. This file serves as the authoritative list of systems to be tested throughout the remainder of the engagement.

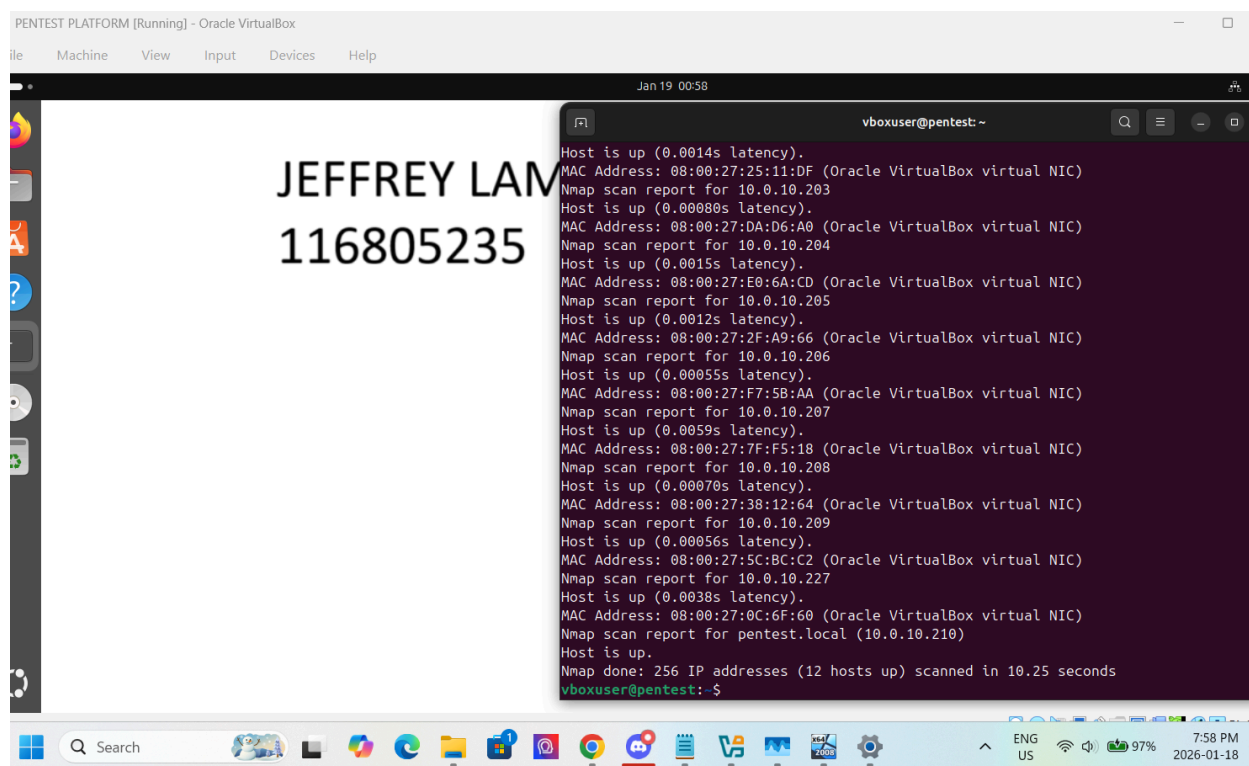
Upon completion of the exercise, the resulting directory structure reflected a clean and methodical organization of discovery data, ensuring that scope, targets, and future testing activities could be conducted in a controlled and repeatable manner.

Appendix A (Screenshots)









PENTEST PLATFORM [Running] - Oracle VirtualBox

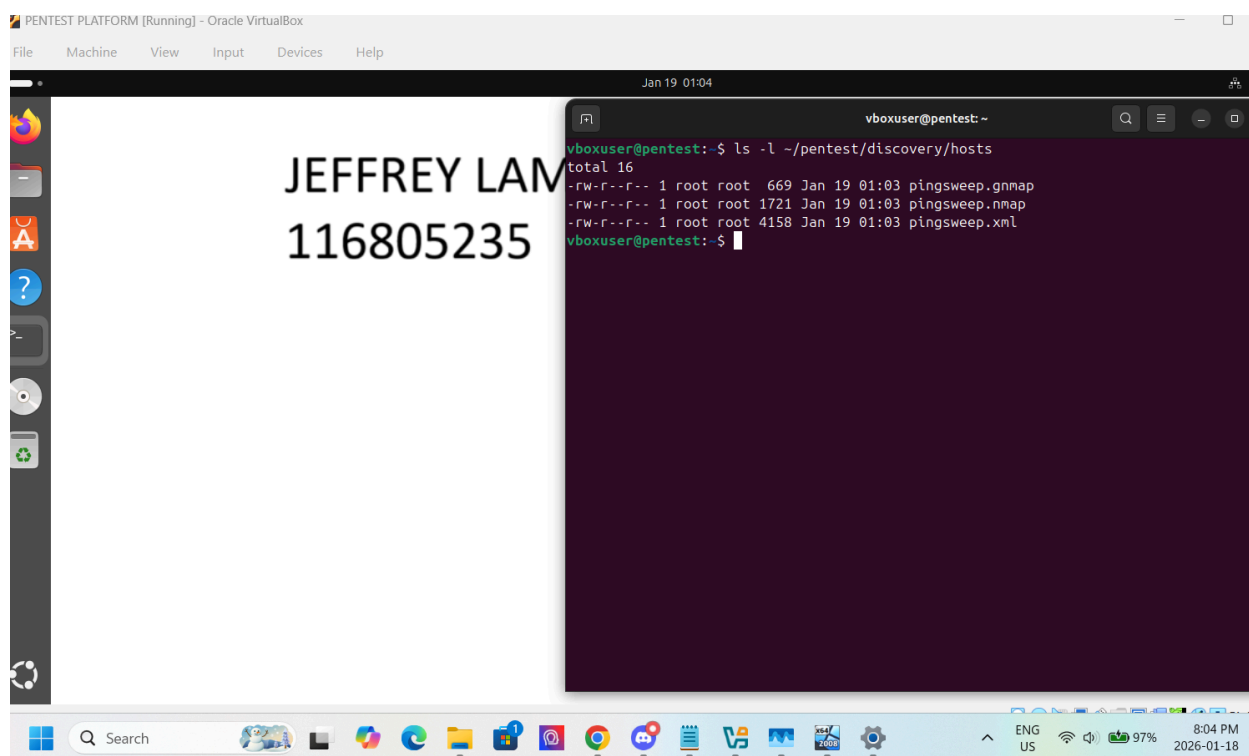
File Machine View Input Devices Help

Jan 19 00:58

vboxuser@pentest: ~

```
Host is up (0.0014s latency).
MAC Address: 08:00:27:25:11:DF (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.203
Host is up (0.00080s latency).
MAC Address: 08:00:27:DA:D6:A0 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.204
Host is up (0.0015s latency).
MAC Address: 08:00:27:E0:6A:CD (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.205
Host is up (0.0012s latency).
MAC Address: 08:00:27:2F:A9:66 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.206
Host is up (0.00055s latency).
MAC Address: 08:00:27:F7:5B:AA (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.207
Host is up (0.0059s latency).
MAC Address: 08:00:27:7F:F5:18 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.208
Host is up (0.00070s latency).
MAC Address: 08:00:27:38:12:64 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.209
Host is up (0.00056s latency).
MAC Address: 08:00:27:5C:BC:C2 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.10.227
Host is up (0.0038s latency).
MAC Address: 08:00:27:0C:6F:60 (Oracle VirtualBox virtual NIC)
Nmap scan report for pentest.local (10.0.10.210)
Host is up.
Nmap done: 256 IP addresses (12 hosts up) scanned in 10.25 seconds
vboxuser@pentest:~$
```

Windows taskbar: Search, icons for various applications, system tray showing ENG US, 97% battery, and time 7:58 PM 2026-01-18.



PENTEST PLATFORM [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Jan 19 01:04

vboxuser@pentest: ~

```
vboxuser@pentest:~$ ls -l ~/pentest/discovery/hosts
total 16
-rw-r--r-- 1 root root 669 Jan 19 01:03 pingsweep.gnmap
-rw-r--r-- 1 root root 1721 Jan 19 01:03 pingsweep.nmap
-rw-r--r-- 1 root root 4158 Jan 19 01:03 pingsweep.xml
vboxuser@pentest:~$
```

Windows taskbar: Search, icons for various applications, system tray showing ENG US, 97% battery, and time 8:04 PM 2026-01-18.

PENTEST PLATFORM [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Jan 19 01:04

vboxuser@pentest: ~/pentest/discovery/hosts

```
vboxuser@pentest:~/pentest/discovery/hosts$ cd ~/pentest/discovery/hosts
grep "Up" pingsweep.gnmap
grep "Up" pingsweep.gnmap | cut -d " " -f2 > targets.txt
cat targets.txt
wc -l targets.txt
Host: 10.0.10.200 ( ) Status: Up
Host: 10.0.10.201 ( ) Status: Up
Host: 10.0.10.202 ( ) Status: Up
Host: 10.0.10.203 ( ) Status: Up
Host: 10.0.10.204 ( ) Status: Up
Host: 10.0.10.205 ( ) Status: Up
Host: 10.0.10.206 ( ) Status: Up
Host: 10.0.10.207 ( ) Status: Up
Host: 10.0.10.208 ( ) Status: Up
Host: 10.0.10.209 ( ) Status: Up
Host: 10.0.10.227 ( ) Status: Up
Host: 10.0.10.210 (pentest.local) Status: Up
10.0.10.200
10.0.10.201
10.0.10.202
10.0.10.203
10.0.10.204
10.0.10.205
10.0.10.206
10.0.10.207
10.0.10.208
10.0.10.209
10.0.10.227
10.0.10.210
12 targets.txt
```

Show hidden icons

ENG US 97% 8:04 PM 2026-01-18

PENTEST PLATFORM [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Jan 19 01:07

vboxuser@pentest: ~/pentest/discovery/hosts

```
vboxuser@pentest:~/pentest/discovery/hosts$ sudo nmap -Pn -n -p 22,80,443,2222,3389 -iL ~/pentest/discovery/ranges.txt -oA ~/pentest/discovery/hosts/rmisweep
Starting Nmap 7.98SVN ( https://nmap.org ) at 2026-01-19 01:06 +0000
Nmap scan report for 10.0.10.200
Host is up (0.0013s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    closed http
443/tcp    closed https
2222/tcp   closed EtherNetIP-1
3389/tcp   closed ms-wbt-server
MAC Address: 08:00:27:93:4B:A1 (Oracle VirtualBox virtual NIC)

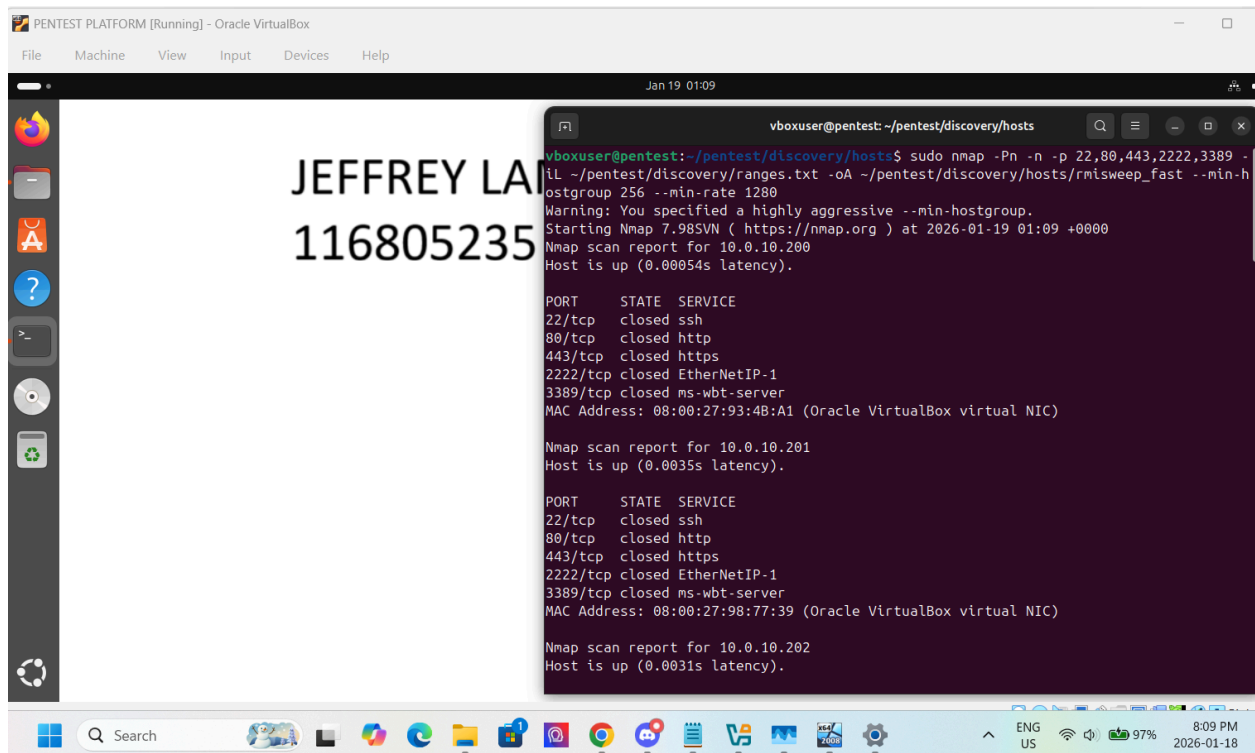
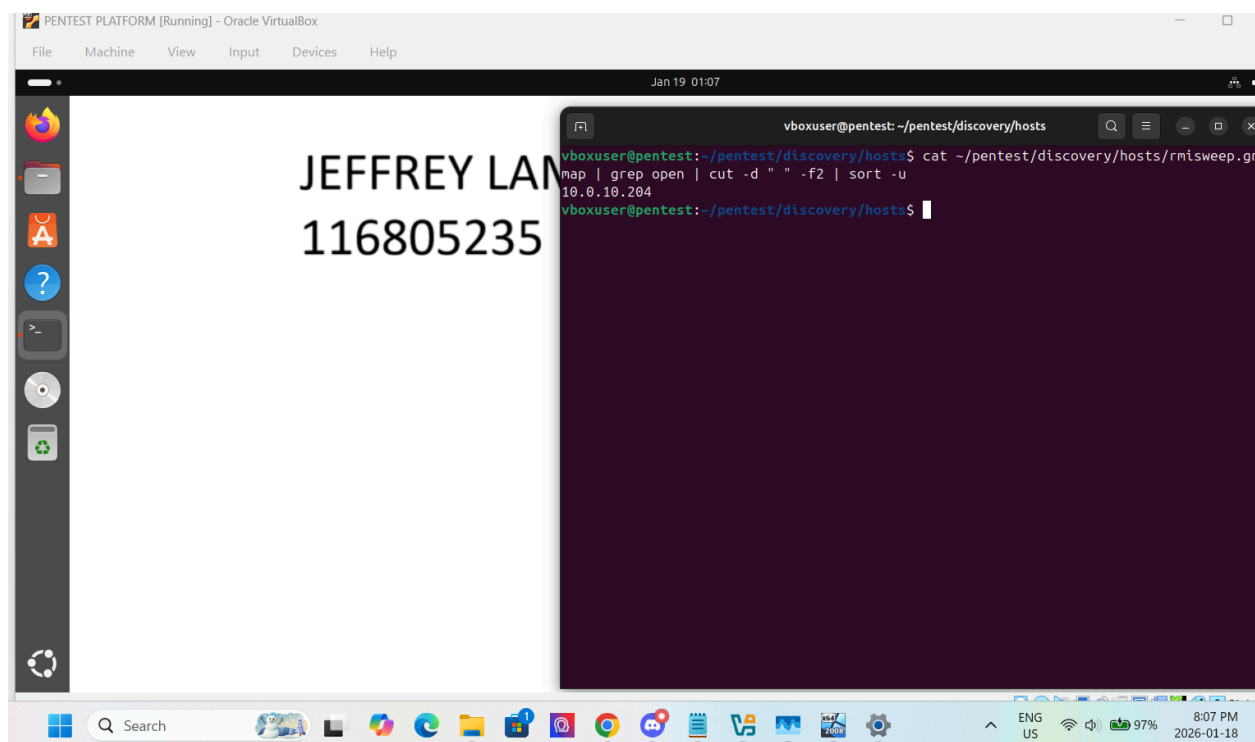
Nmap scan report for 10.0.10.201
Host is up (0.0016s latency).

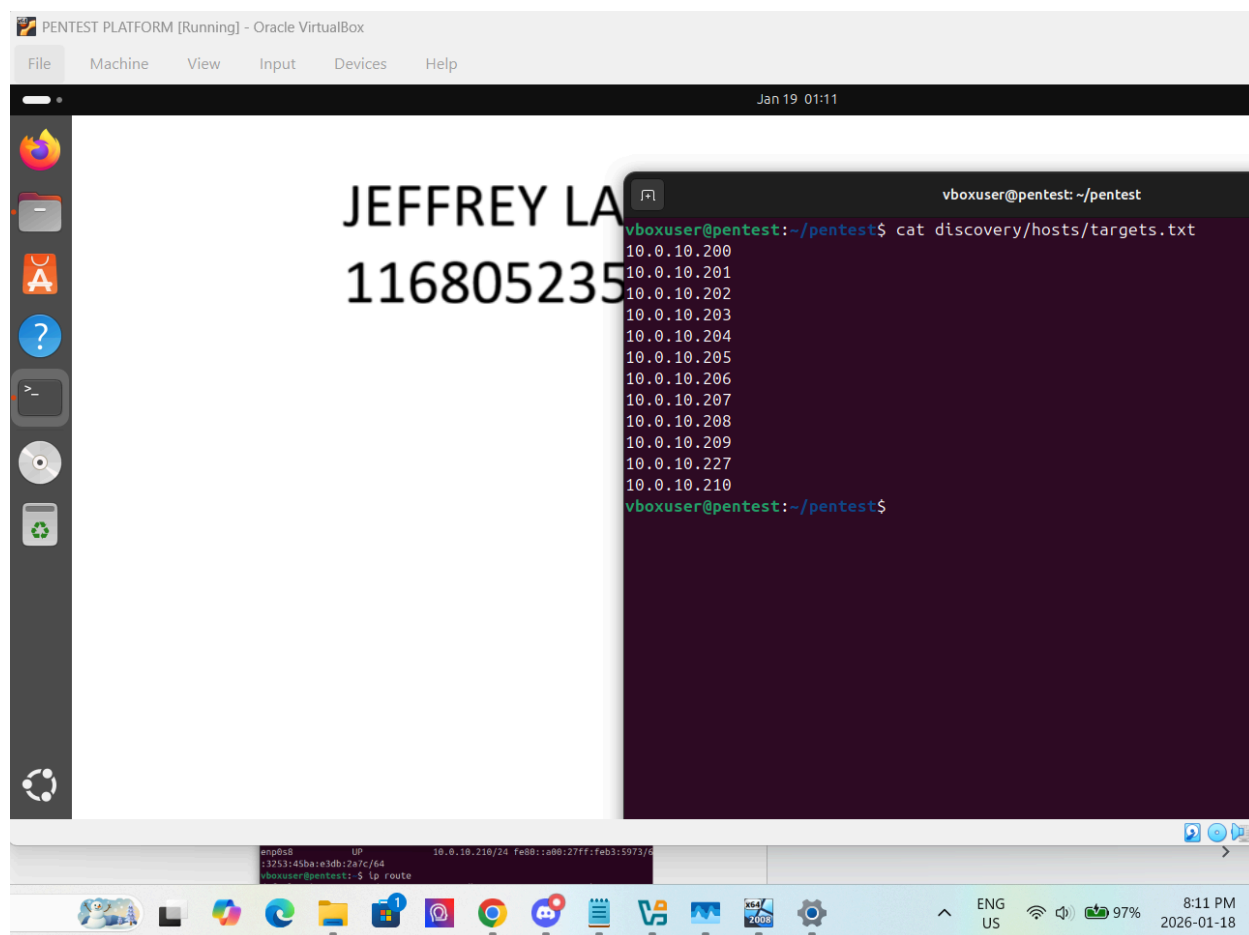
PORT      STATE SERVICE
22/tcp    closed ssh
80/tcp    closed http
443/tcp    closed https
2222/tcp   closed EtherNetIP-1
3389/tcp   closed ms-wbt-server
MAC Address: 08:00:27:98:77:39 (Oracle VirtualBox virtual NIC)

Nmap scan report for 10.0.10.202
Host is up (0.0016s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
```

ENG US 97% 8:07 PM 2026-01-18





Lab 2

Jeffrey Lam Ping Fong
116805235

3rd February 2026

RIS602 NBB

Table of Contents

Executive Summary.....	5
Chapter 3 Summary.....	6
3.1 Network services from an attacker's perspective.....	6
3.2 Port scanning with Nmap.....	7
3.3 Parsing XML output with Ruby.....	7
Chapter 3 Figures.....	8
Chapter 4 Summary.....	9
4.1 Understanding vulnerability discovery.....	9
4.2 Discovering patching vulnerabilities.....	10
4.3 Discovering authentication vulnerabilities.....	10
4.4 Discovering configuration vulnerabilities.....	10
Chapter 4 Figures.....	11
Exercise 3.1.....	16
Exercise 4.1.....	17
Exercise 4.2.....	18
Exercise 4.3.....	19
References.....	20
Appendix A.....	20

Table of Figures

Figure 3.1 Sub-phase B: Service discovery workflow.....	8
Figure 3.2 Illustration of a typical network service request and response.....	9
Figure 4.1 The vulnerability-discovery sub-phase workflow.....	11
Figure 4.2 Searching the public exploit database for “Apache Tomcat 7”.....	12
Figure 4.3 Brute-force password guessing.....	13
Figure 4.4 webshot.....	14
Figure 4.5.....	14
Figure 4.6.....	15
Figure 4.7.....	15

Checklist

ID	What {Figure, Table, Section ...}	Where (Reference)	Completion (%)
1	Chapter 3 Summary	Page 5, Paragraph 1-3	100%
2	3.1 Network services from an attacker's perspective	Page 5, Paragraph 4	100%
3	Figure 3.1	Page 6	100%
4	Figure 3.2	Page 7	100%
5	3.2 Port scanning with Nmap	Page 5, Paragraph 5	100%
6	3.3 Parsing XML output with Ruby	Page 6, Paragraph 1	100%
7	Exercise 3.1	Page 14	100%
8	Chapter 4 Summary	Page 8, Paragraph 1	100%
9	4.1 Understanding vulnerability discovery	Page 8, Paragraph 2	100%
10	Figure 4.1	Page 10	100%
11	4.2 Discovering patching vulnerabilities	Page 9, Paragraph 1	100%
12	Figure 4.2	Page 11	100%
13	Exercise 4.1	Page 16	100%
14	4.3 Discovering authentication vulnerabilities	Page 9, Paragraph 2	100%
15	Figure 4.3	Page 12	100%
16	Exercise 4.2	Page 17	100%
17	Exercise 4.3	Page 18	100%
18	4.4 Discovering configuration	Page 9, Paragraph	100%

	vulnerabilities	3	
19	Figure 4.4	Page 13	100%
20	Figure 4.5	Page 13	100%
21	Figure 4.6	Page 14	100%
22	Figure 4.7	Page 14	100%

Executive Summary

Chapters 3 and 4 document the transition from identifying network exposure to validating real, exploitable weaknesses within the engagement scope. Together, these chapters demonstrate how systematic service discovery and structured vulnerability discovery convert a broad set of live hosts into a prioritized list of confirmed attack vectors suitable for focused penetration testing.

Chapter 3 establishes visibility into the target environment by identifying active network services from an attacker's perspective. Using controlled Nmap scans and structured output parsing, open ports and listening services are enumerated across all in-scope hosts. The resulting protocol-specific target lists (e.g., Windows/SMB, databases, web services, remote administration interfaces) provide a clean, organized foundation for downstream analysis. Exercise 3.1 reinforces disciplined methodology by requiring consistent data handling, structured file organization, and validation of discovery results before proceeding.

Chapter 4 builds directly on this foundation by assessing which identified services are vulnerable in practice. Rather than relying solely on vulnerability databases or automated scanners, the methodology focuses on attacker-relevant weaknesses: missing patches, weak or default credentials, and insecure service configurations. Vulnerability discovery is performed in a controlled, evidence-driven manner using targeted tools aligned to each protocol. Exercise 4.1 emphasizes correlating discovered software versions with public exploit intelligence and validating high-impact patching weaknesses, ensuring findings are both accurate and actionable.

Across both chapters, the methodology emphasizes efficiency, realism, and risk awareness. Findings are documented without premature exploitation, enabling informed decision-making during focused penetration while minimizing unnecessary risk to the environment. The combined outcome is a high-confidence vulnerability set mapped to

specific hosts and services, providing clear justification for attack path selection and remediation prioritization in subsequent phases of the engagement.

Chapter 3 Summary

Chapter 3 covers the service discovery sub-phase of the information-gathering process in an internal network penetration test. The chapter explains how network services represent an organization's attack surface and emphasizes the importance of identifying all listening services before attempting exploitation. Rather than focusing immediately on vulnerabilities, the chapter stresses a methodical, attacker-style approach that prioritizes completeness and accuracy.

Using Nmap as the primary tool, the chapter demonstrates how to perform both targeted scans of commonly attacked ports and comprehensive scans of all TCP ports to uncover less obvious services. It highlights the tradeoff between speed and depth, showing how quick scans provide early insight while exhaustive scans yield detailed intelligence such as service names, versions, and operating system information.

Finally, the chapter introduces structured analysis of scan results through XML parsing and automation. By converting raw Nmap output into organized, protocol-specific target lists, penetration testers can efficiently manage large data sets and prepare for the next phase of the engagement. Overall, Chapter 3 establishes a disciplined workflow for turning discovered hosts into actionable service inventories that directly support effective vulnerability discovery.

3.1 Network services from an attacker's perspective

This section frames network services as an organization's primary attack surface—the “entry points” an attacker can interact with over the network. A service is any application listening on a network port (0–65,535) and responding according to a defined protocol (e.g., HTTP, SSH, SMB). From an attacker's viewpoint, the goal during service discovery is not to exploit immediately, but to identify all reachable services and collect useful details (protocol, product, version, and configuration hints). The chapter emphasizes being thorough and methodical: capture potential attack vectors as you find them, but avoid getting distracted early, because a complete inventory of services produces a stronger foundation for later vulnerability discovery.

3.2 Port scanning with Nmap

This section describes service discovery as structured port scanning using Nmap. The process typically starts with a quick sweep of commonly targeted ports to gain early visibility into likely high-value services (web, remote access, file sharing, databases). Next, an exhaustive scan of all TCP ports is performed to ensure less obvious services are not missed, often combined with service/version detection (-sV) and advanced enumeration (-A) such as OS fingerprinting and NSE script checks. The tradeoff is time: deeper scans generate significantly more actionable intelligence, but require careful planning, performance tuning, and organized output handling. Overall, the goal is to produce a reliable, repeatable set of scan results that accurately represents the environment's exposed services.

3.3 Parsing XML output with Ruby

This section focuses on converting Nmap results into an analysis-ready format. Because large scans generate high-volume output, manually reviewing text results can be error-prone and inefficient. Nmap's XML output enables structured parsing where hosts, ports, services, and script results can be extracted consistently. The chapter introduces using a Ruby-based XML parsing workflow to produce a clean, tabular summary that can be imported into tools like spreadsheets for sorting and filtering. The main objective is to transform raw scan data into protocol-specific target lists (e.g., web, SMB, SQL, VNC), which streamlines the next phase, vulnerability discovery, by letting the tester focus on groups of similar services with the appropriate tools and checks.

Chapter 3 Figures

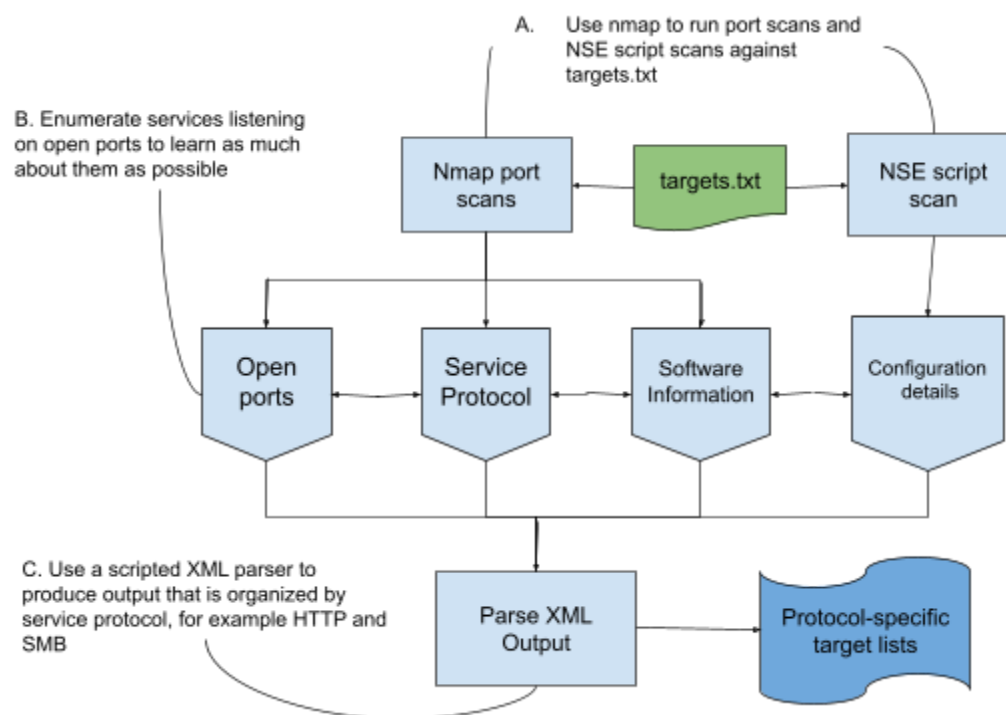


Figure 3.1 Sub-phase B: Service discovery workflow

From *The Art of Network Penetration Testing* (Chapter 3), by R. Davis, 2020, Manning Publications

Figure 3.1 presents a visual overview of the complete service discovery sub-phase, broken down into its individual components. This stage starts with the `targets.txt` file generated during host discovery and concludes with a comprehensive inventory of all identified network services, organized into separate protocol-specific lists that will be used in the next chapter.

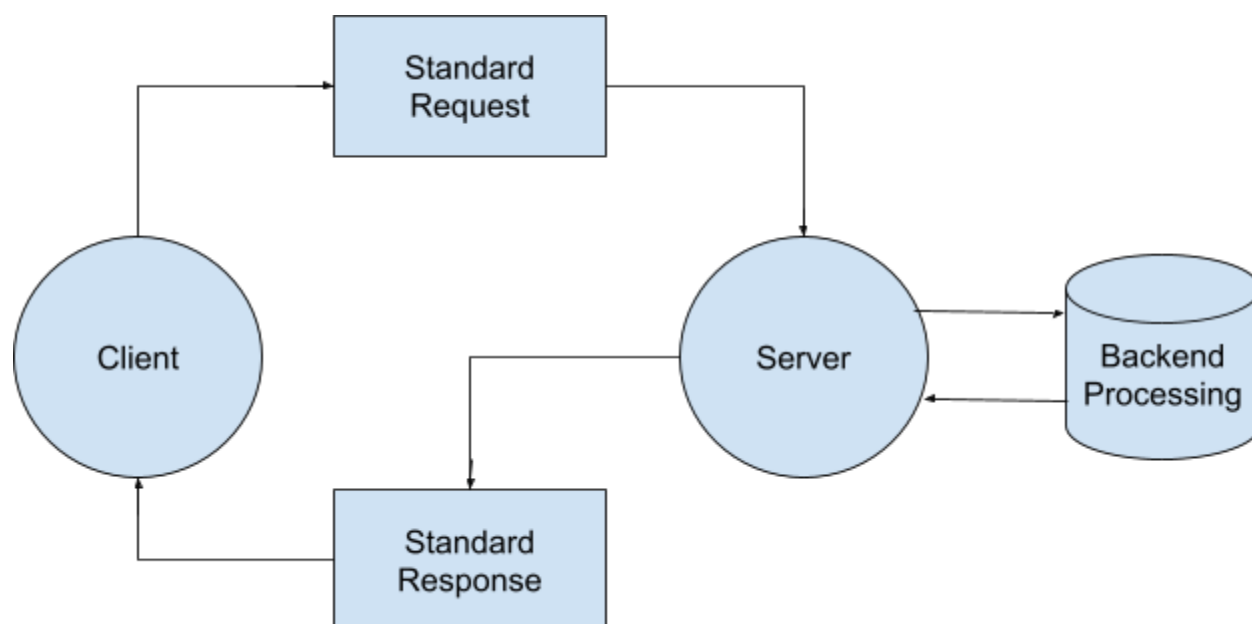


Figure 3.2 Illustration of a typical network service request and response

From *The Art of Network Penetration Testing* (Chapter 3), by R. Davis, 2020, Manning Publications

Shows typical client–server interaction showing a standard request sent from the client to the server, server-side backend processing, and a standard response returned to the client.

Chapter 4 Summary

Chapter 4 transitions from service discovery into vulnerability discovery, focusing on determining which identified services present practical attack opportunities. The chapter emphasizes a “path of least resistance” approach, prioritizing low-effort, high-impact weaknesses such as missing patches, weak or default credentials, and insecure configurations. Rather than immediately exploiting the first finding, the workflow encourages comprehensive identification and documentation of potential attack paths to support safer, more informed decisions during focused penetration. Key deliverables include validated vulnerability evidence, confirmed weak credentials, and a prioritized target set.

4.1 Understanding vulnerability discovery

Vulnerability discovery begins with protocol-specific target lists generated during service discovery (e.g., Windows, databases, web services, remote administration protocols).

These lists are used as inputs to targeted tools and techniques that assess three primary categories: credential exposure, patch level, and web-facing attack surfaces. The approach aligns with attacker-style validation, emphasizing weaknesses that can be reliably confirmed in practice rather than a purely CVE/CVSS-driven vulnerability management process. The expected outcome is a structured set of documented vulnerabilities and candidate attack vectors suitable for follow-on exploitation planning.

4.2 Discovering patching vulnerabilities

Patching vulnerability discovery focuses on identifying outdated software by comparing enumerated service versions against vendor releases and public exploit references. The process includes extracting version information from service discovery outputs, validating whether versions are behind current stable releases, and mapping findings to known exploitability using public exploit databases. For Windows environments, the chapter highlights validation of high-impact legacy vulnerabilities (e.g., MS17-010) by enumerating Windows versions first and then using scanner modules to assess likelihood of exposure. The output of this stage is a list of hosts with evidence of potentially exploitable missing patches, prioritized by impact and confidence.

4.3 Discovering authentication vulnerabilities

Authentication vulnerabilities are defined as default, blank, weak, or easily guessable credentials that enable unauthorized access. The chapter demonstrates creation of a short, organization-tailored password list combining common defaults with predictable permutations that frequently satisfy minimum password complexity requirements. Credential testing is then conducted across services such as SMB/Windows local administrator accounts, MSSQL, MySQL, and VNC using appropriate tooling (e.g., CrackMapExec, Metasploit auxiliary modules, and Medusa for validation). Results are recorded as confirmed credential findings tied to host, service, and access level, providing high-confidence attack paths for subsequent phases.

4.4 Discovering configuration vulnerabilities

Configuration vulnerabilities occur when service settings expose functionality or access paths that enable exploitation (e.g., management consoles, insecure remote administration interfaces, or permissive deployment features). The chapter emphasizes web services as a frequent source of configuration risk due to the volume of HTTP-based interfaces present in enterprise environments and the prevalence of default credentials and exposed administrative panels. To improve triage efficiency, automated web enumeration and screenshot-based review is introduced to rapidly classify web targets and identify high-value interfaces (e.g., application managers,

CI/CD consoles, device admin portals). Manual validation of common default credentials and targeted research into unfamiliar platforms support identification of misconfigurations that may enable further compromise during focused penetration.

Chapter 4 Figures

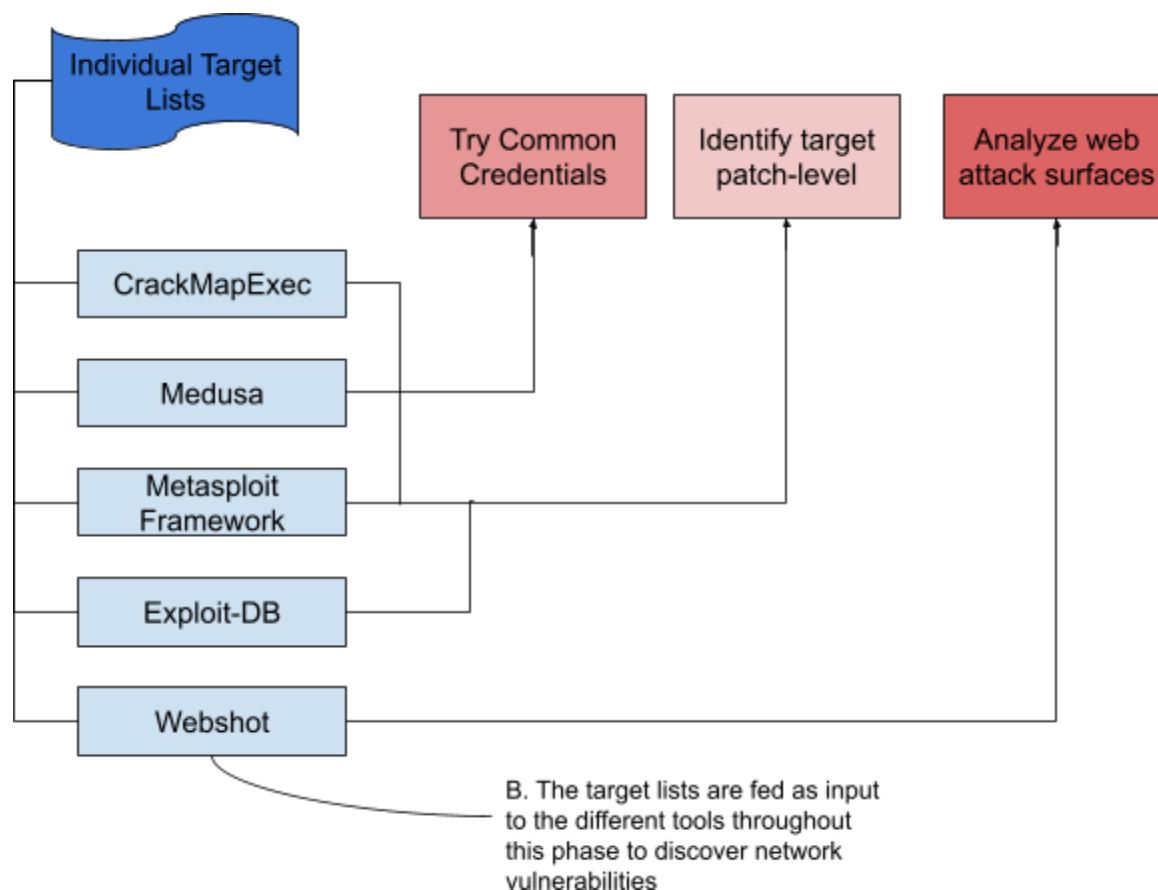


Figure 4.1 The vulnerability-discovery sub-phase workflow

From The Art of Network Penetration Testing (Chapter 4), by R. Davis, 2020, Manning Publications

Workflow illustrating how individual target lists are supplied to multiple security tools (e.g., CrackMapExec, Medusa, Metasploit, Exploit-DB, and Webshot) to test common credentials, identify patch levels, and analyze web attack surfaces during network vulnerability discovery.

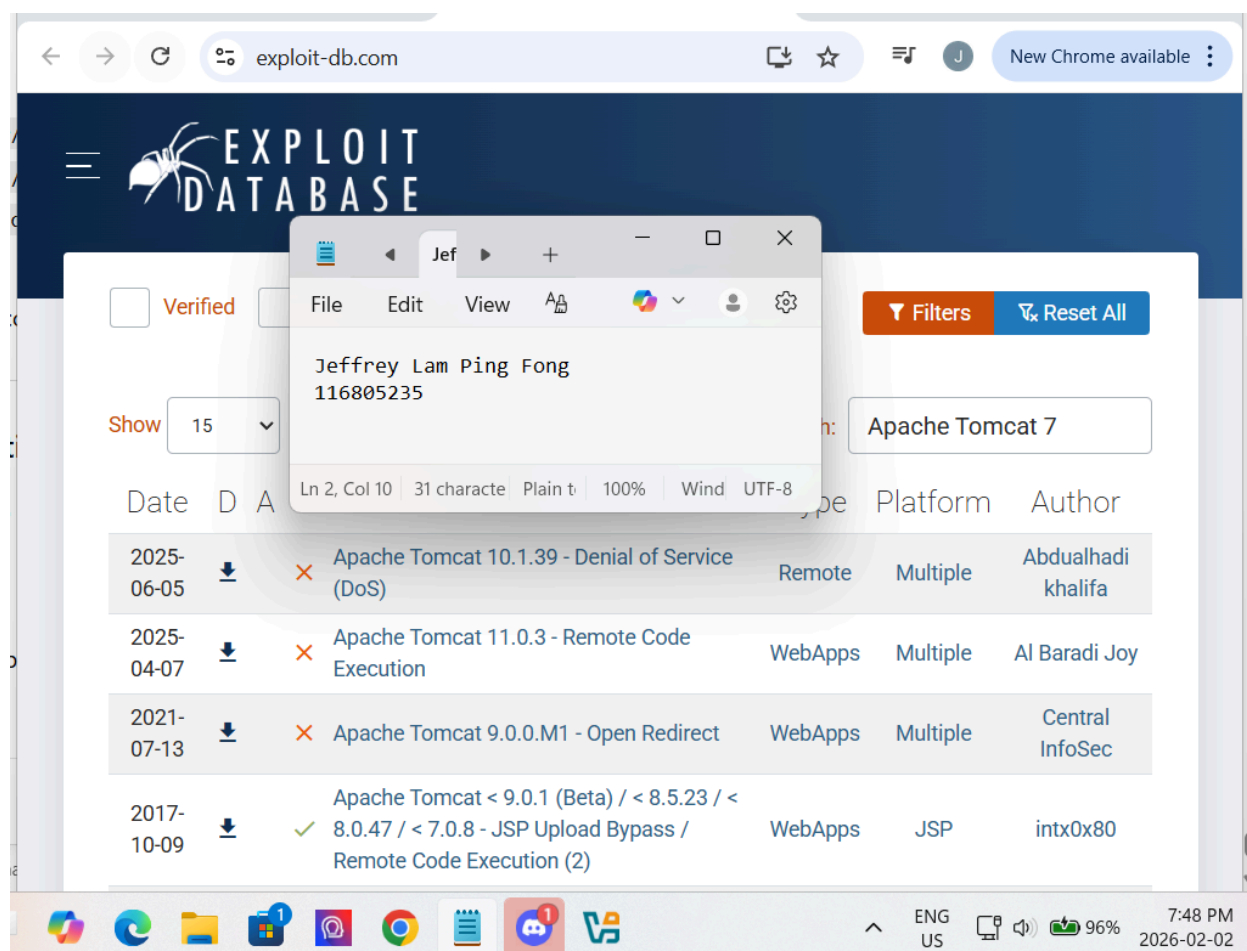


Figure 4.2 Searching the public exploit database for “Apache Tomcat 7”

Figure 4.2 list of all the current known exploitable attack vectors

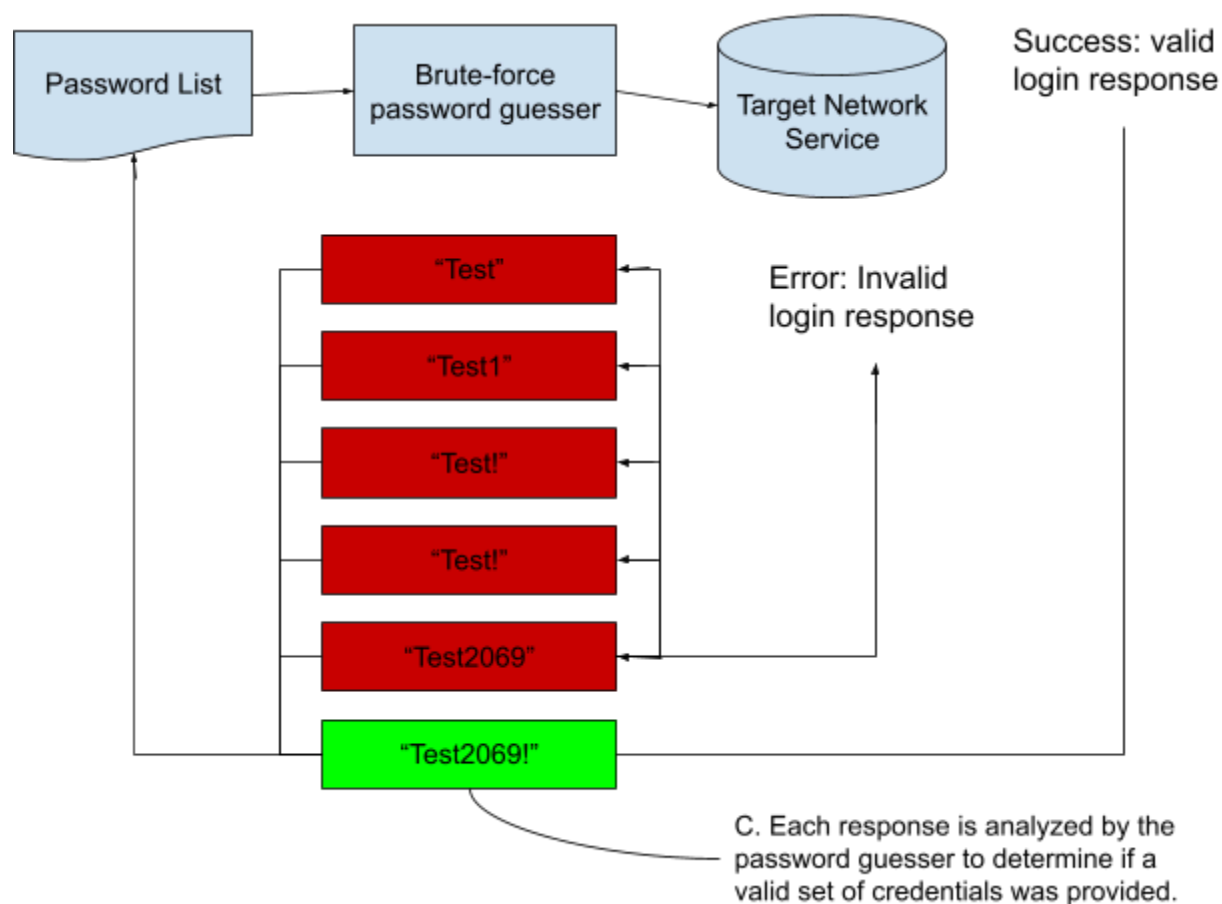
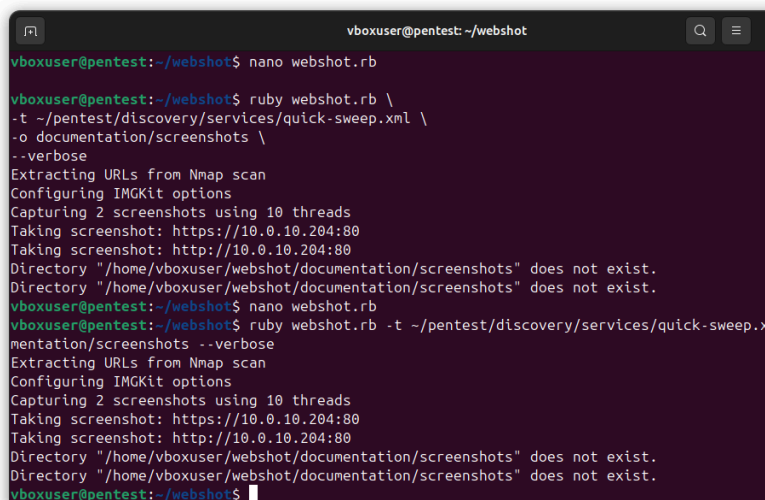


Figure 4.3 Brute-force password guessing

From The Art of Network Penetration Testing (Chapter 4), by R. Davis, 2020, Manning Publications

Diagram illustrating a brute-force password guessing workflow: a password list feeds a guessing tool that submits repeated login attempts to a target service, analyzes error vs. success responses, and continues until valid credentials are found (red = failed attempts, green = successful login).

JEFFREY LAM PING FONG
116805235



```
vboxuser@pentest: ~/webshot
vboxuser@pentest: ~/webshot$ nano webshot.rb
vboxuser@pentest: ~/webshot$ ruby webshot.rb \
-t ~/pentest/discovery/services/quick-sweep.xml \
-o documentation/screenshots \
--verbose
Extracting URLs from Nmap scan
Configuring IMGKit options
Capturing 2 screenshots using 10 threads
Taking screenshot: https://10.0.10.204:80
Taking screenshot: http://10.0.10.204:80
Directory "/home/vboxuser/webshot/documentation/screenshots" does not exist.
Directory "/home/vboxuser/webshot/documentation/screenshots" does not exist.
vboxuser@pentest: ~/webshot$ nano webshot.rb
vboxuser@pentest: ~/webshot$ ruby webshot.rb -t ~/pentest/discovery/services/quick-sweep.x
mentation/screenshots --verbose
Extracting URLs from Nmap scan
Configuring IMGKit options
Capturing 2 screenshots using 10 threads
Taking screenshot: https://10.0.10.204:80
Taking screenshot: http://10.0.10.204:80
Directory "/home/vboxuser/webshot/documentation/screenshots" does not exist.
Directory "/home/vboxuser/webshot/documentation/screenshots" does not exist.
vboxuser@pentest: ~/webshot$
```

Figure 4.4 webshot

Capturing screenshots using webshot of what is in use in the network

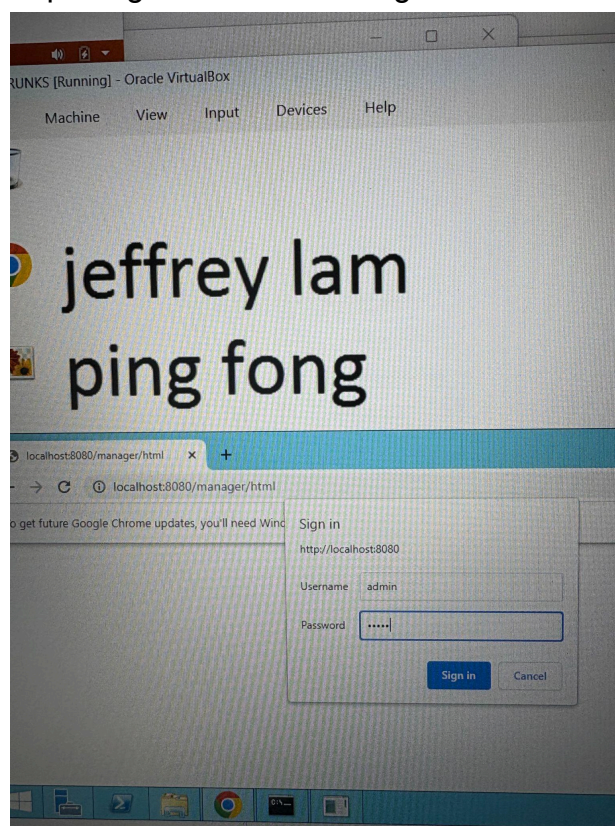


Figure 4.5

Logging into to apache Tomcat

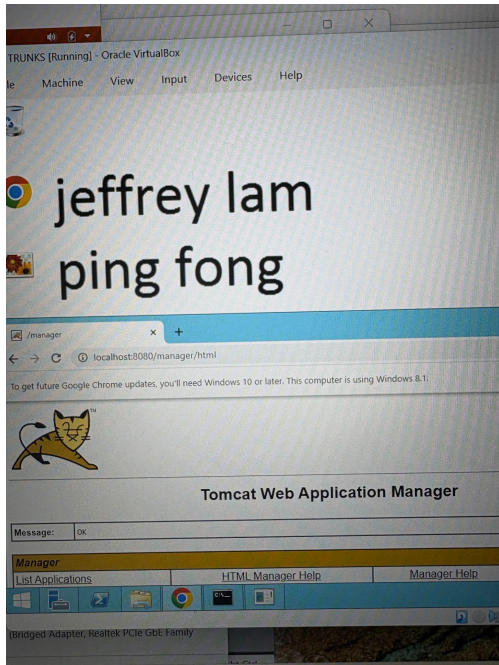


Figure 4.6

Logged in to the Apache Tomcat application manager

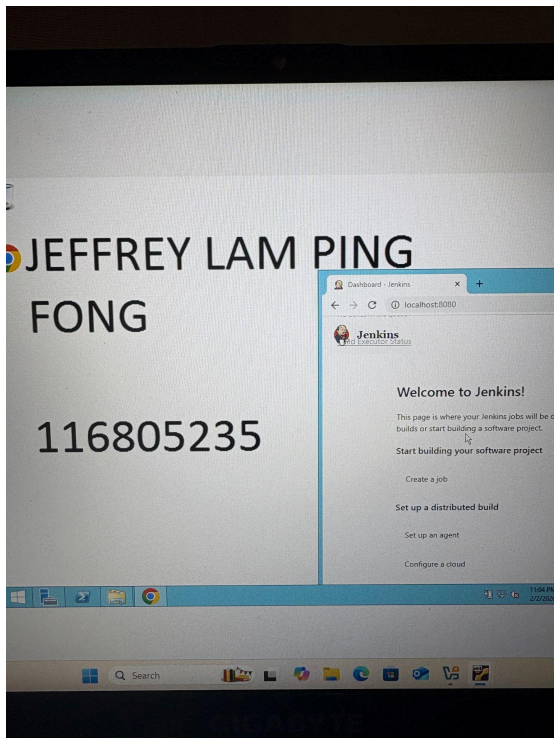


Figure 4.7

Logged in to the Jenkins admin portal

Exercise 3.1

Exercise 3.1 completed service discovery by running Nmap scans against the targets.txt scope, collecting results in structured output files, and parsing them into an all-ports.csv inventory for easier review. The scan results were then organized into protocol-specific target lists (e.g., web.txt, windows.txt, mssql.txt, mysql.txt, vnc.txt), each containing only the IP addresses of systems offering that service. This organization creates a clean workflow for the next phase (vulnerability discovery) by allowing tools to be run against each service category separately rather than manually reviewing large scan outputs.

JEFFREY LAM PING FONG

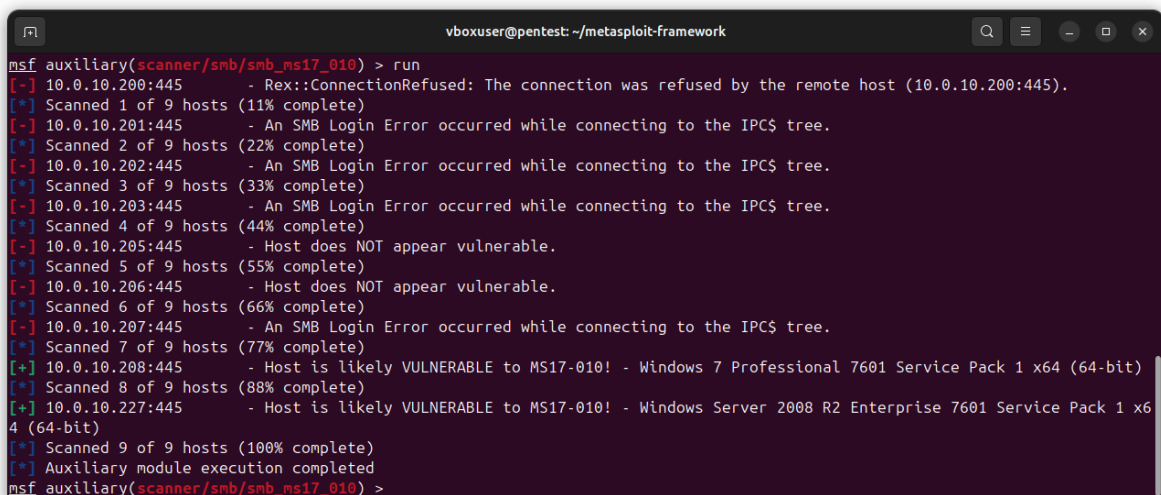
116805235

```
vboxuser@pentest:~/pentest/discovery$ tree
.
├── hosts
│   ├── mssql.txt
│   ├── mysql.txt
│   ├── pingsweep.gnmap
│   ├── pingsweep.nmap
│   ├── pingsweep.xml
│   ├── rmisweep_fast.gnmap
│   ├── rmisweep_fast.nmap
│   ├── rmisweep_fast.xml
│   ├── rmisweep.gnmap
│   ├── rmisweep.nmap
│   ├── rmisweep.xml
│   ├── targets.txt
│   ├── vnc.txt
│   ├── web.txt
│   └── windows.txt
├── ranges.txt
├── services
│   ├── all-ports.csv
│   ├── full-sweep.gnmap
│   ├── full-sweep.nmap
│   ├── full-sweep.xml
│   ├── quick-sweep.gnmap
│   ├── quick-sweep.nmap
│   └── quick-sweep.xml
└── vulnerabilities

4 directories, 23 files
```

Exercise 4.1

JEFFREY LAM PING FONG
116805235

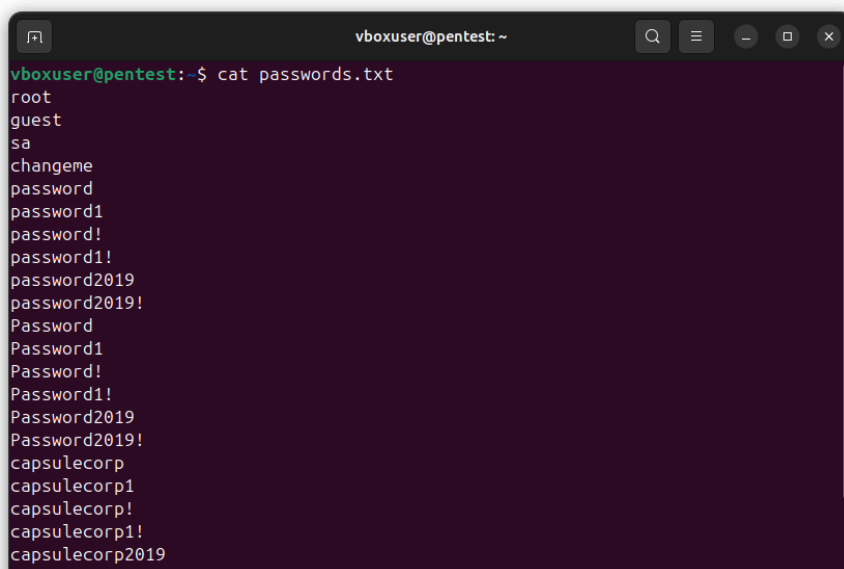


```
vboxuser@pentest: ~/metasploit-framework
msf auxiliary(scanner/smb/smb_ms17_010) > run
[-] 10.0.10.200:445 - Rex::ConnectionRefused: The connection was refused by the remote host (10.0.10.200:445).
[*] Scanned 1 of 9 hosts (11% complete)
[-] 10.0.10.201:445 - An SMB Login Error occurred while connecting to the IPC$ tree.
[*] Scanned 2 of 9 hosts (22% complete)
[-] 10.0.10.202:445 - An SMB Login Error occurred while connecting to the IPC$ tree.
[*] Scanned 3 of 9 hosts (33% complete)
[-] 10.0.10.203:445 - An SMB Login Error occurred while connecting to the IPC$ tree.
[*] Scanned 4 of 9 hosts (44% complete)
[-] 10.0.10.205:445 - Host does NOT appear vulnerable.
[*] Scanned 5 of 9 hosts (55% complete)
[-] 10.0.10.206:445 - Host does NOT appear vulnerable.
[*] Scanned 6 of 9 hosts (66% complete)
[-] 10.0.10.207:445 - An SMB Login Error occurred while connecting to the IPC$ tree.
[*] Scanned 7 of 9 hosts (77% complete)
[+] 10.0.10.208:445 - Host is likely VULNERABLE to MS17-010! - Windows 7 Professional 7601 Service Pack 1 x64 (64-bit)
[*] Scanned 8 of 9 hosts (88% complete)
[+] 10.0.10.227:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2008 R2 Enterprise 7601 Service Pack 1 x64 (64-bit)
[*] Scanned 9 of 9 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/smb/smb_ms17_010) >
```

Exercise 4.1 focused on identifying patching vulnerabilities by correlating discovered service versions with known publicly disclosed exploits. Service and version information obtained during service discovery (Chapter 3) was reviewed against public exploit databases such as Exploit-DB to determine whether systems were running outdated or vulnerable software. For Windows systems, CrackMapExec was used to enumerate operating system versions, followed by execution of the Metasploit MS17-010 auxiliary scanner to identify hosts potentially vulnerable to the EternalBlue vulnerability. Any systems identified as running outdated software or missing critical security patches were recorded as patching vulnerabilities for later exploitation planning.

Exercise 4.2

JEFFREY LAM PING FONG

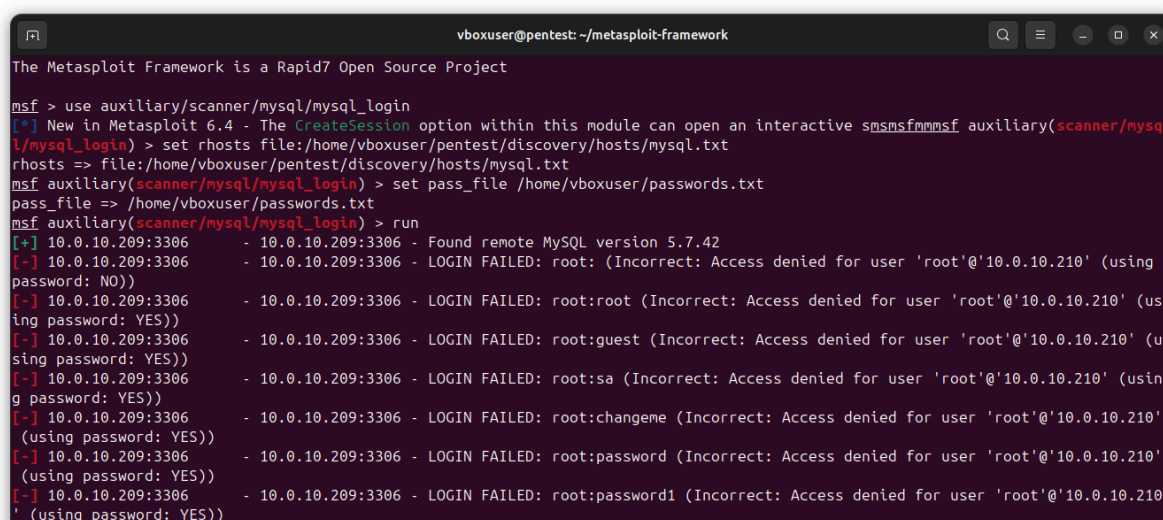
A terminal window titled 'vboxuser@pentest: ~' with a dark purple background. The prompt is 'vboxuser@pentest:~\$' and the command 'cat passwords.txt' has been executed. The output is a list of 20 passwords, each on a new line. The list includes common words like 'root', 'guest', 'sa', and 'changeme', as well as variations of 'password' and 'capsulecorp' with different capitalizations, numeric suffixes, and special characters. The list ends with 'capsulecorp2019!'.

```
vboxuser@pentest:~$ cat passwords.txt
root
guest
sa
changeme
password
password1
password!
password1!
password2019
password2019!
Password
Password1
Password!
Password1!
Password2019
Password2019!
capsulecorp
capsulecorp1
capsulecorp!
capsulecorp1!
capsulecorp2019
capsulecorp2019!
```

Exercise 4.2 involved creating a targeted password list tailored to the assessed environment. Rather than relying on large generic wordlists, a short and efficient list was generated using common base words (such as “password” and the organization name) combined with simple permutations that reflect typical user behavior. These permutations included capitalization changes, numeric suffixes, special characters, and current-year variations. The resulting list was intentionally kept small to enable fast and low-noise password-guessing attempts while still maintaining a high probability of success. The password list was stored in the vulnerabilities directory and reused across multiple authentication tests.

Exercise 4.3

JEFFREY LAM PING FONG
116805235



```

vboxuser@pentest: ~/metasploit-framework
The Metasploit Framework is a Rapid7 Open Source Project

msf > use auxiliary/scanner/mysql/mysql_login
[*] New in Metasploit 6.4 - The CreateSession option within this module can open an interactive session
msf auxiliary(scanner/mysql/mysql_login) > set rhosts file:/home/vboxuser/pentest/discovery/hosts/mysql.txt
rhosts => file:/home/vboxuser/pentest/discovery/hosts/mysql.txt
msf auxiliary(scanner/mysql/mysql_login) > set pass_file /home/vboxuser/passwords.txt
pass_file => /home/vboxuser/passwords.txt
msf auxiliary(scanner/mysql/mysql_login) > run
[+] 10.0.10.209:3306 - 10.0.10.209:3306 - Found remote MySQL version 5.7.42
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root: (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: NO))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:root (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:guest (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:sa (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:changeme (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:password (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
[-] 10.0.10.209:3306 - 10.0.10.209:3306 - LOGIN FAILED: root:password1 (Incorrect: Access denied for user 'root'@'10.0.10.210' (using password: YES))
  
```

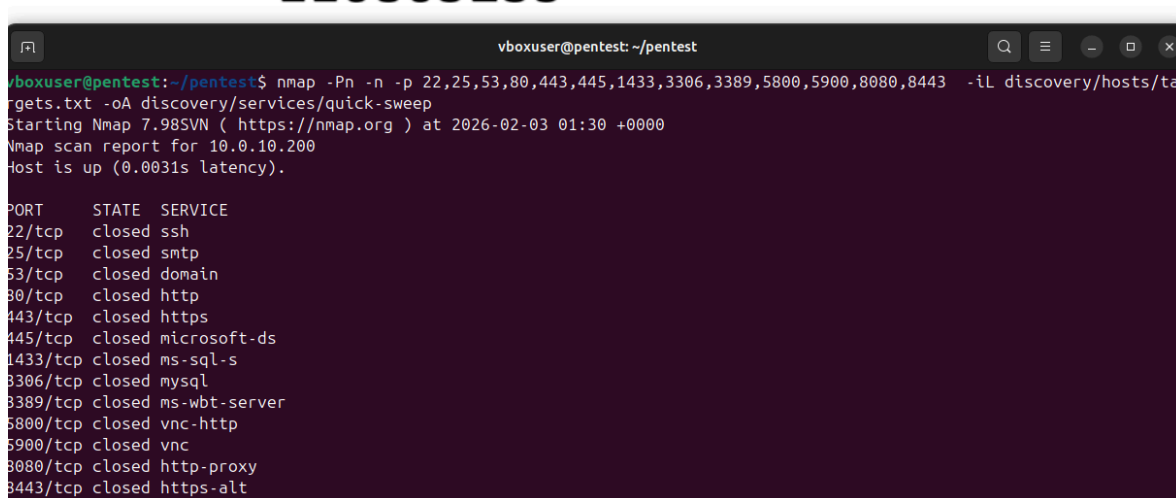
Exercise 4.3 applied the client-specific password list to identify authentication vulnerabilities across multiple network services. CrackMapExec was used to test local administrator credentials on Windows systems, while Metasploit and Medusa were used to perform password-guessing attacks against MSSQL, MySQL, and VNC services using the protocol-specific target lists created earlier. Successful logins indicated the presence of weak, default, or reused credentials and were documented as authentication vulnerabilities along with the affected service and host. These findings established multiple viable attack paths for focused penetration in subsequent phases.

References

Davis, Royce. The Art of Network Penetration Testing. Manning Publications, 2020.

Appendix A

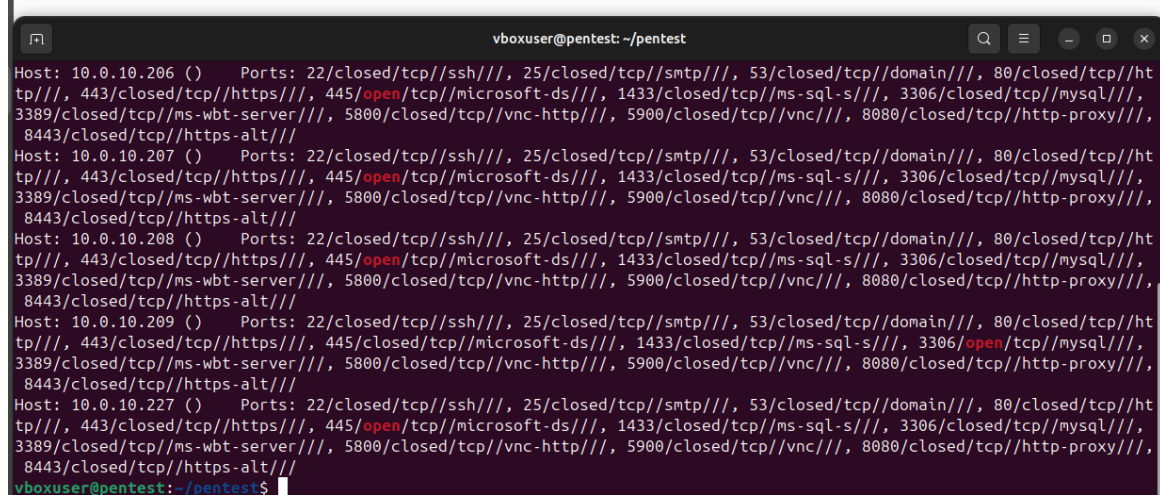
JEFFREY LAM PING FONG
116805235



```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ nmap -Pn -n -p 22,25,53,80,443,445,1433,3306,3389,5800,5900,8080,8443 -iL discovery/hosts/targets.txt -oA discovery/services/quick-sweep
Starting Nmap 7.98SVN ( https://nmap.org ) at 2026-02-03 01:30 +0000
Nmap scan report for 10.0.10.200
Host is up (0.0031s latency).

PORT      STATE SERVICE
22/tcp    closed ssh
25/tcp    closed smtp
53/tcp    closed domain
80/tcp    closed http
443/tcp   closed https
445/tcp   closed microsoft-ds
1433/tcp  closed ms-sql-s
3306/tcp  closed mysql
3389/tcp  closed ms-wbt-server
5800/tcp  closed vnc-http
5900/tcp  closed vnc
8080/tcp  closed http-proxy
8443/tcp  closed https-alt
```

JEFFREY LAM PING FONG
116805235



```
vboxuser@pentest: ~/pentest
Host: 10.0.10.206 () Ports: 22/closed/tcp/ssh///, 25/closed/tcp/smtp///, 53/closed/tcp/domain///, 80/closed/tcp/http///, 443/closed/tcp/https///, 445/open/tcp/microsoft-ds///, 1433/closed/tcp/ms-sql-s///, 3306/closed/tcp/mysql///, 3389/closed/tcp/ms-wbt-server///, 5800/closed/tcp/vnc-http///, 5900/closed/tcp/vnc///, 8080/closed/tcp/http-proxy///, 8443/closed/tcp/https-alt///
Host: 10.0.10.207 () Ports: 22/closed/tcp/ssh///, 25/closed/tcp/smtp///, 53/closed/tcp/domain///, 80/closed/tcp/http///, 443/closed/tcp/https///, 445/open/tcp/microsoft-ds///, 1433/closed/tcp/ms-sql-s///, 3306/closed/tcp/mysql///, 3389/closed/tcp/ms-wbt-server///, 5800/closed/tcp/vnc-http///, 5900/closed/tcp/vnc///, 8080/closed/tcp/http-proxy///, 8443/closed/tcp/https-alt///
Host: 10.0.10.208 () Ports: 22/closed/tcp/ssh///, 25/closed/tcp/smtp///, 53/closed/tcp/domain///, 80/closed/tcp/http///, 443/closed/tcp/https///, 445/open/tcp/microsoft-ds///, 1433/closed/tcp/ms-sql-s///, 3306/closed/tcp/mysql///, 3389/closed/tcp/ms-wbt-server///, 5800/closed/tcp/vnc-http///, 5900/closed/tcp/vnc///, 8080/closed/tcp/http-proxy///, 8443/closed/tcp/https-alt///
Host: 10.0.10.209 () Ports: 22/closed/tcp/ssh///, 25/closed/tcp/smtp///, 53/closed/tcp/domain///, 80/closed/tcp/http///, 443/closed/tcp/https///, 445/closed/tcp/microsoft-ds///, 1433/closed/tcp/ms-sql-s///, 3306/open/tcp/mysql///, 3389/closed/tcp/ms-wbt-server///, 5800/closed/tcp/vnc-http///, 5900/closed/tcp/vnc///, 8080/closed/tcp/http-proxy///, 8443/closed/tcp/https-alt///
Host: 10.0.10.227 () Ports: 22/closed/tcp/ssh///, 25/closed/tcp/smtp///, 53/closed/tcp/domain///, 80/closed/tcp/http///, 443/closed/tcp/https///, 445/open/tcp/microsoft-ds///, 1433/closed/tcp/ms-sql-s///, 3306/closed/tcp/mysql///, 3389/closed/tcp/ms-wbt-server///, 5800/closed/tcp/vnc-http///, 5900/closed/tcp/vnc///, 8080/closed/tcp/http-proxy///, 8443/closed/tcp/https-alt///
vboxuser@pentest:~/pentest$
```

JEFFREY LAM PING FONG

116805235

```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ nmap -Pn -n -iL discovery/hosts/targets.txt -p 0-65535 -sV -A \
-oA discovery/services/full-sweep --min-rate 50000 --min-hostgroup 22
Starting Nmap 7.98SVN ( https://nmap.org ) at 2026-02-03 01:47 +0000
Nmap scan report for 10.0.10.200
Host is up (0.020s latency).
Not shown: 65528 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2026-02-03 04:23:41Z)
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
464/tcp    open  kpasswd5?
49664/tcp  open  msrpc        Microsoft Windows RPC
49665/tcp  open  msrpc        Microsoft Windows RPC
49666/tcp  open  msrpc        Microsoft Windows RPC
49667/tcp  open  msrpc        Microsoft Windows RPC
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_nbstat: NetBIOS name: nil, NetBIOS user: <unknown>, NetBIOS MAC: 08:00:27:93:4b:a1 (Oracle VirtualBox virtual NIC)
|_smb2-security-mode: SMB: Couldn't find a NetBIOS name that works for the server. Sorry!
```

JEFFREY LAM PING FONG

116805235

```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ cat discovery/services/full-sweep.nmap | grep '|_|' |
cut -d '_' -f2 | cut -d ' ' -f1 | sort -u | grep ':'
clock-skew:
http-server-header:
http-title:
nbstat:
smb2-security-mode:
smb2-time:
ssl-date:
vboxuser@pentest:~/pentest$
```

JEFFREY LAM PING FONG
116805235

```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ cat discovery/services/full-sweep.nmap | grep -i http-title
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-title: Service Unavailable
|_ http-title: Service Unavailable
|_ http-title: Not Found
|_ http-title: Not Found
|_ http-title: Service Unavailable
|_ http-title: Not Found
vboxuser@pentest:~/pentest$
```

JEFFREY LAM PING FONG
116805235

```
vboxuser@pentest: ~/parsenmap
vboxuser@pentest:~/parsenmap$ parsenmap ~/pentest/discovery/services/full-sweep.xml
10.0.10.200 88 kerberos-sec Microsoft Windows Kerberos server time: 2026-02-03 04:23:41Z
10.0.10.200 135 msrpc Microsoft Windows RPC
10.0.10.200 139 netbios-ssn Microsoft Windows netbios-ssn
10.0.10.200 464 kpasswd
10.0.10.200 49664 msrpc Microsoft Windows RPC
10.0.10.200 49665 msrpc Microsoft Windows RPC
10.0.10.200 49666 msrpc Microsoft Windows RPC
10.0.10.200 49667 msrpc Microsoft Windows RPC
10.0.10.201 135 msrpc Microsoft Windows RPC
10.0.10.201 139 netbios-ssn Microsoft Windows netbios-ssn
10.0.10.201 445 microsoft-ds Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
10.0.10.201 5985 http Microsoft HTTPAPI httpd 2.0 SSDP/UPnP
10.0.10.201 47001 http Microsoft HTTPAPI httpd 2.0 SSDP/UPnP
10.0.10.201 49664 msrpc Microsoft Windows RPC
10.0.10.201 49665 msrpc Microsoft Windows RPC
10.0.10.201 49666 msrpc Microsoft Windows RPC
10.0.10.201 49667 msrpc Microsoft Windows RPC
10.0.10.201 49668 msrpc Microsoft Windows RPC
10.0.10.201 49669 msrpc Microsoft Windows RPC
```

JEFFREY LAM PING FONG

116805235

```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ sudo snap install crackmapexec
crackmapexec v5.4.0-1355-gd8c50c8 from Jitendra Patro (jitpatro) installed
vboxuser@pentest:~/pentest$ crackmapexec smb discovery/hosts/windows.txt
[*] First time use detected
[*] Creating home directory structure
[*] Creating missing folder logs
[*] Creating missing folder modules
[*] Creating missing folder protocols
[*] Creating missing folder workspaces
[*] Creating missing folder obfuscated_scripts
[*] Creating missing folder screenshots
[*] Copying default configuration file
SMB 10.0.10.202 445 VEGETA [*] Windows Server 2012 R2 Datacenter Evaluation 9600 x64 (name:VEGETA) (domain:capsulecorp.local) (signing:False) (SMBv1:True)
SMB 10.0.10.208 445 TIEN-PC [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:TIEN-PC) (domain:Tien-PC) (signing:False) (SMBv1:True)
SMB 10.0.10.201 445 GOHAN [*] Windows Server 2016 Standard 14393 x64 (name:GOHAN) (domain:capsulecorp.local) (signing:False) (SMBv1:True)
SMB 10.0.10.227 445 WIN-JOSM7RM05LF [*] Windows Server 2008 R2 Enterprise 7601 Service Pack 1 x64 (name:WIN-JOSM7RM05LF) (domain:WIN-JOSM7RM05LF) (signing:False) (SMBv1:True)
SMB 10.0.10.207 445 RADITZ [*] Windows Server 2016 Datacenter 14393 x64 (name:RADITZ) (domain:capsulecorp.local) (signing:False) (SMBv1:True)
SMB 10.0.10.205 445 KRILLIN [*] Windows 10 Pro 10240 x64 (name:KRILLIN) (domain:capsulecorp.local) (signing:False) (SMBv1:True)
SMB 10.0.10.203 445 TRUNKS [*] Windows Server 2012 R2 Datacenter Evaluation 9600 x64 (name:TRUNKS) (domain:capsulecorp.local) (signing:False) (SMBv1:True)
```

JEFFREY LAM PING FONG

116805235

```
vboxuser@pentest: ~/pentest
vboxuser@pentest:~/pentest$ crackmapexec smb discovery/hosts/windows.txt --local-auth -u Administrator -p passwords.txt
SMB 10.0.10.203 445 TRUNKS [*] Windows Server 2012 R2 Datacenter Evaluation 9600 x64 (name:TRUNKS) (domain:TRUNKS) (signing:False) (SMBv1:True)
SMB 10.0.10.208 445 TIEN-PC [*] Windows 7 Professional 7601 Service Pack 1 x64 (name:TIEN-PC) (domain:TIEN-PC) (signing:False) (SMBv1:True)
SMB 10.0.10.206 445 YAMCHA [*] Windows 10 Pro 10240 x64 (name:YAMCHA) (domain:YAMCHA) (signing:False) (SMBv1:True)
SMB 10.0.10.227 445 WIN-JOSM7RM05LF [*] Windows Server 2008 R2 Enterprise 7601 Service Pack 1 x64 (name:WIN-JOSM7RM05LF) (domain:WIN-JOSM7RM05LF) (signing:False) (SMBv1:True)
SMB 10.0.10.201 445 GOHAN [*] Windows Server 2016 Standard 14393 x64 (name:GOHAN) (domain:GOHAN) (signing:False) (SMBv1:True)
SMB 10.0.10.207 445 RADITZ [*] Windows Server 2016 Datacenter 14393 x64 (name:RADITZ) (domain:RADITZ) (signing:False) (SMBv1:True)
SMB 10.0.10.202 445 VEGETA [*] Windows Server 2012 R2 Datacenter Evaluation 9600 x64 (name:VEGETA) (domain:VEGETA) (signing:False) (SMBv1:True)
SMB 10.0.10.205 445 KRILLIN [*] Windows 10 Pro 10240 x64 (name:KRILLIN) (domain:KRILLIN) (signing:False) (SMBv1:True)
SMB 10.0.10.203 445 TRUNKS [-] TRUNKS\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.208 445 TIEN-PC [-] TIEN-PC\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.206 445 YAMCHA [-] YAMCHA\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.227 445 WIN-JOSM7RM05LF [-] WIN-JOSM7RM05LF\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.201 445 GOHAN [-] GOHAN\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.207 445 RADITZ [-] RADITZ\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.202 445 VEGETA [-] VEGETA\Administrator:passwords.txt STATUS_LOGON_FAILURE
SMB 10.0.10.205 445 KRILLIN [-] KRILLIN\Administrator:passwords.txt STATUS_LOGON_FAILURE
Running CME against 9 targets 100% 0:00:00
vboxuser@pentest:~/pentest$ ~
```

Lab 3

Jeffrey Lam Ping Fong 116805235

RIS602NBB

Mark Shtern

February 16th 2026

Table of Contents

Executive Summary.....	6
Summary of Chapter 5.....	6
5.1 Understanding Phase 2: Focused Penetration.....	7
5.2 Gaining an Initial Foothold.....	7
5.3 Compromising a Vulnerable Tomcat Server.....	7
5.4 Interactive vs. Non-Interactive Shells.....	8
5.5 Upgrading to an Interactive Shell.....	8
5.6 Compromising a Vulnerable Jenkins Server.....	9
Chapter 5 Figures.....	10
Figure 5.1 Phase 2: focused-penetration workflow.....	10
Figure 5.2 An Apache Tomcat server listening on port 8080.....	11
Figure 5.3 The WAR file Deploy section of the Tomcat manager page.....	12
Figure 5.4 The webshell is deployed and is now accessible from the menu.....	13
Figure 5.5 Running OS commands with the web shell.....	14
Figure 5.6 The Sticky Keys prompt after pressing Shift five times.....	15
Figure 5.7 Result after issuing the sethc.exe backup command.....	15
Figure 5.8 SYSTEM-level command prompt instead of Sticky Keys.....	16
Figure 5.9 The Jenkins Groovy Script Console page.....	17
Figure 5.10 Executing OS commands using Groovy Script.....	18
Exercise 5.1.....	18
Summary of Chapter 6.....	19
6.1 Compromising Microsoft SQL Server.....	19
6.2 Stealing Windows Account Password Hashes.....	20
6.3 Extracting Password Hashes with Credump.....	20
Chapter 6 Figures.....	21
Figure 6.1 How Windows uses password hashes to authenticate users.....	21
Exercise 6.1.....	21
References.....	22

Table of Figures

Figure 5.1 Phase 2: focused-penetration workflow.....	10
Figure 5.2 An Apache Tomcat server listening on port 8080.....	11
Figure 5.3 The WAR file Deploy section of the Tomcat manager page.....	12
Figure 5.4 The webshell is deployed and is now accessible from the menu.....	13
Figure 5.5 Running OS commands with the web shell.....	14
Figure 5.6 The Sticky Keys prompt after pressing Shift five times.....	15
Figure 5.7 Result after issuing the sethc.exe backup command.....	15
Figure 5.8 SYSTEM-level command prompt instead of Sticky Keys.....	16
Figure 5.9 The Jenkins Groovy Script Console page.....	17
Figure 5.10 Executing OS commands using Groovy Script.....	18

Checklist

ID	What {Figure, Table, Section ...}	Where (Reference)	Completion (%)
1	Summary of Chapter 5	6	100%
2	5.1 Understanding phase 2: Focused penetration	7	100%
3	5.2 Gaining an initial foothold	7	100%
4	5.3 Compromising a vulnerable Tomcat server	7	100%
5	5.4 Interactive vs. non-interactive shells	8	100%
6	5.5 Upgrading to an interactive shell	8	100%
7	5.6 Compromising a vulnerable Jenkins server	9	100%
8	Figure 5.1	10	100%
9	Figure 5.2	11	100%
10	Figure 5.3	11	100%
11	Figure 5.4	12	100%
12	Figure 5.5	13	100%
13	Figure 5.6	14	100%
14	Figure 5.7	14	100%
15	Figure 5.8	15	100%
16	Figure 5.9	16	100%
17	Figure 5.10	17	100%
18	Exercise 5.1	17	100%
19	Summary of Chapter 6		100%

20	6.1 Compromising Microsoft SQL Server		100%
21	6.2 Stealing Windows account password hashes		100%
22	6.3 Extracting password hashes with creddump		100%
23	Figure 6.1		100%
24	Exercise 6.1		50%

Executive Summary

Chapter 5–6 document the focused-penetration phase of an internal network penetration test, where vulnerabilities identified during reconnaissance are systematically validated to obtain initial “level-one” access and expand control within the environment. Across both chapters, the primary theme is that weak authentication practices and exposed administrative functionality in common enterprise services (web platforms and database servers) can enable remote command execution and progression from service-level access to host-level control. The work highlights practical constraints of non-interactive access methods and the operational value of achieving stable, interactive control when permitted by the environment. Overall, these chapters demonstrate how misconfigurations such as default or easily guessed credentials, insecure management interfaces, and overly permissive service roles can create reliable footholds for escalation and follow-on actions, reinforcing the importance of hardening administrative endpoints, enforcing strong credential policies, applying least privilege, and monitoring privileged activity for early detection.

Summary of Chapter 5

Chapter 5 focuses on Phase 2 of the internal network penetration test: focused penetration. After identifying vulnerabilities during the information-gathering phase, the objective shifts to actively exploiting those weaknesses in order to gain initial footholds on vulnerable systems. These compromised systems, referred to as level-one hosts, provide remote control access and serve as entry points for further lateral movement within the network.

The chapter demonstrates how vulnerable web services can be leveraged to obtain system-level access. Specifically, it outlines the compromise of an Apache Tomcat server through deployment of a malicious WAR file containing a JSP web shell, as well as exploitation of a Jenkins server via the Groovy Script Console to execute operating system commands. Both scenarios illustrate how administrative web interfaces can introduce remote code execution risks when misconfigured or protected by weak credentials.

Additionally, the chapter explains the distinction between non-interactive and interactive shells. While non-interactive shells allow single-command execution, they impose operational limitations. Therefore, upgrading access to a fully interactive shell is a priority. This is demonstrated through the Sticky Keys backdoor technique on Windows systems, which enables SYSTEM-level command execution when RDP access is available.

5.1 Understanding Phase 2: Focused Penetration

Phase 2 of the internal network penetration test (INPT) transitions from information gathering to active exploitation. The objective of focused penetration is to systematically compromise vulnerable “level-one” hosts identified during the reconnaissance phase. A level-one host is any system that contains a direct access vulnerability that allows remote control, such as a web shell, remote desktop access, or command execution functionality.

This phase emphasizes a structured and methodical approach. Rather than pursuing a single vulnerability, testers move through the full list of discovered weaknesses, targeting web services first, followed by remote management interfaces and unpatched systems. The goal is to gain initial footholds on as many systems as possible to increase the likelihood of privilege escalation and broader network access.

5.2 Gaining an Initial Foothold

Gaining an initial foothold refers to obtaining the first form of remote control over a vulnerable system within the target environment. This access may take the form of a web shell, remote management login, or command execution interface. Although initial access may be limited, it provides a critical entry point into restricted network segments.

In practice, penetration testers must leverage the vulnerabilities available, even if they are not ideal. Initial footholds may involve deploying backdoor mechanisms or exploiting weak credentials. While these actions may temporarily modify system configurations, they are necessary steps toward deeper access, provided they are documented and later remediated.

5.3 Compromising a Vulnerable Tomcat Server

The Apache Tomcat server was identified as vulnerable due to default administrative credentials. Using access to the Tomcat Manager interface, a malicious web application archive (WAR) file containing a simple JSP web shell was created and deployed. The WAR file packages a web application structure that enables operating system command execution via a browser.

Once deployed, the web shell allowed remote execution of OS-level commands on the underlying Windows host. Commands such as `whoami` and `ipconfig /all` were used to gather system and network information. This method demonstrates how web application management interfaces can be abused to deploy backdoors and gain system-level access.

All modifications, including WAR deployment, must be documented and removed during post-engagement cleanup to avoid leaving persistent backdoors in the environment.

5.4 Interactive vs. Non-Interactive Shells

A shell obtained through a web interface, such as a JSP web shell or Jenkins script console, is typically non-interactive. Non-interactive shells allow execution of single commands but do not support multi-stage processes that require user input during execution.

This limitation requires commands to be structured carefully, often suppressing prompts or using flags that bypass interaction (e.g., automated confirmations). While useful for reconnaissance and basic system manipulation, non-interactive shells restrict flexibility and limit more advanced post-exploitation activities.

Understanding these constraints is critical for effective command execution during penetration testing engagements.

5.5 Upgrading to an Interactive Shell

Upgrading from a non-interactive shell to an interactive shell significantly enhances control over the compromised system. One demonstrated method involved leveraging the Windows Sticky Keys feature to establish a backdoor.

This technique required:

- Backing up the original `sethc.exe` file.
- Modifying file access control lists (ACLs) to grant sufficient permissions.
- Replacing `sethc.exe` with `cmd.exe`.

- Triggering Sticky Keys at the Windows login screen via RDP to obtain a SYSTEM-level command prompt.

An interactive shell provides full command-line functionality, enabling complex operations and privilege escalation techniques that are not possible through a non-interactive web shell. As with all backdoor methods, proper documentation and removal are mandatory during engagement cleanup to prevent ongoing risk exposure.

5.6 Compromising a Vulnerable Jenkins Server

A Jenkins server was identified with weak authentication controls and an enabled Groovy Script Console. The Groovy console allows execution of arbitrary operating system commands directly from the web interface.

By leveraging Groovy's `.execute()` method, commands such as `ipconfig /all` were run on the underlying host. This functionality effectively acts as a built-in non-interactive web shell. Similar to the Tomcat compromise, this access could be escalated using additional techniques, such as the Sticky Keys method, to achieve interactive system control.

This scenario illustrates how administrative web applications with script execution capabilities can introduce critical remote code execution (RCE) risks when improperly secured.

Chapter 5 Figures

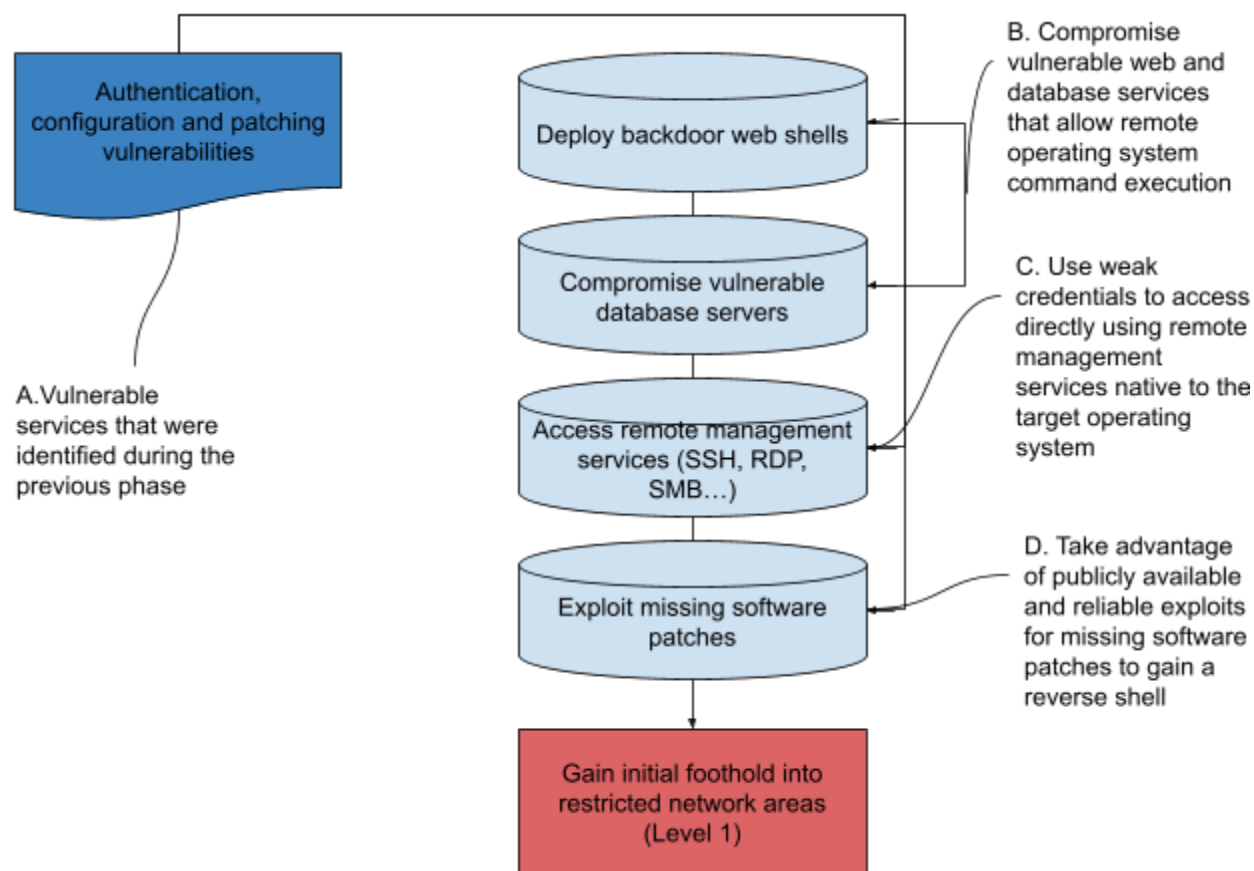


Figure 5.1 Phase 2: focused-penetration workflow

From *The Art of Network Penetration Testing* (Chapter 6), by R. Davis, 2020, Manning Publications

This figure shows the Phase 2 focused-penetration workflow. It demonstrates how previously identified vulnerabilities, such as weak credentials, exposed services, or missing patches are exploited to gain an initial Level 1 foothold inside the network.

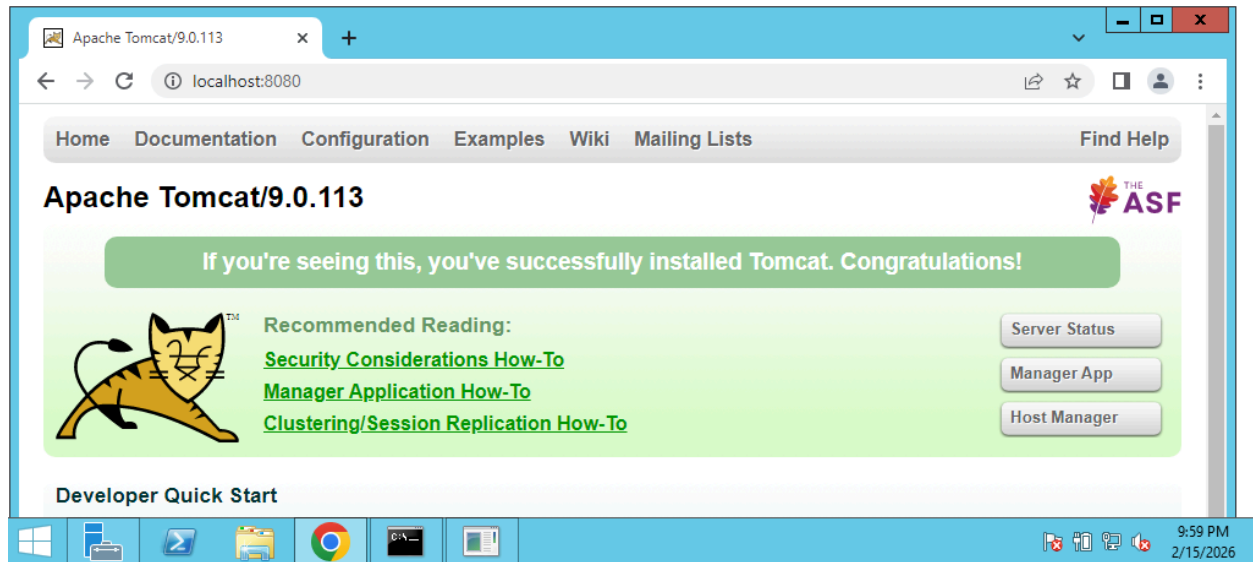
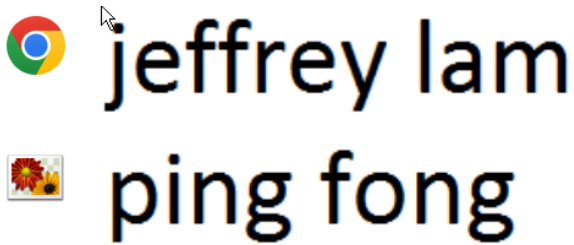


Figure 5.2 An Apache Tomcat server listening on port 8080

This screenshot shows the default Apache Tomcat 9.0.113 landing page accessed at localhost:8080, confirming that the Tomcat web server has been successfully installed and is running. It also provides links to documentation and management interfaces such as the Manager App and Host Manager.

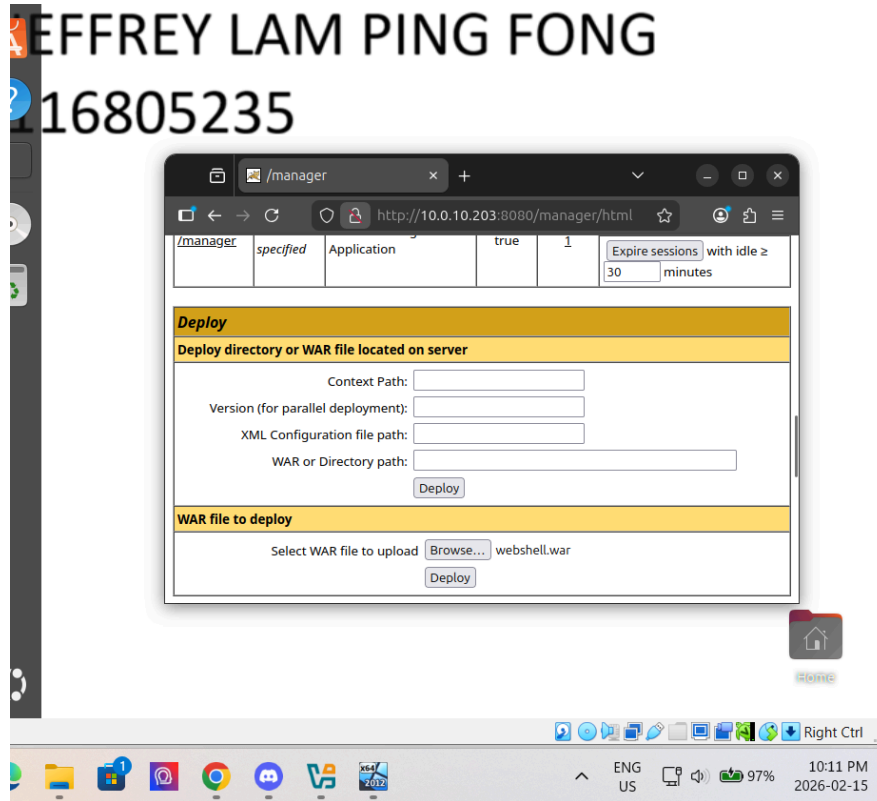


Figure 5.3 The WAR file Deploy section of the Tomcat manager page

This screenshot shows the Apache Tomcat Manager interface at /manager/html, specifically the WAR file deployment section. It allows a user to upload or deploy a WAR file to the server, which can install and run a new web application on Tomcat.

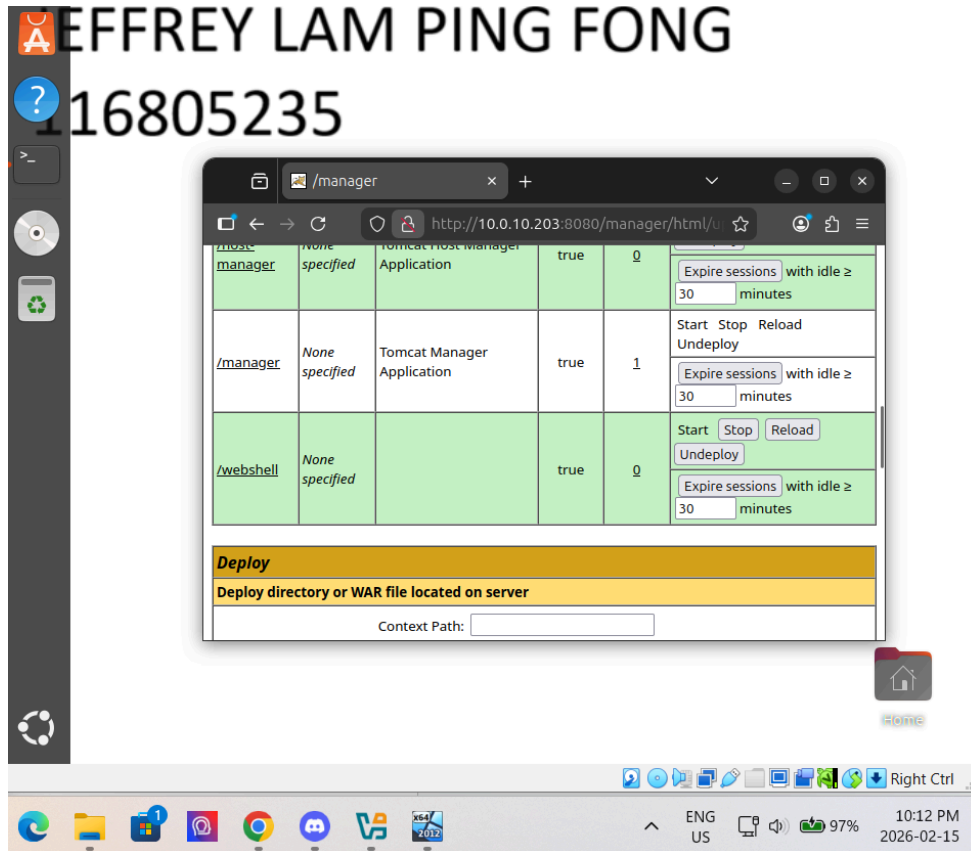


Figure 5.4 The webshell is deployed and is now accessible from the menu.

This screenshot shows the Tomcat Manager page listing deployed applications, including a newly deployed `/webshell` application. The presence of `/webshell` in the application list indicates that the WAR file was successfully deployed and is now accessible from the server menu.

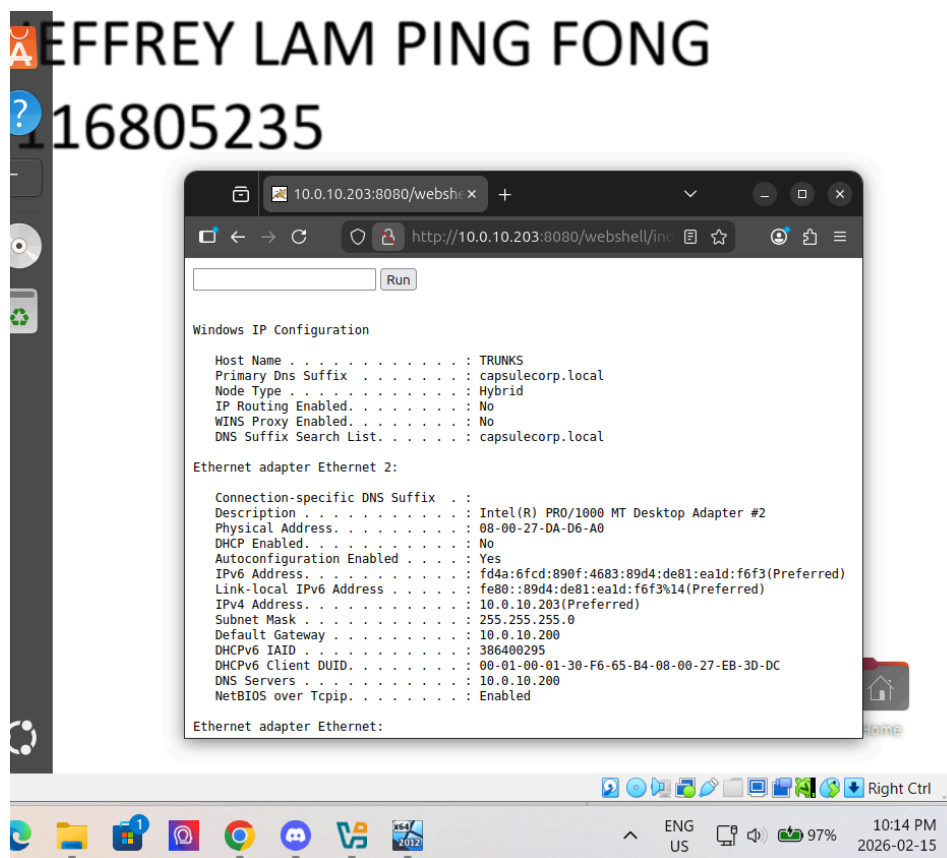


Figure 5.5 Running OS commands with the web shell

This screenshot shows the deployed /webshell application being used to execute operating system commands on the server. The displayed Windows IP configuration output confirms that the web shell is successfully running commands remotely on the host system.

jeffrey lam
ping fong

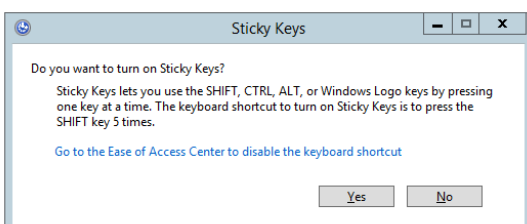


Figure 5.6 The Sticky Keys prompt after pressing Shift five times

This screenshot shows the Windows Sticky Keys prompt that appears after pressing the Shift key five times. It asks whether the user wants to enable Sticky Keys, an accessibility feature that allows modifier keys to be used one at a time.

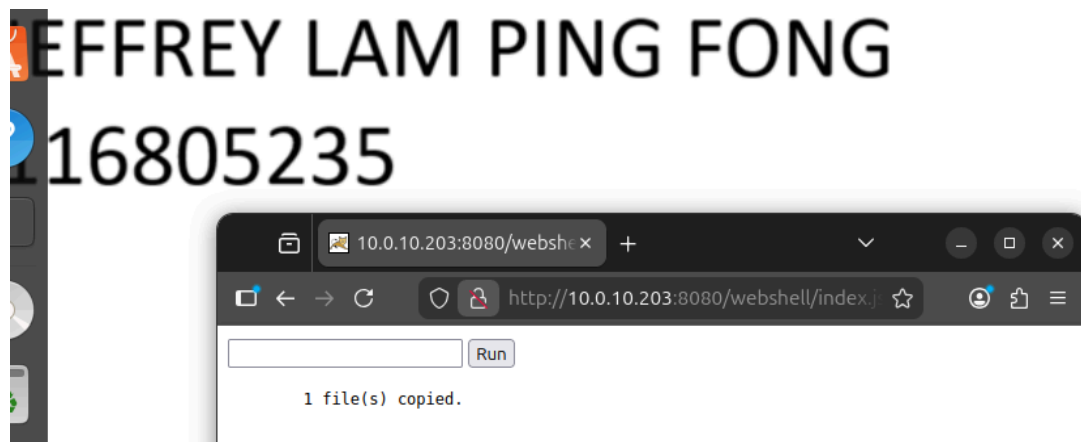


Figure 5.7 Result after issuing the sethc.exe backup command

This screenshot shows the web shell confirming that the command `cmd.exe /c copy c:\windows\system32\sethc.exe c:\windows\system32\sethc.exe.backup` was executed successfully. The message "1 file(s) copied." indicates that the original Sticky Keys executable (sethc.exe) was backed up in the System32 directory.

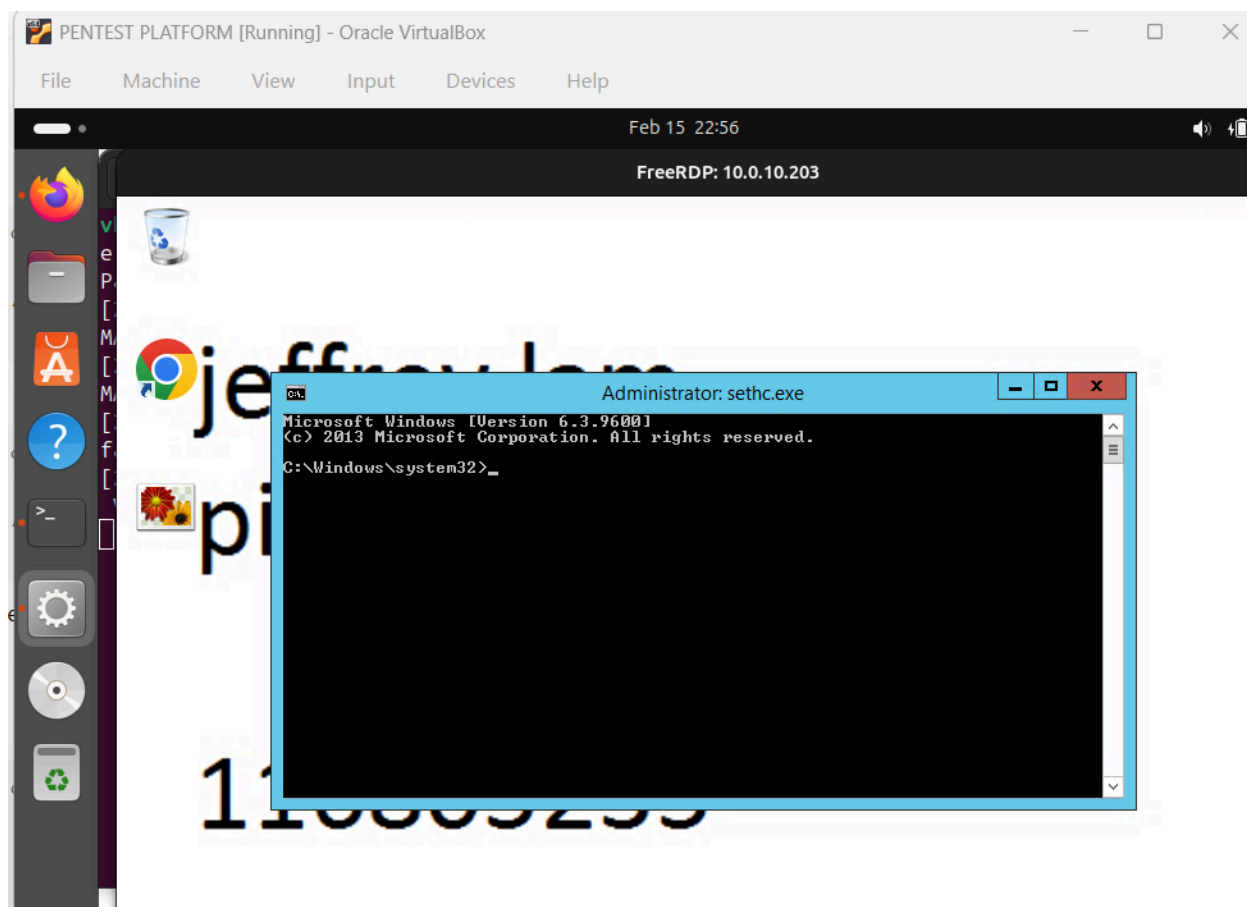


Figure 5.8 SYSTEM-level command prompt instead of Sticky Keys

Figure 5.8 shows a SYSTEM-level command prompt opening in place of the Sticky Keys accessibility feature after pressing Shift five times. It indicates that the Sticky Keys executable was replaced, allowing elevated command execution from the login screen.

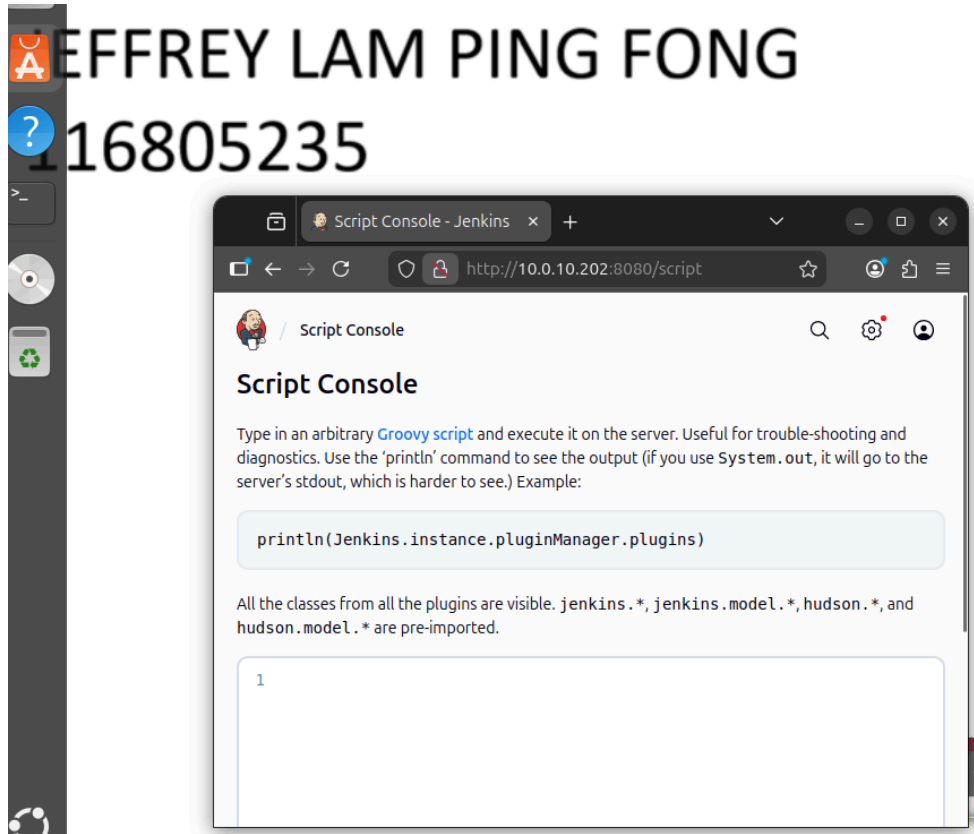


Figure 5.9 The Jenkins Groovy Script Console page

It shows the Jenkins Script Console page, which allows administrators to execute arbitrary Groovy scripts directly on the Jenkins server. It provides powerful access for troubleshooting and automation, as scripts run with the same privileges as the Jenkins service.

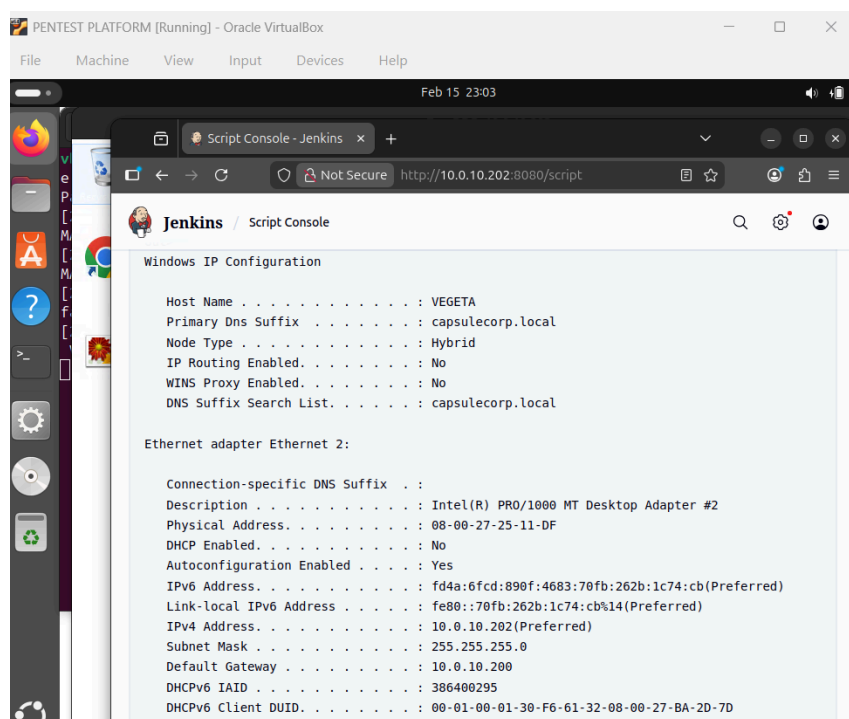


Figure 5.10 Executing OS commands using Groovy Script

The Jenkins Script Console is used to execute operating system commands through a Groovy script. The displayed Windows IP configuration output confirms that system-level commands were successfully run from within Jenkins.

Exercise 5.1

Please refer to Figures 5.2 through 5.5 for a visual demonstration of the deployment process and resulting system access.

Exercise 5.1 demonstrates the security risk associated with improperly secured Apache Tomcat management interfaces. The objective was to validate whether a Tomcat server with weak or default credentials could be leveraged to deploy a custom web application archive (WAR) file and obtain command execution on the underlying host.

In this exercise, a simple JSP-based web application was packaged into a WAR file and deployed through the Tomcat Manager interface. Once deployed, the application became accessible via a browser and allowed execution of operating system commands, returning the output to the web interface. This provided non-interactive shell access to the server and confirmed successful compromise of the host.

The exercise highlights the critical impact of exposed administrative interfaces, weak authentication controls, and unrestricted deployment functionality. It reinforces the importance of securing application management portals, disabling default credentials, and restricting administrative access to trusted networks only.

Summary of Chapter 6

Chapter 6 shifts the focused-penetration workflow from vulnerable web services to vulnerable database services, using Microsoft SQL Server (MSSQL) as the primary example. The chapter shows how weak database credentials can provide a reliable entry point into a Windows host, especially when web applications and databases are deployed together and credentials are reused or stored insecurely. Once access is obtained, the emphasis moves from database enumeration to host-level control by enabling OS command execution through MSSQL features, then leveraging that control to collect high-value post-exploitation artifacts—specifically Windows password hashes. The chapter also highlights the importance of documenting configuration changes and removing artifacts (e.g., temporary shares and copied hive files) during cleanup.

6.1 Compromising Microsoft SQL Server

This section demonstrates how valid MSSQL credentials can be used to gain access to a database server and pivot toward host-level compromise. After verifying a successful login, the key technique is enabling and using the `xp_cmdshell` stored procedure to execute operating system commands through SQL. Because `xp_cmdshell` is typically disabled by default, the chapter explains how an administrative database account can enable advanced options and activate `xp_cmdshell` using standard configuration commands via an MSSQL client.

Once enabled, `xp_cmdshell` effectively provides a non-interactive command execution channel on the underlying Windows system. The section emphasizes validating execution context (e.g., confirming which service account is running MSSQL) and determining privilege level, since service accounts may hold local administrator rights that significantly increase the impact of the compromise.

6.2 Stealing Windows Account Password Hashes

This section introduces password hash harvesting as a key post-compromise objective, especially when local administrator access has been achieved. It explains how Windows authentication relies on password hashes stored in system-protected locations (notably the SAM registry hive for local accounts) and why attackers value hashes: they can be reused for authentication in certain scenarios without needing to recover the plaintext password.

Operationally, the chapter outlines how to obtain the required registry hive data by using built-in Windows tooling to create accessible copies of protected hive files. It then describes a practical exfiltration approach: adjusting permissions on the copied files and creating a temporary anonymous network share to download the hive copies to the attacker's system for offline processing. The section reinforces that these steps create detectable artifacts and must be logged and removed during cleanup.

6.3 Extracting Password Hashes with Creddump

This section focuses on extracting password hashes from the downloaded SYSTEM and SAM hive copies using the Creddump framework. The workflow is intentionally performed off-host to reduce the need to upload suspicious binaries to the victim system and to avoid triggering endpoint security controls. Using Creddump's `pwdump.py`, the copied hive files are parsed to recover account hash data in a structured format.

The section also explains how to interpret the output, including the key fields returned for each account (username, RID, and hash values). The primary value of this step is that the extracted hashes can later support credential-based movement techniques and broader compromise pathways, making hash collection a consistent objective for each successfully compromised level-one host.

Chapter 6 Figures

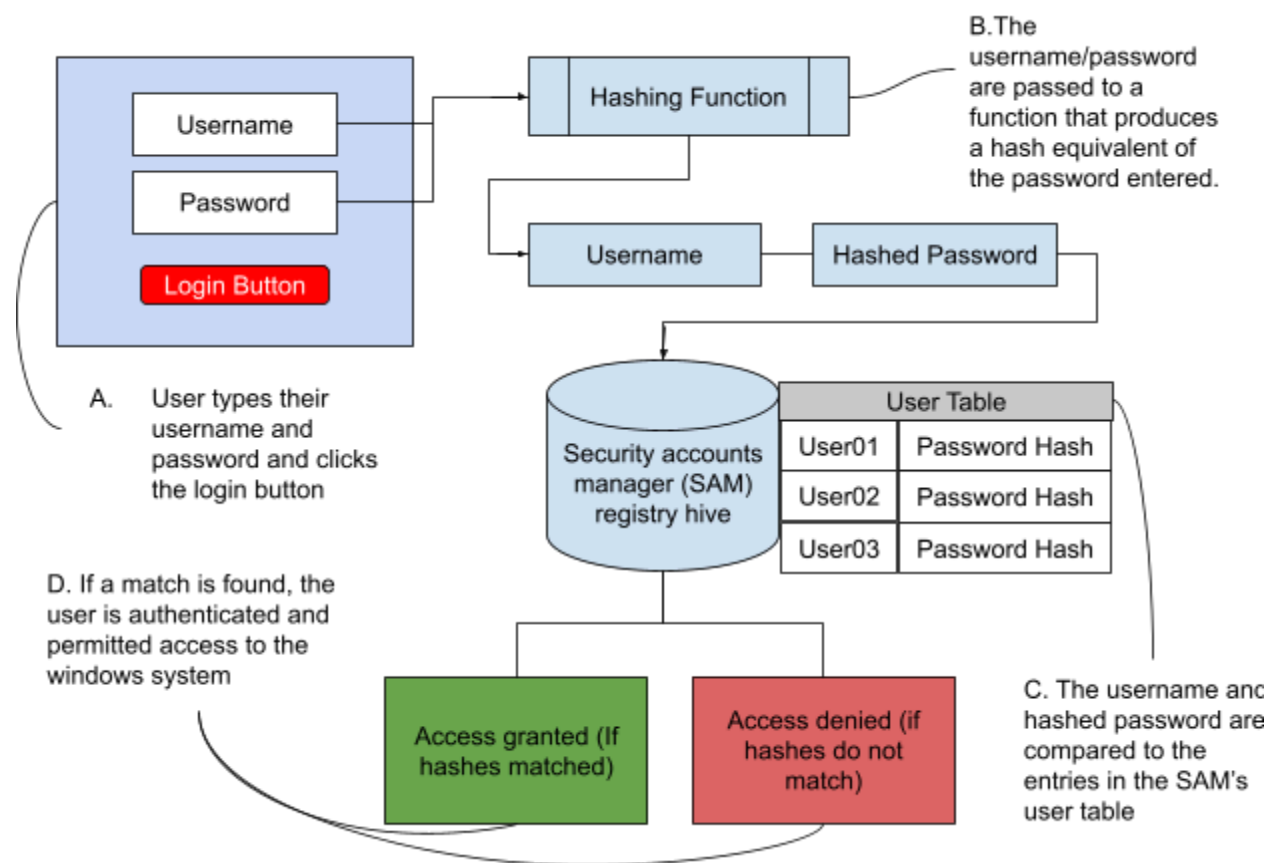


Figure 6.1 How Windows uses password hashes to authenticate users

From The Art of Network Penetration Testing (Chapter 6), by R. Davis, 2020, Manning Publications

This diagram shows how Windows authenticates a user using password hashing and the SAM database. When a user enters their username and password, the password is converted into a hash and compared to the stored password hash in the SAM user table. If the hashes match, access is granted; if they do not match, access is denied.

Exercise 6.1

During assessment of the Gohan server (10.0.10.201), the MSSQL database service was found to be using weak credentials for the sa account. Successful authentication via sqlcmd provided database console access with elevated privileges.

The xp_cmdshell stored procedure was enabled, allowing operating system command execution through the database interface. The MSSQL service was running under the NT SERVICE\MSSQLSERVER account with local administrator privileges, providing unrestricted system access.

Using the native reg.exe utility, copies of the SAM and SYSTEM registry hives were extracted to C:\windows\temp. Due to share creation restrictions, files were exfiltrated using Impacket's SMB server. The pwdump.py tool successfully extracted NTLM password hashes for local user accounts from the registry hives.

References

Davis, Royce. The Art of Network Penetration Testing. Manning Publications, 2020.

Lab 4

Jeffrey Lam Ping Fong 116805235

RIS602NBB

Mark Shtern

4th March 2026

Table of Contents

Executive Summary.....	5
Summary of Chapter 7.....	6
7.1 Understanding software exploits.....	6
7.2 Understanding the typical exploit life cycle.....	6
7.3 Compromising MS17-010 with Metasploit.....	7
7.4 The Meterpreter shell payload.....	7
7.5 Cautions about the public exploit database.....	7
Exercise 7.1.....	8
Summary of Chapter 8.....	9
8.1 Fundamental post-exploitation objectives.....	9
8.2 Maintaining reliable re-entry with Meterpreter.....	9
8.3 Harvesting credentials with Mimikatz.....	10
8.4 Harvesting domain cached credentials.....	10
8.5 Harvesting credentials from the filesystem.....	10
8.6 Moving laterally with Pass-the-Hash.....	11
Exercise 8.1.....	13
References.....	13

Table of Figures

Chapter 7 Figures.....	8
Figure 7.1 Shellcode section of exploit 47468.....	8
Chapter 8 Figures.....	12
Figure 8.1 Post-exploitation workflow.....	12

Checklist

ID	What {Figure, Table, Section ...}	Where (Reference)	Completion (%)
1	Summary of Chapter 7	Page 6	100%
2	7.1 Understanding software exploits	Page 6	100%
3	7.2 Understanding the typical exploit life cycle	Page 6	100%
4	7.3 Compromising MS17-010 with Metasploit	Page 7	100%
5	7.4 The Meterpreter shell payload	Page 7	100%
6	7.5 Cautions about the public exploit database	Page 7	100%
7	Figure 7.1	Page 8	100%
8	Exercise 7.1	Page 8	100%
9	Summary of Chapter 8	Page 9	100%
10	8.1 Fundamental post-exploitation objectives	Page 9	100%
11	8.2 Maintaining reliable re-entry with Meterpreter	Page 9	100%
12	8.3 Harvesting credentials with Mimikatz	Page 10	100%
13	8.4 Harvesting domain cached credentials	Page 10	100%
14	8.5 Harvesting credentials from the filesystem	Page 10	100%
15	8.6 Moving laterally with Pass-the-Hash	Page 11	100%
16	Figure 8.1	Page 12	100%

17	Exercise 8.1	Page 13	100%
----	--------------	---------	------

Executive Summary

Chapters 7 and 8 together provide a comprehensive walkthrough of two critical phases of an internal network penetration test: exploiting unpatched services and conducting post-exploitation activity on compromised Windows systems.

Chapter 7 establishes the foundational knowledge needed to understand how software vulnerabilities are discovered, disclosed, and eventually exploited. Using MS17-010 (EternalBlue) as the central case study, the chapter demonstrates how a single missing patch on a Windows system running the SMB service can give an attacker complete SYSTEM-level access with no user interaction required. Through Metasploit's ms17_010_psexec module, the process of verifying vulnerability, selecting a payload, and obtaining a reverse shell is shown to be straightforward and highly reliable. The chapter then introduces the Meterpreter shell as a significantly more capable alternative to a basic command prompt, offering built-in features for credential dumping, file transfer, process inspection, and post-exploitation modules. It closes with an important warning about the risks of using unverified exploits from public databases like exploit-db.com, stressing that shellcode from unknown sources should never be trusted without first being replaced with custom shellcode generated via msfvenom.

Chapter 8 continues where Chapter 7 left off, shifting focus to what an attacker does once they are inside a system. Three core objectives frame the entire chapter: maintaining reliable re-entry, harvesting credentials, and moving laterally. Persistent access is established through Meterpreter's autorun backdoor, ensuring the attacker retains their foothold even across reboots. Credential harvesting is explored through multiple avenues, extracting clear-text passwords from memory using Mimikatz, cracking domain cached credential hashes with John the Ripper and the rockyou.txt dictionary, and manually searching the filesystem for configuration files containing plain-text database credentials. Finally, the chapter demonstrates how the Pass-the-Hash technique allows an attacker to use NTLM hashes extracted from a single compromised host to authenticate to other systems on the network, turning one vulnerable machine into a gateway to many. The practical exercises throughout both chapters reinforce how quickly a single unpatched service can snowball into a full network compromise, underscoring the importance of robust patch management, strong and unique local administrator passwords, and credential hygiene across enterprise environments.

Summary of Chapter 7

Chapter 7 focuses on how attackers can exploit unpatched services on a network, using the infamous MS17-010 (EternalBlue) vulnerability as the primary case study. It walks through how software exploits are discovered and developed, how to verify a target is vulnerable, and how to use Metasploit to gain remote access to a compromised system. The chapter also introduces the Meterpreter shell as a more powerful alternative to a basic command prompt, and closes with a cautionary look at using exploits sourced from public databases like exploit-db.com.

7.1 Understanding software exploits

A software exploit takes advantage of a bug in a piece of code that causes an application to behave unexpectedly when it receives unusual input. If this bug causes the program to crash, an attacker may be able to hijack the execution flow and run their own machine-level instructions on the affected system. Writing a small program to do this is known as exploit development or software exploitation. While this chapter doesn't go deep into how exploits are built from scratch, it covers the key concepts needed to understand how publicly available exploits can be used during an internal network penetration test (INPT).

7.2 Understanding the typical exploit life cycle

The typical exploit life cycle begins when an independent security researcher discovers an exploitable bug in commercial software through reverse engineering or fuzzing. Importantly, hackers don't create these bugs, they find them. The researcher then responsibly discloses the vulnerability to the vendor (in this case, Microsoft), who verifies it and releases a patch. Once the patch is public, the vulnerable code can be reverse engineered by others, and working exploits tend to surface shortly after. MS17-010, also known as EternalBlue, followed this exact process and was patched in 2017. It remains one of the most well-known and exploited Windows vulnerabilities in history.

7.3 Compromising MS17-010 with Metasploit

MS17-010 targets the Windows SMB service, which listens on TCP port 445 by default and requires no user interaction to exploit, making it particularly dangerous. Before running the exploit, the auxiliary scanner module in Metasploit can be used to confirm whether a target is likely missing the patch. Once confirmed, the `ms17_010_psexec` exploit module is loaded, the target IP (`rhost`), local attacking IP (`lhost`), and desired payload are set, and the exploit is launched. A successful run results in a SYSTEM-level shell on the target machine. The chapter uses a reverse TCP shell payload to demonstrate this, where the victim machine connects back to the attacker rather than the attacker connecting in, this is the preferred approach because it gives the attacker more control and is more reliable in practice.

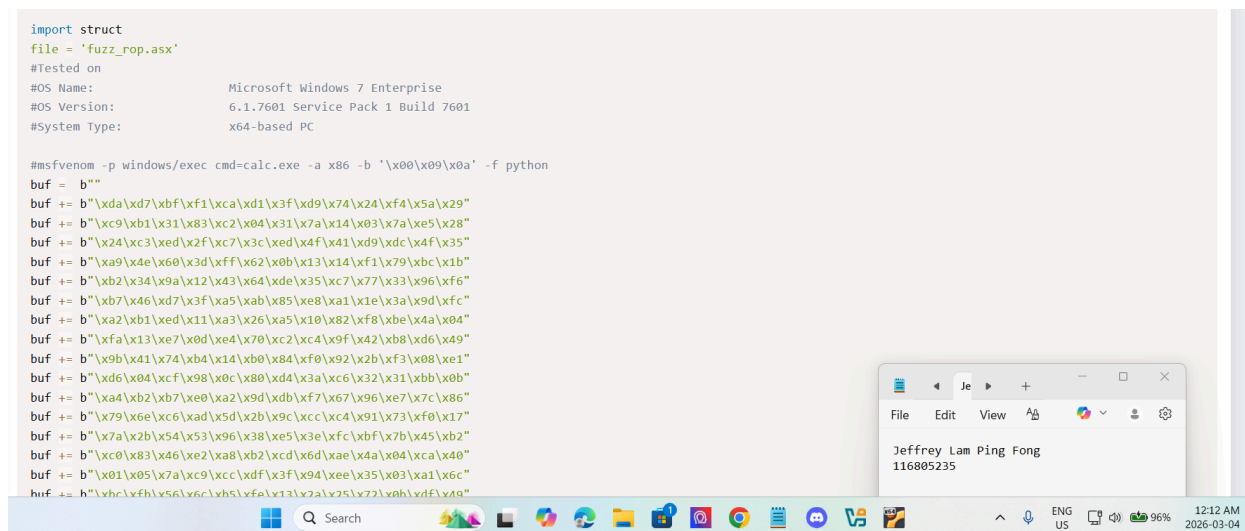
7.4 The Meterpreter shell payload

Rather than using a basic Windows command prompt, Metasploit offers a far more powerful payload called the Meterpreter shell. It supports file upload and download, post-exploitation modules, process listing, credential dumping, and much more, all without needing to drop additional tools onto the target. The chapter demonstrates swapping the standard reverse TCP shell for a `windows/x64/meterpreter/reverse_https` payload. Once inside a Meterpreter session, useful commands include `ps` to list running processes (which can reveal logged-in domain users), `shell` to drop into a Windows command prompt, and `run post/windows/gather/smart_hashdump` to extract local account password hashes. Meterpreter is highlighted as one of the most versatile and practical tools available to a pentester.

7.5 Cautions about the public exploit database

Exploit-db.com hosts thousands of proof-of-concept exploits, but unlike Metasploit modules, these are not quality-controlled or reliability-tested. Some may contain broken or even malicious shellcode. The chapter strongly advises against using anything from exploit-db.com unless the researcher is confident enough to read and understand the source code. If an exploit from the site must be used, the shellcode section should always be replaced with custom shellcode generated using `msfvenom`. The chapter demonstrates how to generate shellcode with `msfvenom` using a specified payload, `LHOST`, and `LPORT`, and then swap it into the exploit script. This section is presented as an introduction to the concept rather than standard INPT practice, since most pentesters rely on Metasploit's trusted framework during time-constrained engagements.

Chapter 7 Figures



```
import struct
file = 'fuzz_rop.asx'
#Tested on
#OS Name: Microsoft Windows 7 Enterprise
#OS Version: 6.1.7601 Service Pack 1 Build 7601
#System Type: x64-based PC

#msfvenom -p windows/exec cmd=calc.exe -a x86 -b '\x00\x09\x0a' -f python
buf = b""
buf += b"\xda\xd7\xbf\xf1\xca\xd1\x3f\xd9\x74\x24\xf4\x5a\x29"
buf += b"\xc9\xb1\x31\x83\xc2\x04\x31\x7a\x14\x03\x7a\xe5\x28"
buf += b"\x24\xc3\xed\x2f\xc7\x3c\xed\x4f\x41\xd9\xdc\x4f\x35"
buf += b"\xa9\x4e\x60\x3d\xff\x62\x0b\x13\x14\xf1\x79\xbc\x1b"
buf += b"\xb2\x34\x9a\x12\x43\x64\xde\x35\xc7\x77\x33\x96\xf6"
buf += b"\xb7\x46\xd7\x3f\xa5\xab\x85\xe8\xa1\xe3\x9d\xfc"
buf += b"\xa2\xb1\xed\x11\xa3\x26\xa5\x10\x82\xf8\xbe\x4a\x04"
buf += b"\xfa\x13\xe7\x0d\xe4\x70\xc2\xc4\x9f\x42\xb8\xd6\x49"
buf += b"\x9b\x41\x74\xb4\x14\xb0\x84\xf0\x92\x2b\xf3\x08\xe1"
buf += b"\xd6\x04\xcf\x98\x0c\x80\xd4\x3a\xc6\x32\x31\xb5\x0b"
buf += b"\xa4\xb2\xb7\xe0\xa2\x9d\xdb\xf7\x67\x96\xe7\x7c\x86"
buf += b"\x79\x6e\xc6\xad\x5d\x2b\x9c\xcc\xc4\x91\x73\xf0\x17"
buf += b"\x7a\x2b\x54\x53\x96\x38\xe5\x3e\xfc\xbf\x7b\x45\xb2"
buf += b"\xc0\x83\x46\xe2\xa8\xb2\xcd\x6d\xae\x4a\x04\xca\x40"
buf += b"\x01\x05\x7a\xc9\xcc\xdf\x3f\x94\xee\x35\x03\xa1\x6c"
buf += b"\xb8\xfb\x56\x6c\xb5\xfa\x13\x2a\x25\x22\x0b\xdf\x49"
```

Figure 7.1 Shellcode section of exploit 47468

Figure 7.1 shows the shellcode section of exploit 47468, written in Python. The script imports the struct module and specifies the target file fuzz_rop.asx, which is a Windows Media Player playlist file used as the attack vector. The highlighted lines contain a long sequence of hexadecimal byte values that represent shellcode, which is machine-level code executed after the exploit successfully triggers the vulnerability

Exercise 7.1

Exercise 7.1 puts the theory from the chapter into practice by tasking the student with identifying and exploiting the MS17-010 vulnerability on the tien.capsulecorp.local system. Using the windows.txt host file created during earlier reconnaissance, the student sweeps for targets missing the MS17-010 patch and confirms that tien is vulnerable. The ms17_010_eternalblue exploit module is then used alongside a meterpreter/reverse_tcp payload to exploit the system and obtain a remote shell. Once inside, the student is tasked with navigating to tien's desktop folder and retrieving the contents of a file called flag.txt, reinforcing the real-world impact of an unpatched service, an attacker with this level of access can read, modify, or exfiltrate any data on the system.

Summary of Chapter 8

Chapter 8 covers what happens after an attacker has successfully compromised a system, a phase known as post-exploitation. Using Windows as the focus, the chapter outlines the three core objectives of post-exploitation: maintaining reliable re-entry, harvesting credentials, and moving laterally to other systems. Tools like Meterpreter, Mimikatz, John the Ripper, and CrackMapExec are all used throughout the chapter to achieve these goals. The chapter builds on the access gained in Chapter 7 and shows how a single compromised system can quickly snowball into full network takeover.

8.1 Fundamental post-exploitation objectives

Once a system has been compromised, post-exploitation activities fall into three broad categories. The first is maintaining reliable re-entry, which ensures the attacker can get back into the system even if the session drops or the system reboots. The second is harvesting credentials, including local account hashes, domain cached credentials, and clear-text passwords stored in configuration files. The third is moving laterally, which involves using credentials obtained from level-one targets to gain access to additional systems that had no directly exploitable vulnerabilities, these are called level-two targets. Understanding this distinction matters when reporting findings to clients, as it helps frame the severity of a single unpatched system relative to the wider network impact.

8.2 Maintaining reliable re-entry with Meterpreter

To ensure continued access to a compromised host, Metasploit's persistence script can be used to install a Meterpreter backdoor that automatically runs every time the system boots. The run persistence command places a VBScript file on the target system and registers it as an autorun entry in the Windows registry. Arguments specify where the file is written, how often it attempts to reconnect, which port to use, and the IP address to connect back to. Each time the system reboots, a new Meterpreter session opens automatically on the attacker's machine. Critically, the cleanup file generated by this process must be used at the end of the engagement to remove the backdoor and restore the system to its original state. Leaving persistence mechanisms behind after a pentest is a serious professional failure.

8.3 Harvesting credentials with Mimikatz

A flaw in the design of the Windows Local Security Authority Subsystem Service (LSASS) means that when a user logs in, their clear-text password is stored in virtual memory. Mimikatz, developed by Benjamin Delpy, is a tool that exploits this flaw to extract those plain-text credentials directly from memory. Within a Meterpreter session, the Mimikatz extension can be loaded with `load mimikatz`. Commands like `tspkg` and `wdigest` can then be used to retrieve clear-text credentials for any users who have recently logged in to the system. In the chapter's example, the domain credentials for the user `CAPSULECORP\ten` are successfully extracted in plain text. This technique only works when there is an active or recent user session, so results will vary depending on the target system.

8.4 Harvesting domain cached credentials

Windows systems store a limited number of hashed domain credentials locally to allow users to log in when they're not connected to the corporate network, a feature called domain cached credentials. These hashes use a different hashing function (mscache2 on modern systems) and are stored in the Windows SECURITY registry hive. The Metasploit post module `post/windows/gather/cachedump` can extract these from a Meterpreter session. While these hashes can't be used in Pass-the-Hash attacks due to how Windows remote authentication works, they can be cracked offline using a tool like John the Ripper. Using the popular `rockyou.txt` dictionary file, which contains over 14 million real-world passwords from previous data breaches, the chapter demonstrates cracking the `ten` account password (`Password82$`) in just over eight minutes.

8.5 Harvesting credentials from the filesystem

Beyond memory-based credential extraction, clear-text credentials can often be found sitting in configuration files on the filesystem. Web servers in particular almost always connect to backend databases, meaning configuration files like `web.config`, `tomcat-users.xml`, and `config.inc.php` frequently contain database usernames and passwords stored in plain text. Windows command-line tools make searching for these straightforward, the `findstr` command can search the contents of every file on the system for strings like `"password="`, while the `where` command can be used to locate specific files by name. User home directories are also worth investigating manually, as people commonly store passwords in text documents or Word files. While this process can be tedious, it pays off often enough that it should always be part of the post-exploitation workflow.

8.6 Moving laterally with Pass-the-Hash

Windows authentication allows users to authenticate using the NTLM hash of their password rather than the plain-text password itself. This means that if a local administrator hash is extracted from one compromised system, it can potentially be used to log in to other systems on the network that share the same local administrator credentials, a technique known as Pass-the-Hash. Using either the `smb_login` module in Metasploit or CrackMapExec with the `-H` flag, the extracted NTLM hash can be tested against all Windows hosts on the network. In the chapter's example, the administrator hash from TIEN successfully authenticates to RADITZ, a system that had no directly exploitable vulnerabilities, making it a level-two target. From there, the same post-exploitation process repeats, potentially unlocking access to even more systems and illustrating just how quickly a single missed patch can lead to widespread network compromise.

Chapter 8 Figures

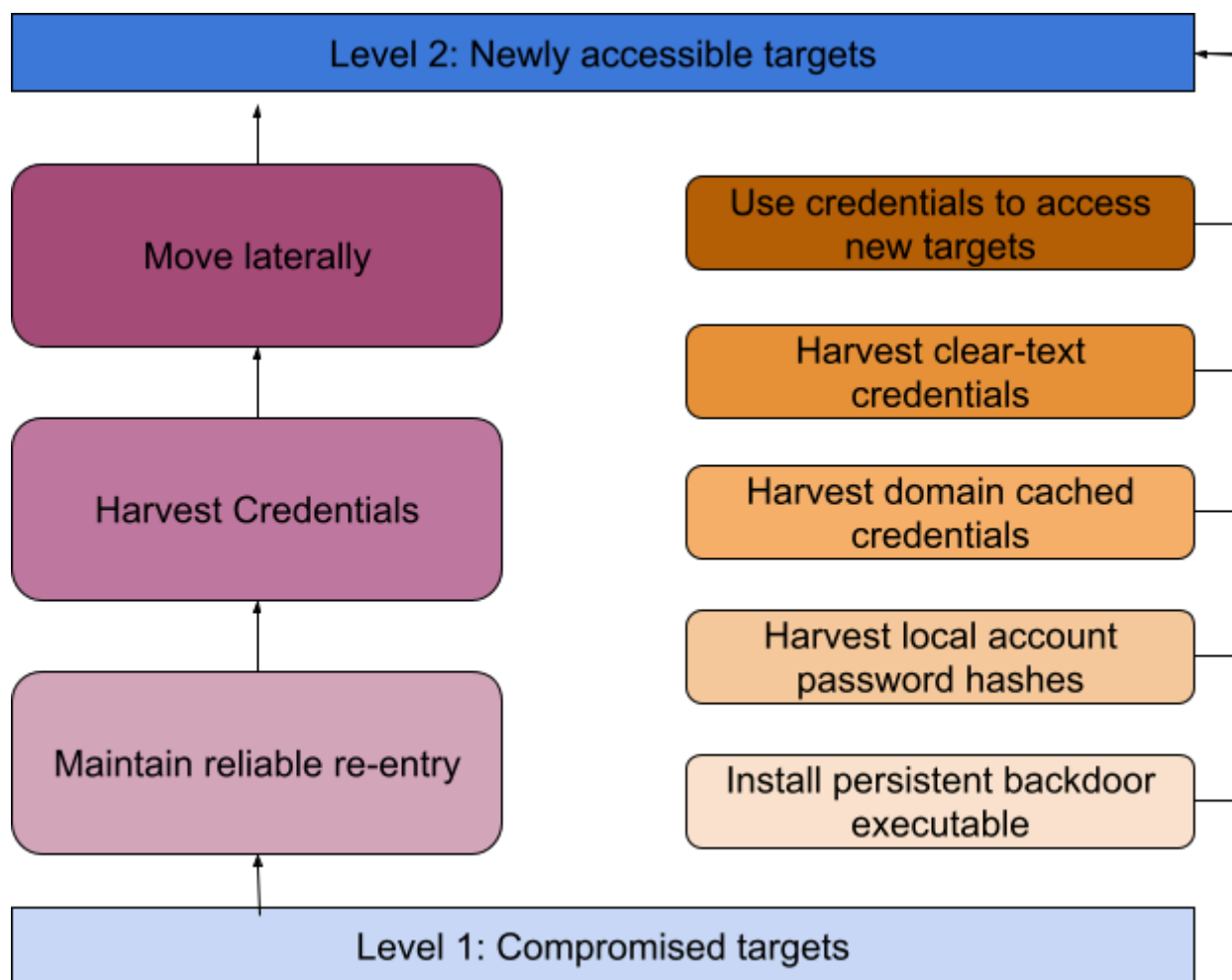


Figure 8.1 Post-exploitation workflow

From *The Art of Network Penetration Testing* (Chapter 8), by R. Davis, 2020, Manning Publications

Figure 8.1 illustrates the three-stage post-exploitation workflow that follows the compromise of a Level 1 target. Starting from the bottom, the attacker first installs a persistent backdoor to maintain reliable re-entry if the session is lost, then harvests credentials through methods such as password hash dumping, cached domain credentials, and clear-text configuration files, before finally using those credentials to move laterally across the network and unlock previously inaccessible Level 2 targets. The diagram highlights how each stage feeds directly into the next, showing how a single compromised machine can quickly escalate into a much wider network breach.

Exercise 8.1

Exercise 8.1 builds directly on the access gained in the previous exercise by using the local account password hashes extracted from `tien.capsulecorp.local` to perform a Pass-the-Hash attack. The student tests these hashes across the Windows hosts on the network using either Metasploit's `smb_login` module or CrackMapExec and discovers that the RADITZ system shares the same local administrator credentials as TIEN, despite having no known directly exploitable vulnerabilities. This makes RADITZ a level-two target, accessible only because of the credentials harvested from a level-one compromise. The student is then tasked with retrieving the contents of `c:\flag.txt` on the RADITZ server, demonstrating how lateral movement can expand an attacker's foothold far beyond the initial point of entry.

References

Davis, Royce. *The Art of Network Penetration Testing*. Manning Publications, 2020.

Lab 06

Jeffrey Lam Ping Fong
116805235

RIS602 NBB
30th March 2026

Table of Contents

Executive Summary.....	5
Chapter 11 Summary.....	6
11.1 Killing Active Shell Connections.....	6
11.2 Deactivating Local User Accounts.....	6
11.3 Removing Leftover Files from the Filesystem.....	6
11.4 Reversing Configuration Changes.....	7
11.5 Closing Backdoors.....	7
Exercise 11.1.....	9
Chapter 12 Summary.....	10
12.1 Eight Components of a Solid Pentest Deliverable.....	10
12.2 Executive Summary.....	11
12.3 Engagement Methodology.....	11
12.4 Attack Narrative.....	11
12.5 Technical Observations.....	11
12.6 Appendices.....	12
12.7 Wrapping It Up.....	12
12.8 What Now?.....	12
Exercise 12.1.....	14
Executive Summary.....	14
Engagement Methodology.....	14
Attack Narrative.....	15
Technical Observations.....	16
Appendix 1: Severity Definitions.....	19
Appendix 2: Hosts and Services.....	20
Appendix 3: Tools List.....	21
Appendix 4: Additional References.....	21
References.....	22

Table of Figures

Chapter 11 Figures.....	8
Figure 11.1 - Click Undeploy to undeploy webshell.....	8
Figure 11.2 - Confirmation that webshell is undeployed.....	8
Figure 11.3 Confirming that the WAR file has been undeployed.....	9
Figure 11.4 Confirming that Sticky Keys works properly.....	9
Chapter 12 Figures.....	13
Figure 12.1 The eight components of a solid pentest deliverable.....	13

Checklist

ID	What {Figure, Table, Section ...}	Where (Reference)	Completion (%)
1	Chapter 11 Summary	6	100%
2	11.1 Killing active shell connections	6	100%
3	11.2 Deactivating local user accounts	6	100%
4	11.3 Removing leftover files from the filesystem	6	100%
5	11.4 Reversing configuration changes	7	100%
6	11.5 Closing backdoors	7	100%
7	Figure 11.1	8	100%
8	Figure 11.2	8	100%
9	Figure 11.3	9	100%
10	Figure 11.4	9	100%
11	Exercise 11.1	9	100%
12	Chapter 12 Summary	10	100%
13	12.1 Eight components of a solid pentest deliverable	10	100%
14	12.2 Executive summary	11	100%
15	12.3 Engagement methodology	11	100%
16	12.4 Attack narrative	11	100%
17	12.5 Technical observations	11	100%
18	12.6 Appendices	12	100%
19	12.7 Wrapping it up	12	100%
20	12.8 What now?	12	100%

21	Figure 12.1	13	100%
22	Exercise 12.1	14	100%

Executive Summary

Chapters 11 and 12 of *The Art of Network Penetration Testing* cover the final two stages of an internal network penetration test: post-engagement cleanup and writing the deliverable. These chapters close out the engagement lifecycle by ensuring the client's environment is returned to its original state and that the results are documented in a professional, actionable report.

Chapter 11 focuses on post-engagement cleanup, which is organized into five categories: killing active shell connections, deactivating user accounts, removing leftover files, reversing configuration changes, and closing backdoors. Each category addresses specific artifacts left behind during the Capsulecorp pentest, including Meterpreter sessions, a backdoor entry in `/etc/passwd`, registry hive and `ntds.dit` copies, the `xp_cmdshell` stored procedure, an anonymous file share, a `crontab` entry, a deployed WAR file, the Sticky Keys backdoor, and a persistent Meterpreter autorun. The chapter emphasizes that thorough engagement notes are essential for tracking everything that needs to be cleaned up, and that failing to do so is both unprofessional and potentially harmful to the client.

Chapter 12 covers the pentest deliverable, which is the only tangible work product the client receives after the engagement. The report is broken into eight components: the executive summary, engagement methodology, attack narrative, technical observations, and four appendices covering severity definitions, hosts and services, a tool list, and additional references. The executive summary provides a high-level overview for non-technical stakeholders. The methodology section explains the four-phase testing approach and the type of attacker being emulated. The attack narrative tells the story of how the tester went from no access to full domain compromise. The technical observations section contains the detailed findings, each with a severity rating, description, impact statement, evidence, affected assets, and remediation recommendation. The appendices provide supporting information that rounds out the report.

Together, these two chapters ensure that a penetration test is completed responsibly and that the results are communicated clearly to both technical and non-technical audiences.

Chapter 11 Summary

Chapter 11 covers the post-engagement cleanup process that should be carried out after completing an internal network penetration test. The goal is to return the client's environment to the same state it was in before testing began. Cleanup targets fall into five categories: active shell connections, user accounts, leftover files, configuration changes, and backdoors. Proper cleanup depends on maintaining detailed notes throughout the engagement, as these notes serve as the checklist for identifying everything that needs to be addressed. Failing to clean up is unprofessional and can leave the client more vulnerable than they were before the engagement started.

11.1 Killing Active Shell Connections

Any Meterpreter or other shell sessions opened during the engagement need to be terminated. In the Capsulecorp pentest, two Meterpreter sessions were established, one through exploiting an unpatched Windows host and another through accessing a system with a logged-in domain admin. These are killed using the sessions -K command in msfconsole, and verified with sessions -l. If sessions persist, the console can be hard-exited with exit -y. Any active listeners for persistent callbacks should also be stopped using jobs -k.

11.2 Deactivating Local User Accounts

Any user accounts created during the engagement must be removed or deactivated. In the Capsulecorp pentest, no new accounts were explicitly created, but a pentest entry was added to the /etc/passwd file on a compromised Linux server, effectively creating a backdoor root account. Cleanup involves SSHing into the server, opening /etc/passwd in a text editor, deleting the pentest line, and verifying removal by attempting to switch to that user with su pentest which should return an error confirming the entry no longer exists.

11.3 Removing Leftover Files from the Filesystem

Files placed on compromised systems during testing such as registry hive copies, SSH keys, scripts, and ntds.dit extracts need to be deleted. Specific cleanup actions from the Capsulecorp pentest include deleting the SAM and SYSTEM hive copies from c:\windows\temp on a Windows target, removing the SSH key pair and callback.sh script from the compromised Linux server, and deleting the ntds.dit and SYSTEM copies from the domain controller's C: drive. On Windows, files should also be permanently removed from the Recycle Bin to prevent recovery. Good engagement notes are essential here, as you need to know exactly which files were placed on which hosts.

11.4 Reversing Configuration Changes

Any system or application configurations modified during the pentest must be reverted. Three changes were made during the Capsulecorp engagement. First, the xp_cmdshell stored procedure on the MSSQL server was enabled and needs to be disabled by setting its value back to 0 using sp_configure, followed by a reconfigure command. Second, an anonymous file share named "pentest" was created on a Windows server and must be removed using net share pentest /delete. Third, a crontab entry was added on a Linux server to run a callback script every five minutes, which is removed with crontab -r and verified with crontab -l.

11.5 Closing Backdoors

Backdoors installed to maintain persistent access must be fully removed. Three backdoors were created during the Capsulecorp pentest. The malicious WAR file deployed to the Apache Tomcat server is removed by clicking Undeploy in the Tomcat web management interface, then confirming the application returns a 404. The Sticky Keys backdoor, created by replacing sethc.exe with cmd.exe, is reversed by copying the original sethc.exe.backup file back over the modified binary, then verifying that pressing Shift five times launches the Sticky Keys dialog instead of a command prompt. The persistent Meterpreter autorun backdoor is cleaned up using the resource file Metasploit generated in the ~/.msf4/logs/persistence folder, which contains the commands to delete the .vbs script and remove the associated registry key from the Run hive.

Chapter 11 Figures

Applications					
Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/webshell	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Figure 11.1 - Click Undeploy to undeploy webshell

After logging in to the Tomcat web management interface, next to /webshell is a button “undeploy”



Figure 11.2 - Confirmation that webshell is undeployed

After I undeployed the webshell I got a confirmation message.



Figure 11.3 Confirming that the WAR file has been undeployed

Once the WAR file has been undeployed, we can confirm it by using the browser and browsing to the application

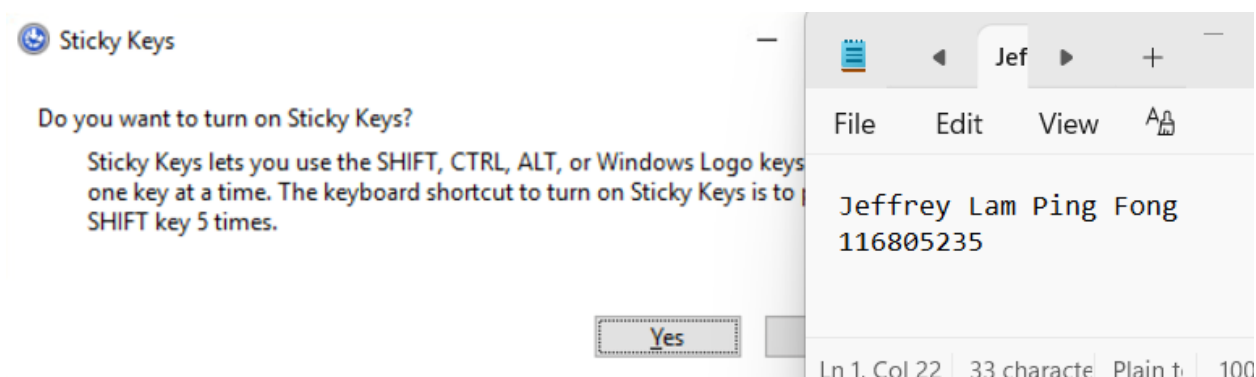


Figure 11.4 Confirming that Sticky Keys works properly

After checking to see if the backdoor was removed, click shift 5 times and the sticky keys prompt should show up.

Exercise 11.1

Post-engagement cleanup was performed across the Capsulecorp Pentest environment using engagement notes as a reference. All five categories of cleanup were addressed.

First, all active Meterpreter shell sessions were terminated using the sessions -K command in msfconsole, and verification with sessions -l confirmed no active sessions remained. Active listeners were also stopped using jobs -k.

Second, the pentest user entry added to the /etc/passwd file on the compromised Linux server was removed. The file was opened in a text editor, the backdoor line was deleted, and removal was verified by attempting su pentest, which returned an error confirming the entry no longer exists.

Third, all leftover files were removed from compromised hosts. This included deleting the SAM and SYSTEM registry hive copies from c:\windows\temp on gohan, removing the SSH key pair and callback.sh script from the compromised Linux server, and deleting the ntds.dit and SYSTEM copies from the C: drive on the domain controller

goku. On Windows systems, deleted files were also permanently removed from the Recycle Bin.

Fourth, all configuration changes were reversed. The xp_cmdshell stored procedure on the MSSQL server was disabled by setting its value to 0 with sp_configure and running reconfigure. The anonymous file share named "pentest" was removed using net share pentest /delete. The crontab entry that ran the callback script every five minutes was removed with crontab -r.

Finally, all backdoors were closed. The malicious WAR file was undeployed from the Apache Tomcat management interface and confirmed removed by browsing to the application and receiving a 404 response. The Sticky Keys backdoor was reversed by restoring the original sethc.exe.backup file over the modified binary, and pressing Shift five times confirmed the proper Sticky Keys dialog appeared. The persistent Meterpreter autorun backdoor was cleaned up by deleting the .vbs script and removing the associated registry key from HKLM\Software\Microsoft\Windows\CurrentVersion\Run, using the commands found in the Metasploit-generated resource file.

The environment was returned to the state it was in before the engagement began.

Chapter 12 Summary

Chapter 12 covers the final phase of an internal network penetration test: writing the deliverable. The deliverable is the only tangible work product a client receives after the engagement, so it needs to effectively communicate what was tested, what was found, and what should be done about it. The chapter walks through eight components that should appear in every solid pentest report, regardless of the company producing it. While style, branding, and formatting will vary between organizations, the core structure remains consistent across the industry.

12.1 Eight Components of a Solid Pentest Deliverable

A complete pentest deliverable contains eight sections: the executive summary, engagement methodology, attack narrative, technical observations (findings), and four appendices covering severity definitions, hosts and services, a tool list, and additional references. These sections move from high-level business context down into technical detail, with the appendices providing supplemental information that supports the main

body of the report. The executive summary answers the who, what, where, and when, while the remaining sections focus on the how.

12.2 Executive Summary

The executive summary is a one- to two-page overview of the entire engagement, written for a non-technical audience. It should be able to stand on its own as a document that could be handed to an executive with no additional context. At minimum, it covers the goals and objectives of the engagement, the dates and times testing took place, the scope of systems tested, and the high-level results including whether the test was successful and what the recommended next steps are. The deliverable should always be provided as a PDF, never as an editable document, to preserve the integrity of the report.

12.3 Engagement Methodology

This section explains how the testing was conducted, what type of attacker was being emulated, and what level of information was provided up front (white box, grey box, or black box). It sets expectations with the reader and establishes a shared language for the rest of the report. The methodology section is also valuable in cases where a "clean report" has to be delivered with no findings, as it demonstrates the breadth of testing activities that were attempted. A template-based approach works well here, listing the four-phase methodology and bullet points for each testing activity, so the section does not need to be rewritten from scratch after every engagement.

12.4 Attack Narrative

The attack narrative tells the story of the engagement in a linear, step-by-step format. It describes how the tester went from initial network access to full compromise, outlining the systems that had to be attacked along the way. The narrative can be somewhat high-level since the engagement methodology section already defines the technical terms and processes used. Screenshots are optional. The key requirement is that the narrative clearly explains the path taken and why the tester was able to achieve the level of access they obtained.

12.5 Technical Observations

Technical observations, more commonly called findings, are the core of the deliverable and the section most clients turn to first. Each finding should include seven elements: a severity rating, a descriptive title, a detailed observation, an impact statement that communicates the business risk, evidence such as screenshots or command output, a

list of affected assets, and a recommendation for remediation. Findings should map directly to the authentication, configuration, and patching vulnerabilities identified during testing. Only vulnerabilities that were actually used to compromise a target should be included; best-practice deficiencies that were not exploited belong in a vulnerability assessment, not a pentest report. Recommendations should address the security issue without prescribing specific vendor products, and should instead point clients toward enforcing strong passwords, applying hardening standards, or patching to the latest version.

12.6 Appendices

Four appendices round out the deliverable. The severity definitions appendix provides objective, concrete criteria for each rating level (such as critical, high, medium, low), which helps prevent subjective disputes over finding severity. The hosts and services appendix is a table of all discovered IP addresses, hostnames, and listening ports and services, sometimes provided as a supplemental spreadsheet for large environments. The tools list is a simple bulleted list of every tool used during the engagement with links to each tool's website or repository. The additional references appendix provides links to external hardening guides and security standards from organizations like NIST, CIS, and OWASP, offering the client further reading beyond the scope of the engagement.

12.7 Wrapping It Up

Once the deliverable is complete, a close-out meeting is typically held with the client's key stakeholders to walk through the findings and answer technical questions. For consultants, the engagement largely ends after this meeting. For internal pentesters, the work continues well beyond the report, as they are often involved in remediation discussions, follow-up meetings, and presentations for months afterward. The chapter notes that a common career path is to spend several years consulting before transitioning to an internal security role, though some prefer the variety and independence that consulting offers.

12.8 What Now?

The chapter closes by encouraging continued growth through hands-on practice. The most effective way to improve is by completing real engagements and learning to research unfamiliar systems on the fly. For those without upcoming engagements, the author provides a list of resources including training platforms like Hack The Box and Offensive Security, bug bounty programs like HackerOne and Bugcrowd, and recommended books covering web application hacking, Metasploit, and general penetration testing techniques. The key takeaway is that pentesting is a process, not

magic, and the ability to search for answers and adapt to new technologies is the most important skill a tester can develop.

Chapter 12 Figures

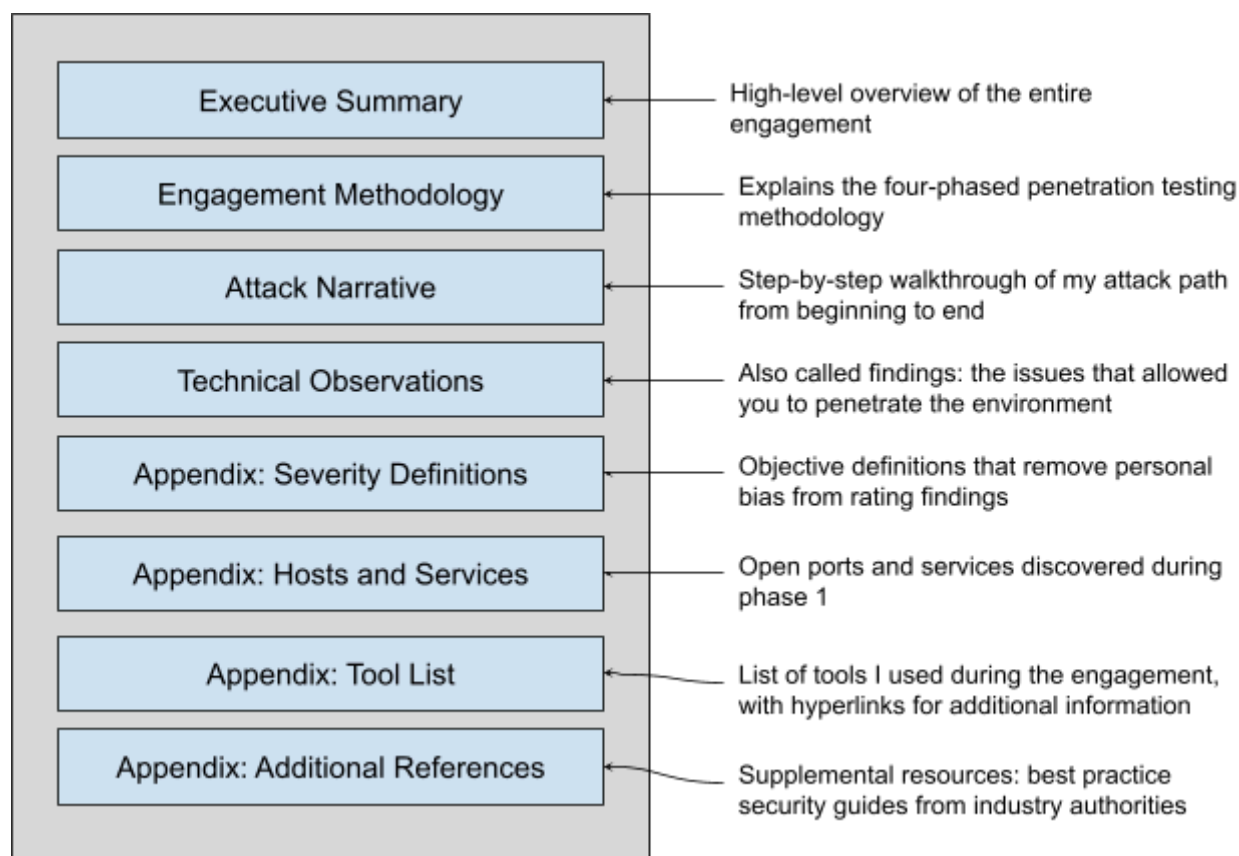


Figure 12.1 The eight components of a solid pentest deliverable

From *The Art of Network Penetration Testing* (Chapter 12), by R. Davis, 2020, Manning Publications

This diagram shows the eight sections of a pentest report in their typical order, from the high-level Executive Summary down through the Engagement Methodology, Attack Narrative, and Technical Observations, followed by four appendices covering severity definitions, hosts and services, tools used, and additional security references

Exercise 12.1

Executive Summary

Seneca Polytechnic (SP) was engaged by Capsulecorp, Inc. (CC) to conduct an Internal Network Penetration Test (INPT) against CC's corporate IT infrastructure. The purpose of the engagement was to evaluate the security posture of CC's internal network and determine its susceptibility to known attack vectors. Testing was conducted from CC's corporate headquarters at 123 Sesame Street. Engagement activities began on March 26, 2026, and concluded on March 30, 2026. This document represents a point-in-time summary of the technical results observed by SP during the testing window.

CC provided the IP address range 10.0.10.0/24 and the Active Directory domain capsulecorp.local. SP was authorized to perform blind host discovery and treat all enumerable hosts as in-scope.

During the engagement, SP identified multiple security deficiencies that allowed direct compromise of CC assets. SP exploited missing operating system patches, default or easily guessable credentials, and insecure application configurations to compromise production systems. Through shared credentials on compromised hosts, SP was able to escalate access and ultimately obtain full domain admin-level control of the CAPSULECORP.local Active Directory domain. If a real attacker with malicious intent obtained this level of access, the resulting business impact could be potentially catastrophic.

SP recommends the following actions to strengthen CC's security posture: improve operating system patching procedures, enhance system hardening policies, ensure all hosts and services use complex and unique passwords, and limit the use of shared credentials across systems.

Engagement Methodology

SP used a four-phase methodology modeled after real-world attacker behavior observed in modern corporate environments. The methodology assumes the attacker has no upfront knowledge of the network and no access beyond physically plugging a device into CC's network. This emulates an external attacker who gains physical entry under false pretenses, as well as a malicious insider, vendor, or custodial worker with physical access to the building.

Phase 1 -- Information Gathering: Starting with only a list of IP address ranges, SP performed host discovery sweeps using open source tools to identify live targets. Identified hosts were then scanned for listening network services, which represent the attack surface. Each service was analyzed for default credentials, missing security updates, and insecure configuration settings.

Phase 2 -- Focused Penetration: Identified weaknesses were exploited in a controlled manner designed to minimize disruption to production services. No denial-of-service attacks were used. Once access was obtained, SP searched for credentials stored in sensitive locations such as text files, application configuration files, and operating system credential stores like Windows registry hive files.

Phase 3 -- Post-Exploitation and Privilege Escalation: Credentials obtained from compromised systems were tested against additional hosts to expand network reach. The goal was to identify and impersonate critical users with unrestricted access. SP simulated persistent re-entry on select compromised hosts and accessed production domain controllers to obtain hashed credentials from the ntds.dit database using non-destructive methods.

Phase 4 -- Documentation and Cleanup: All compromises were logged and evidenced with screenshots for the final deliverable. Post-engagement cleanup ensured CC systems were returned to their pre-engagement state. Files created during testing were securely destroyed, and all non-destructive configuration changes were reversed.

Attack Narrative

SP began the engagement with no knowledge beyond the provided scope and no access beyond an unused data port in an unoccupied conference room. Host and service discovery was performed using Nmap to establish a list of network targets and enumerate their attack surface. Identified services were organized into protocol-specific target lists, against which SP attempted vulnerability discovery focusing on low-hanging-fruit attack vectors commonly exploited in real-world breaches.

SP identified three targets susceptible to compromise due to insufficient patching, weak or default credentials, and insecure configurations. The compromised hosts were `tien.capsulecorp.local`, `gohan.capsulecorp.local`, and `trunks.capsulecorp.local`, all compromised using freely available open source tools.

After gaining access to these systems, SP used credentials obtained from them to access additional hosts sharing the same credentials. This led to access on `raditz.capsulecorp.local`, which had a privileged domain admin account logged in at the time of testing. SP used Mimikatz to extract the clear-text credentials for `serveradmin@capsulecorp.local` from that machine. With this account, it was trivial to access the domain controller `goku.capsulecorp.local` with unrestricted administrator privileges, giving SP complete control over the CAPSULECORP.local Active Directory domain.

Technical Observations

Finding 1: Default Credentials Found on Apache Tomcat -- High

Observation: One Apache Tomcat server was identified as using a default password for the administrator account. SP was able to authenticate to the Tomcat web management interface and control the application through a web browser.

Impact: An attacker could deploy a custom WAR file to execute operating system commands on the underlying Windows server. In CC's environment, the Tomcat application was running with administrative privileges, meaning an attacker would have unrestricted access to the server.

Asset Affected: 10.0.10.203, `trunks.capsulecorp.local`

Recommendation: CC should change all default passwords and enforce strong password requirements for all accounts with access to Apache Tomcat. CC should consult its internal password policy or create one following industry standards if none exists. Additionally, CC should evaluate whether the Tomcat Manager web app is needed and disable it via configuration if no business need is present.

Reference:

https://wiki.owasp.org/index.php/Securing_tomcat#Securing_Manager_WebApp

Finding 2: Default Credentials Found on Jenkins -- High

Observation: One Jenkins server was identified as using a default password for the administrator account. SP was able to authenticate to the Jenkins web management interface and control the application through a web browser.

Impact: An attacker could execute arbitrary Groovy Script code to run operating system commands on the underlying Windows server. The Jenkins application was running with administrative privileges, granting unrestricted access to the server.

Asset Affected: 10.0.10.203, vegeta.capsulecorp.local

Recommendation: CC should change all default passwords and enforce strong password requirements for all accounts with access to Jenkins. CC should evaluate whether the Jenkins Script console is needed and disable it if no business requirement exists.

Finding 3: Default Credentials Found on Microsoft SQL Database -- High

Observation: One Microsoft SQL database server was identified as using a default password for the built-in sa administrator account. SP was able to authenticate with full administrative privileges.

Impact: An attacker could access the database to create, read, update, or delete confidential records. Additionally, the attacker could use the xp_cmdshell stored procedure to execute operating system commands on the underlying Windows server. The database was running with administrative privileges, giving an attacker unrestricted access.

Asset Affected: 10.0.10.201, gohan.capsulecorp.local

Recommendation: CC should enforce strong and complex passwords across all database user accounts. The database server should be reconfigured to run under a less privileged, non-administrative user account. CC should review Microsoft's "Securing SQL Server" documentation and ensure all security best practices are met.
Reference:

<https://docs.microsoft.com/en-us/sql/relational-databases/security/securing-sql-server>

Finding 4: Missing Microsoft Security Update MS17-010 -- High

Observation: One Windows server was missing a critical Microsoft security update. MS17-010 (EternalBlue) was absent from the affected host. SP used publicly available exploit code to compromise the system and gain control of the operating system.

Impact: An attacker could trivially exploit this vulnerability to gain system-level access on the target, allowing them to alter, copy, or destroy confidential information.

Asset Affected: 10.0.10.208, tien.capsulecorp.local

Recommendation: CC should investigate why this patch from 2017 was missing and ensure all corporate assets are properly up to date with the latest patches and security updates. Security updates should be tested in a pre-production staging environment before being applied to production systems.

Finding 5: Shared Local Administrator Account Credentials -- Medium

Observation: Two systems were identified as sharing the same password for the local administrator account.

Impact: An attacker who compromises one of these systems can trivially access the other due to the shared credentials. In CC's environment, SP used this shared access to ultimately gain complete control of the CAPSULECORP.local Active Directory domain.

Assets Affected: 10.0.10.208, tien.capsulecorp.local and 10.0.10.207, raditz.capsulecorp.local

Recommendation: CC should ensure that passwords are not shared across multiple user accounts or machines.

Appendix 1: Severity Definitions

Critical: A finding that poses a direct threat to business operations. Successful exploitation would have a potentially catastrophic impact on the organization's ability to function normally.

High: A finding that allows for a direct compromise of a system or application. An otherwise restricted area of the scoped environment could be accessed and used to alter confidential systems or data.

Medium: A finding that could potentially result in a direct compromise of a system or application. To exploit a medium finding, an attacker would need one additional piece of information, access, or another medium finding to achieve full compromise.

Low: A best practice deficiency rather than a direct risk. A low finding alone would not provide an attacker with a means to compromise targets but may provide information useful in another attack.

Appendix 2: Hosts and Services

The following hosts, ports, and services were enumerated during the engagement:

10.0.10.1 -- Ports 53 (domain), 80 (http)

10.0.10.125 -- Port 80 (http)

10.0.10.138 -- Port 80 (http)

10.0.10.151 -- Port 57143

10.0.10.188 -- Ports 22 (OpenSSH 7.6p1), 80 (Apache httpd 2.4.29)

10.0.10.200 (goku, domain controller) -- Ports 53, 88 (Kerberos), 135 (MSRPC), 139 (NetBIOS), 389 (LDAP), 445 (SMB), 464, 593, 636, 3268, 3269, 3389 (RDP), 5357, 5985, 9389, and multiple high-range MSRPC ports

10.0.10.201 (gohan) -- Ports 80, 135, 139, 445, 1433 (MSSQL 2014 SP3), 2383, 3389 (RDP), 5985, 47001, and multiple high-range MSRPC ports

10.0.10.202 (vegeta) -- Ports 80 (IIS 8.5), 135, 443, 445, 3389, 5985, 8080 (Jetty 9.4.z-SNAPSHOT), 49154

10.0.10.203 (trunks) -- Ports 80 (Apache 2.4.39), 135, 139, 443, 445, 3306 (MariaDB), 3389, 5985, 8009 (AJP13), 8080 (Tomcat), 47001, and multiple high-range MSRPC ports

10.0.10.204 -- Port 22 (OpenSSH 7.6p1)

10.0.10.205 -- Ports 135, 139, 445, 3389 (RDP), 5040, 5800 (TightVNC), 5900 (VNC), 49667

10.0.10.206 -- Ports 135, 139, 445, 3389 (RDP), 5040, 5800 (UltraVNC), 5900 (VNC), 49668

10.0.10.207 (raditz) -- Ports 25 (Exchange SMTP), 80 (IIS 10), 135, 139, 443 (IIS 10), 445, 587 (SMTP), 593, 808, 1801, 3389 (RDP), 5985, 6001-6406 (multiple MSRPC and RPC over HTTP ports), 47001, 64327 (Exchange log copier), and multiple additional MSRPC ports

10.0.10.208 (tien) -- Identified during testing, specific services not listed separately in scope table

Appendix 3: Tools List

Metasploit Framework -- <https://github.com/rapid7/metasploit-framework>

Nmap -- <https://nmap.org>

CrackMapExec -- <https://github.com/byt3bl33d3r/CrackMapExec>

John the Ripper -- <https://www.openwall.com/john>

Impacket -- <https://github.com/SecureAuthCorp/impacket>

Parsenmap -- <https://github.com/R3dy/parsenmap>

Ubuntu Linux -- <https://ubuntu.com>

Exploit-DB -- <https://www.exploit-db.com>

Mssql-cli -- <https://github.com/dbcli/mssql-cli>

Creddump -- <https://github.com/moyix/creddump>

Mimikatz -- <https://github.com/gentilkiwi/mimikatz>

Appendix 4: Additional References

Apache Tomcat:

<http://tomcat.apache.org/tomcat-9.0-doc/security-howto.html>

https://wiki.owasp.org/index.php/Securing_tomcat

Jenkins:

<https://www.jenkins.io/doc/book/system-administration/security/>

<https://www.pentestgeek.com/penetration-testing/hacking-jenkins-servers-with-no-pass-word>

Microsoft SQL Server:

<https://docs.microsoft.com/en-us/sql/relational-databases/security/securing-sql-server>

Active Directory:

docs.microsoft.com/..../best-practices-for-securing-active-directory

Ubuntu Linux:

<https://ubuntu.com/security>

References

Davis, Royce. The Art of Network Penetration Testing. Manning Publications, 2020.